Week 1 — From Physical State to CPU Control

Format: seven daily sections collected into one weekly module

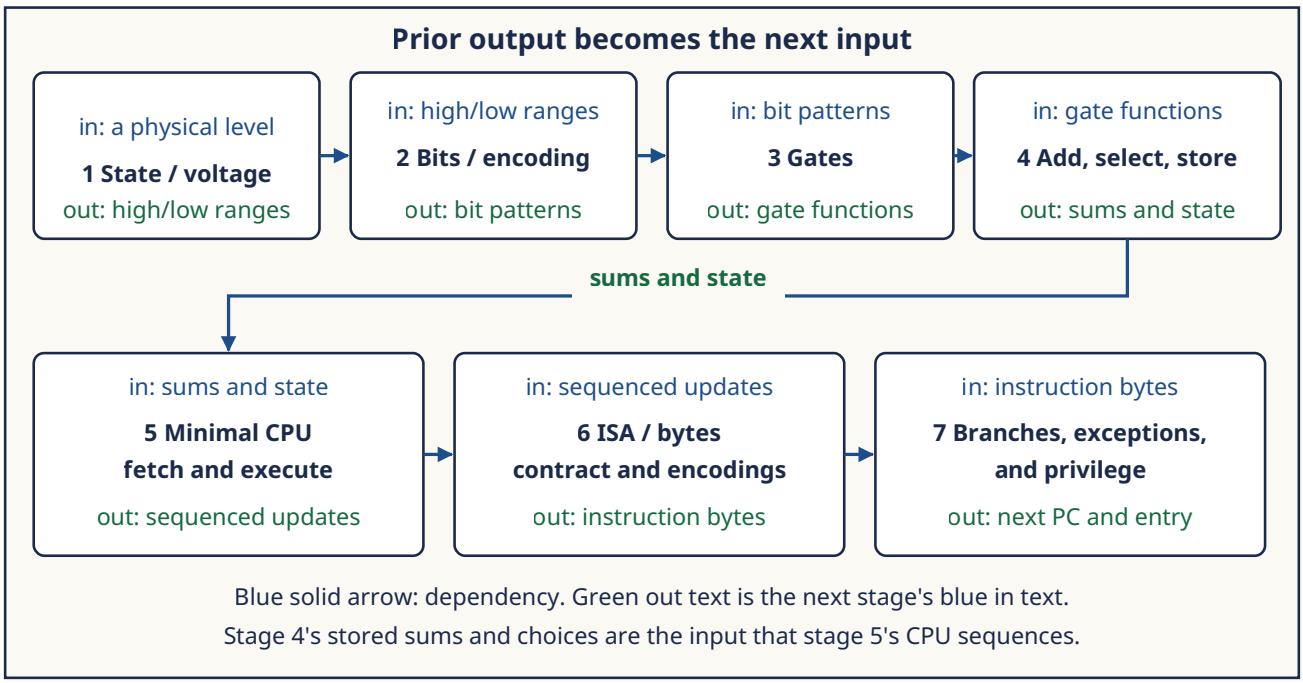
Suggested workload: approximately 21 focused hours, adjustable to mastery

> This week reconstructs the path from a physical state that can represent one bit to a processor that can fetch instructions, branch, cross privilege boundaries, and enter an operating-system handler.

Route through the curriculum: use Week 1 for the conceptual chain from physical state to a minimal CPU. Revisit the device physics and quantitative electrical behavior in Week 4, then follow a full instruction trace through an implemented processor in Week 5.

Why these seven days belong together

The modern CPU is easy to memorize as a block diagram and difficult to understand as a causal system. This week builds it in dependency order:



Each stage's output is the next stage's input, from voltage ranges to the bytes that later request a branch or a privileged entry.

Every section answers two questions:

1. What problem forced this abstraction to exist?
2. Where does it physically or operationally exist in a running computer?

Daily roadmap

Day 1 — Foundations: From State to Meaning

Physical state, voltage ranges, noise margin, binary patterns, positional notation, encodings, information, and the observation that identical bytes can be numbers, text, pixels, or instructions.

Day 2 — Signed Numbers and Bitwise Reasoning

Two's complement, modular arithmetic, masks, packed register fields, carry/overflow, extension, C promotion preview, and the bridge toward an ALU.

Day 3 — Boolean Logic and Physical Gates

Boole, Shannon, relays, vacuum tubes, transistors, truth tables, six common gates, De Morgan's laws, universal gates, combinational logic, and a half-adder preview.

Day 4 — Adders, Selection, and Memory

Half/full adders, ripple carry, muxes, decoders, encoders, feedback, latches, flip-flops, clocks, registers, and a full physical-to-logical trace of `3 + 5 = 8`.

Day 5 — Building a Minimal CPU

Architectural state, PC, IR, registers, ALU, instruction/data memory, datapath versus control, a tiny ISA, fetch/decode/execute, and cycle-visible simulation.

Day 6 — Instruction Sets and Machine Code

ISA versus microarchitecture, opcodes, operands, immediates, addressing, load/store design, careful RISC/CISC comparison, x86-64 and RISC-V examples, ELF disassembly, and fictional ISA design.

Day 7 — Control Flow, Exceptions, and Privilege

Flags, branches, loops, calls, user/kernel privilege, synchronous exceptions, asynchronous interrupts, vectoring, saved context, return from exception, Unix signals, and system-call entry.

Weekly evidence

Do not treat reading the combined PDF as completion. Collect evidence:

- **Explain:** reconstruct each day's central causal chain without notes.
- **Draw:** produce the physical-state-to-CPU diagram and the exception-flow diagram.
- **Observe:** run Linux disassembly, tracing, simulator, and safe exception experiments.
- **Build:** complete the register-mask lab, digital-circuit predictions, tiny CPU extensions, fictional ISA exercise, and machine-code lab.

Weekly mastery checkpoint

Starting with the physical voltage state of a bit, explain how a CPU can:

1. represent and add `3 + 5`;
2. preserve the result;
3. fetch and decode an instruction requesting the operation;
4. choose the next instruction through control flow;
5. stop ordinary user execution and safely enter an operating-system handler.

If any arrow requires “the computer just knows,” return to the corresponding daily section and make that mechanism explicit.

Day 1 — Foundations: From State to Meaning

Curriculum alignment: Stage 1 — Foundations of Computation · Electricity, States, Bits, and Information

Target time: approximately 3 hours

- Part I — physical state and reliable bits: about 50 minutes
- Part II — patterns, notation, and meaning: about 55 minutes
- Part III — Linux observation and integrated challenge: about 55 minutes
- Mastery explanation and review: about 20 minutes

> A computer can display a photograph, execute an instruction, add two numbers, and print a sentence. Yet at its foundation it has only physical things changing state. How can such a machine represent anything at all?

That is the problem for Day 1.

We will not begin by declaring that computers “speak binary.” That phrase hides every interesting engineering decision. We will begin lower down, with a real physical machine in a noisy world, and build the chain ourselves:

```
physical state
    → reliable logical alternatives
    → bit patterns
    → encodings
    → useful meaning
```

By the end, you should be able to explain why the same stored pattern can be a number, a character, an instruction, or a row of pixels—and why that fact is the necessary starting point for logic circuits and CPUs.

---

1. The old problem beneath the modern machine

Before electronics, calculation was work done by people. Fingers, tally marks, counting boards, and abaci helped because they moved part of the work out of a person's memory and into the state of a physical object. A bead on one side of a rod could preserve a partial result while the person considered the next step.

Mechanical calculators pushed the idea further. Wheels and gears could be arranged so that one complete rotation carried into the next position. The machine's geometry enforced part of the arithmetic rule. Later designs made storage, arithmetic, and control increasingly distinct mechanisms.

The ambition was already recognizable: preserve facts, transform them by repeatable rules, and avoid asking a human to perform every intermediate step.

But the medium imposed limits. Precision mechanisms were expensive, slow, subject to wear, and difficult to scale. Adding more interacting states meant adding more mechanical complexity.

Punched media solved a different part of the problem. A loom could use holes in cards to select weaving operations. Telegraph codes could turn characters into patterns that traveled through a narrow communication channel. Hollerith's census machinery encoded facts by the positions of holes in cards. When a reader pin passed through a hole, it completed an electrical path; where there was no hole, it did not.

Notice the causal progression:

```
human memory is limited
    → put state into a durable physical object

manual transformation is slow
    → build mechanisms whose structure enforces a rule

facts are too varied for one fixed mechanism
    → encode them as reusable patterns

mechanical motion is slow and difficult to scale
    → use electrical switching
```

No single person or machine suddenly invented the modern computer. Many developments solved different constraints: representation, switching, storage, arithmetic, control, and programmability. Today we establish the representation layer on which the rest depends.

---

2. Computation starts with physical state

A useful first model of computation is:

```
current physical state + input + rule → new physical state
```

An abacus changes bead positions. A mechanical calculator changes gear positions. A relay machine changes which contacts are open or closed. A modern processor changes charge and voltage across networks of transistors.

The materials differ, but computation always needs real state. A value that can affect later events cannot be stored “nowhere.” It must be represented by some physical condition:

- the position of an object,
- an open or closed electrical contact,
- voltage on a conductor,
- charge in a capacitor,
- magnetic orientation,
- or another distinguishable physical state.

Software is not exempt from physics. A variable in C is an abstraction, but while the program runs, its value is represented in registers, caches, or memory by physical states. A file is an abstraction, but its data ultimately depends on states in storage devices.

What exactly is a state?

A **state** is the condition of a system at a particular moment.

Consider a simple switch:

```
physical position:      DOWN       UP
assigned symbol:          0         1
```

The positions are physical. The symbols are our chosen labels. Nature does not insist that DOWN means zero; we could reverse the mapping and the switch would behave exactly as before.

This distinction will remain important throughout systems engineering:

```
what physically exists  ≠  what our convention says it means
```

For a state to be useful in a computer, it should be:

1. distinguishable from alternatives,
2. stable long enough to be used,
3. intentionally changeable,
4. readable with acceptably low ambiguity.

Those demands—not a mystical preference for zero and one—lead us toward digital electronics.

---

3. Electricity is continuous; digital logic is engineered

A common shortcut says:

```
0 volts = 0
3.3 volts = 1
```

That is a useful cartoon, but it is not how a real input specification works. Voltage is continuous. A wire may be at 0.00 V, 0.19 V, 1.42 V, 2.87 V, or any of countless values between. Real circuits also face:

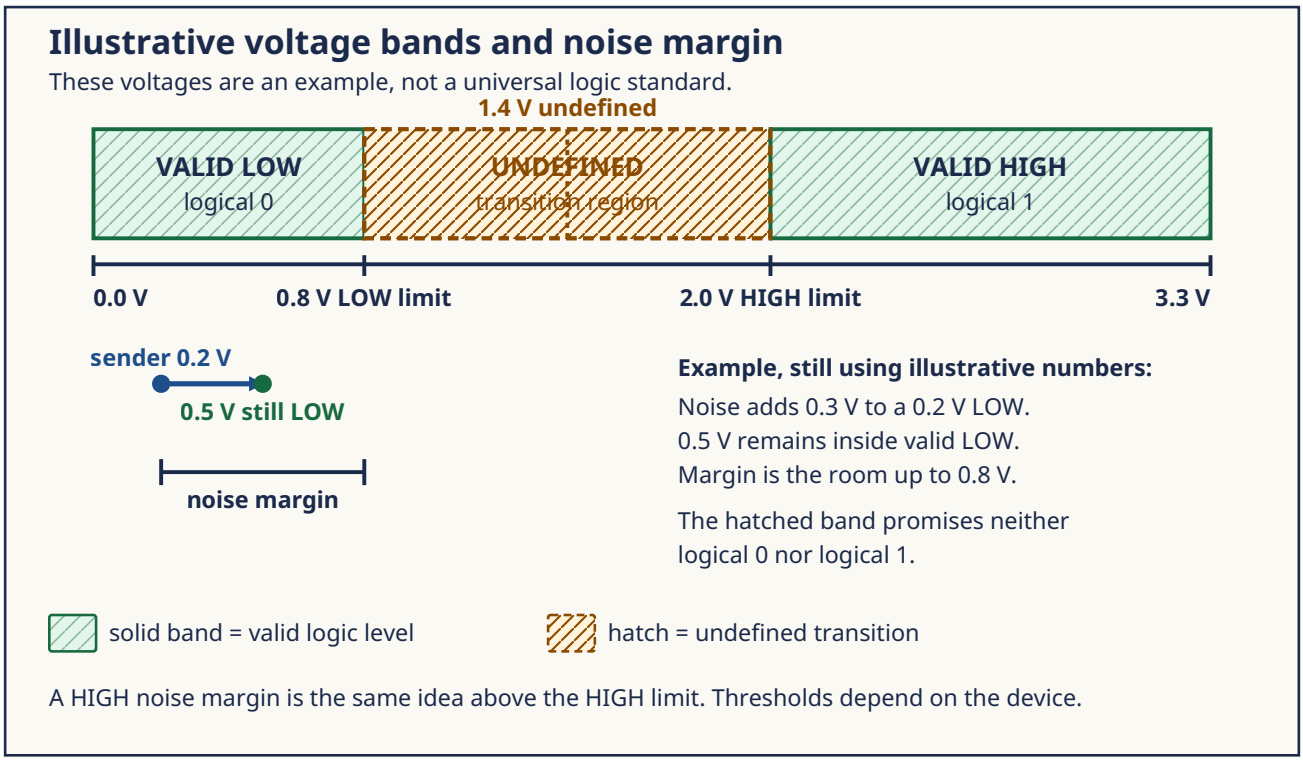
- electrical noise,
- coupling from nearby signals,
- resistance and voltage drop,
- power-supply variation,
- manufacturing differences,
- and changing temperature.

A quick electronics bridge: charge, current, and capacitance

Voltage describes an electrical potential difference; **current is the rate at which charge flows**. A circuit node also has capacitance, so changing its voltage means changing stored charge. In the simple capacitor model, `Q = CV`: for capacitance `C`, moving a node through a voltage change requires charge to move into or out of it. Thus changing a physical bit is not an

abstract relabeling—transistors briefly conduct current to move charge and raise or lower node voltages. This intuition is enough for Week 1; Week 4 develops [capacitance, delay, fanout, and power](#) quantitatively.

A practical digital receiver therefore does not demand two perfect voltage points. It recognizes ranges. As an illustrative example:



Illustrative thresholds only: valid LOW and valid HIGH are accepted ranges, the hatched middle is an undefined transition, and noise margin is the room between a sender's level and the LOW threshold.

The actual thresholds depend on the device and logic family. The durable idea is this:

> Logical zero and one are produced by classifying broad ranges of imperfect physical values.

Suppose a sender intends a LOW signal near 0.2 V and noise adds 0.3 V. The receiver sees 0.5 V, which is still safely in the LOW range. The separation between what a sender guarantees and what a receiver will accept provides a **noise margin**.

What about 1.4 V in the middle region? The specification does not promise a reliable logical result there. One device might read LOW while another reads HIGH; the answer may also vary with temperature or timing. Circuit designers arrange valid signals to pass through that region quickly and settle into a valid range.

The clean bit is therefore an abstraction boundary:

```
continuous, imperfect physical quantity
                ↓ thresholding and restoration
stable logical alternative: 0 or 1
```

Above that boundary, we can reason about exact logical values. Below it, engineers still have to manage analog behavior.

Why only two logical states?

Imagine dividing a 0-to-3.3-volt range into ten bands so one wire could directly represent a decimal digit. Adjacent values would be close together. Small disturbances could push a signal into the next band, and the receiver would need to distinguish many levels accurately.

Two states use the available physical range more generously:

```
many levels                       two levels
-----------                       ----------
more information per element      less information per element
narrower separation               wider noise margin
more precise receiver             simpler switching decision
harder restoration                easier restoration
```

Binary is not the only possible choice. Multi-level storage exists and can increase density, but it pays with tighter margins, more complex sensing, slower operations, wear management, or error correction.

Binary became foundational because it is an excellent engineering bargain:

> Give up density in one physical element to gain reliable discrimination, simple switching, and easy regeneration. Recover capacity by combining many elements.

---

4. From one bit to many patterns

A **bit** is one binary digit: one resolved choice between two logical alternatives.

The bit is not the voltage itself. Voltage is one possible implementation:

```
PHYSICAL IMPLEMENTATION            LOGICAL VALUE
low/high voltage                →      0 / 1
uncharged/charged storage cell  →      0 / 1
magnetic orientation A/B        →     0 / 1
open/closed relay               →     0 / 1
```

One bit has two possible patterns:

```
0
1
```

With two independent bits, each old pattern has two extensions:

```
00  01  10  11
```

With three:

```
000  001  010  011  100  101  110  111
```

Each added independent bit doubles the possibilities. Every existing pattern gets one version with the new bit equal to zero and one with it equal to one:

```
n bits → 2ⁿ distinct patterns
```

Do not treat the exponent as magic notation. It records repeated two-way choice:

```
2 × 2 × 2 × ... × 2 = 2ⁿ
```

This also solves the reverse problem. To distinguish at least 20 states:

```
4 bits → 16 patterns  (not enough)
5 bits → 32 patterns  (enough)
```

An encoding may leave some of those 32 patterns unused or reserve them for special purposes.

Bits, bytes, and words

A **byte** is eight bits grouped together:

```
01000110
```

Eight bits provide `2⁸ = 256` patterns. The eight-bit byte is a successful historical convention, not a law of physics.

A **word** is a context-dependent natural data width for an architecture. A modern processor may have 64-bit general-purpose registers, while an instruction-set manual may retain older names such as *word* for 16 bits, *doubleword* for 32 bits, and *quadword* for 64 bits. Ask which architecture and which convention are in use.

The hierarchy so far is:

```
physical state → logical bit → grouped bit pattern
```

But a pattern is not yet a number, a letter, or an instruction.

---

5. Positional notation: decimal, binary, and hexadecimal

We already use positional encoding every day. In decimal:

```
347 = 3×10² + 4×10¹ + 7×10⁰
```

A digit's contribution depends on both the digit and its position. Binary uses the same idea with base two:

```
bit position:       3   2   1   0
place value:        8   4   2   1
pattern:            1   0   1   1
value:              8 + 0 + 2 + 1 = 11
```

Under the **unsigned binary integer encoding**, 1011_2 means decimal 11. The pattern does not intrinsically contain “eleven.” We selected a rule that assigns powers-of-two place values.

Binary to decimal

Write the powers of two and add the active positions:

```
10110110₂

bits:          1   0   1   1   0   1   1   0
place values: 128  64  32  16   8   4   2   1

128 + 32 + 16 + 4 + 2 = 182
```

Decimal to binary

Find powers of two that sum to the value:

```
70 = 64 + 4 + 2
   = 01000110₂
```

You can also repeatedly divide by two and collect remainders, but the powers-of-two method makes the representation easier to understand.

Why hexadecimal is useful

Long binary patterns are difficult for humans to scan. Decimal is compact, but it hides bit boundaries. Hexadecimal works especially well because:

```
16 = 2⁴
```

Each hexadecimal digit corresponds exactly to four bits:

```
binary:       1111 1010 1101 1110
hexadecimal:     F    A    D    E
```

To convert binary to hex, group bits from the right in sets of four:

```
10110110₂ = 1011 0110₂ = 0xB6
```

This is why addresses, register values, masks, memory dumps, and instruction bytes are commonly shown in hex. Hexadecimal is not a different physical layer. It is a compact human notation for the same bit pattern.

Keep these statements separate:

```
01000110₂     binary notation
70            decimal notation
0x46          hexadecimal notation
```

Under an unsigned-integer rule, all three written forms denote the same numeric value.

---

6. Pattern first, meaning second

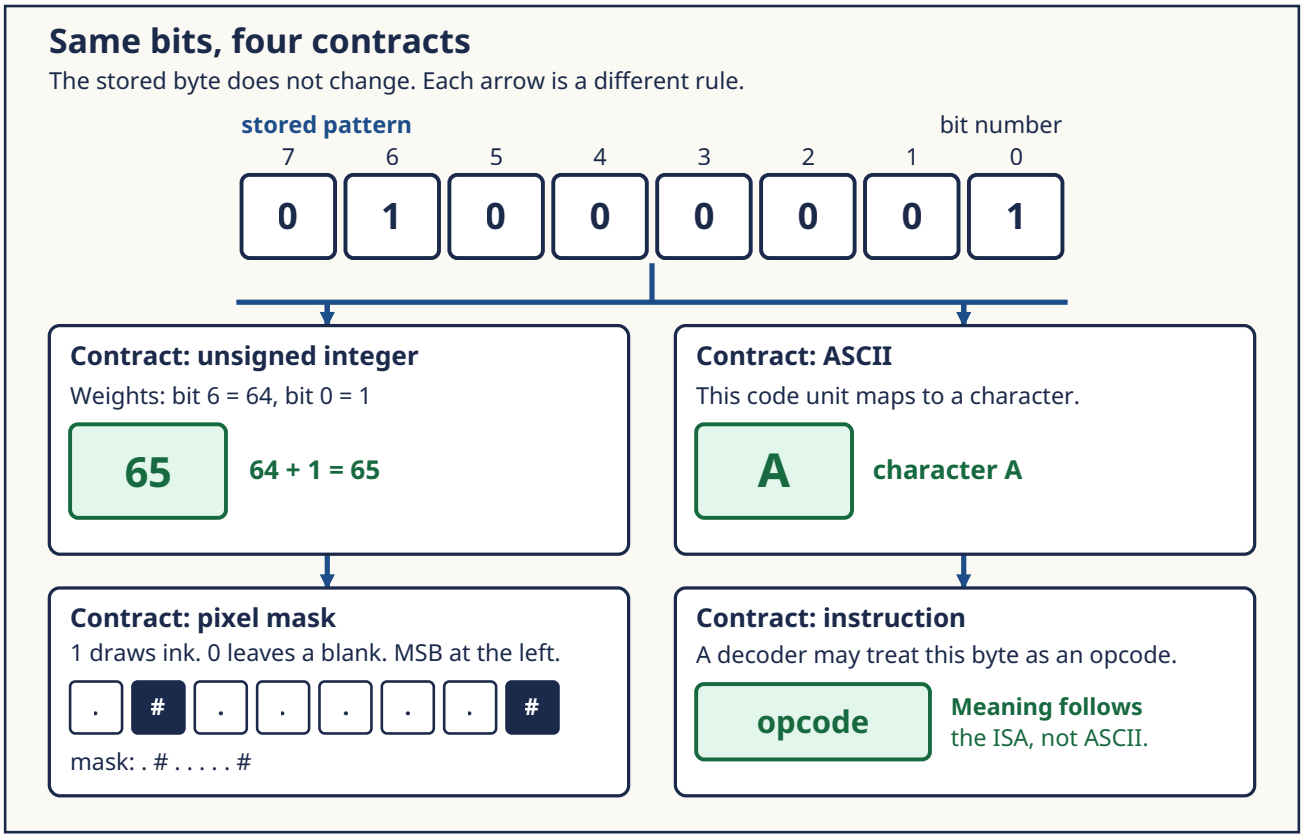
Now return to the byte:

```
01000110
```

Hand it to several readers:

```
                  ┌─ unsigned integer rule ─→ 70
01000110 ─────────├─ ASCII rule ──────────────→ F
                  ├─ bitmap rule ─────────────→ .#...##.
                  └─ instruction decoder ─────→ part of an operation
```

The bits did not change. The interpretation did.



The same stored byte 01000001 is unsigned 65, ASCII A, a pixel mask, or an opcode, and only the contract selects which reading is active.

An **encoding** is an agreed mapping between patterns and meanings:

```
pattern + interpretation rule → meaning
```

That idea did not begin with electronic computers. Writing maps marks to language. Braille maps raised-dot patterns to symbols. Telegraph systems map signal patterns to characters. Punched-card systems map hole positions to facts or operations.

Hollerith's tabulating system makes the bridge particularly clear:

```
fact about a person
    → clerk follows a card encoding
    → holes occupy selected positions
    → reader senses hole/no-hole electrically
    → machinery counts or sorts the encoded fact
```

A hole did not inherently mean an occupation or an age category. Its meaning came from the card format and the machinery built to interpret that format. A modern file format, character encoding, or instruction set is vastly more capable, but it relies on the same separation between carrier and convention.

The four-layer model

```
LAYER 1 — PHYSICAL
voltage, charge, magnetic orientation
continuous, imperfect implementation
        │
        │ thresholding and circuit design
        ▼
LAYER 2 — LOGICAL
0 or 1
discrete abstraction
        │
        │ combine independent bits
        ▼
LAYER 3 — PATTERN
01000110
one of 256 possible byte patterns
        │
        │ apply an encoding or interpretation
        ▼
LAYER 4 — MEANING
70, `F`, pixels, or an instruction fragment
```

Hardware can be the interpreter. A CPU's instruction decoder applies the instruction set's encoding. Software can also be the interpreter. A terminal applies character conventions; an image viewer applies a file and pixel format.

The pattern itself does not announce which reader should be used.

---

7. Information is not the same as storage

We often say that a device “stores information,” but storage capacity and information are not identical.

Imagine one physical bit permanently wired to zero. It occupies a bit-sized place in a design, but reading it tells you nothing you did not already know. There was no uncertainty about its value.

Now imagine a fair coin toss stored in one bit. Before reading it, either outcome is possible. After reading, one two-way uncertainty has been resolved.

At this level, think of **information** as a reduction in uncertainty. Think of **storage** as the physical capacity to preserve distinguishable patterns.

That distinction prevents two common mistakes:

1. A large storage area need not contain much new information. A million repeated zeros are highly predictable.
2. Meaning is not measured merely by counting storage cells. The same byte can participate in very different messages under different encodings.

Later, compression and information theory will make this quantitative. For now, preserve the conceptual boundary:

```
storage provides possible distinguishable states
information depends on which uncertainty those states resolve
meaning depends on an interpretation and context
```

---

8. Observe one pattern through several Linux readers

Now let the machine expose the abstraction layers. We will create four bytes once and then refuse to change them:

```
printf 'FADE' > /tmp/day1-pattern.bin
xxd -b /tmp/day1-pattern.bin
```

The bytes are:

```
01000110 01000001 01000100 01000101
```

Before each command, predict what kind of interpretation it requests.

View the bytes as numbers, hex, and characters

```
od -An -tu1 -tx1 -tc /tmp/day1-pattern.bin
```

The views should correspond to:

```
70 65 68 69       unsigned decimal byte values
46 41 44 45       hexadecimal byte values
 F  A  D  E       characters
```

The file has not changed between output rows.

View the bytes as instructions

```
objdump -D -b binary -m i386:x86-64 /tmp/day1-pattern.bin
```

The decoder applies the x86-64 instruction encoding. These bytes were not chosen to form a useful program, so do not memorize the resulting assembly. Observe the more important fact: the tool treats the same bytes as machine code rather than text.

View each byte as eight pixels

```
python3 - <<'PY'
for byte in open('/tmp/day1-pattern.bin', 'rb').read():
    row = ' '.join('#' if (byte >> bit) & 1 else '.'
                   for bit in range(7, -1, -1))
    print(row)
PY
```

Expected rows:

```
. # . . . # # .
. # . . . . . #
. # . . . # . .
. # . . . # . #
```

The Python code supplied a bitmap convention: one means `#`, zero means `.`, and the most significant bit is displayed first.

The one-pattern-many-meanings observation

This small experiment reaches surprisingly far into Linux:

- A debugger can display the same memory as hex, signed or unsigned numbers, characters, addresses, or instructions.
- A type tells compiled software how to interpret bits and which operations make sense; the type is not painted onto the RAM cells.
- A filename extension does not transform bytes. It helps select a program that knows an expected format.
- A memory-safety exploit may try to make bytes supplied as data become bytes fetched as instructions. The electrical storage still works; the dangerous change is interpretation and control flow.

When a low-level value looks wrong, ask:

1. What bits are actually present?
2. Which reader and encoding are interpreting them?

That habit is useful in debuggers, kernel logs, packet traces, memory dumps, and disassemblers.

---

9. Integrated challenge — rebuild the whole chain

Use paper first. Predict before running commands. Do not check earlier examples until you have committed to an answer.

Part A — physical reality

Assume a device guarantees:

```
LOW:  at or below 0.8 V
HIGH: at or above 2.0 V
```

1. Classify 0.3 V, 2.8 V, and 1.4 V.
2. Explain why the middle value must not be trusted.
3. A LOW signal is sent at 0.2 V and gains 0.4 V of noise. What logical value should the receiver still recognize?
4. Explain why ranges make a large digital system more reliable than exact voltage points would.
5. Give one reason a designer might still choose multiple physical levels per cell.

Part B — patterns and capacity

1. List all three-bit patterns.
2. Explain why there are eight without merely quoting 2^3.
3. How many patterns do six bits provide?
4. What is the minimum number of bits needed to distinguish 50 states?
5. If an encoding uses only 40 of those available patterns, what could the remaining patterns be used for?

Part C — conversions

Convert without a calculator and show your place values or nibble groups:

1. 01101101_2 to decimal and hexadecimal.
2. `0xA7` to binary and decimal.
3. decimal `99` to binary and hexadecimal.
4. 10110110_2 to decimal and hexadecimal.
5. Explain why hexadecimal preserves visible bit groupings better than decimal.

For every mistake, identify the cause: wrong place value, arithmetic slip, incorrect grouping, or confusion between a pattern and its interpretation.

Part D — predict, observe, explain

Create one byte:

```
printf '\x41' > /tmp/day1-check.bin
```

Write your predictions before running:

```
xxd -b /tmp/day1-check.bin
od -An -tu1 /tmp/day1-check.bin
cat /tmp/day1-check.bin
objdump -D -b binary -m i386:x86-64 /tmp/day1-check.bin
```

Then answer:

1. Did any command alter the file?
2. Which command requested binary notation?
3. Which requested an unsigned numeric interpretation?

4. Which caused the terminal to present a character?
5. Why did the instruction decoder not treat the byte as text?
6. Which result surprised you, and which layer had you accidentally assumed?

Choose a second byte yourself. Correctly predict its binary, decimal, hex, character, and instruction-decoder views before testing it.

Part E — repair this explanation

Rewrite the following so every noun belongs to the correct layer:

> A computer stores zero as exactly 0 volts and one as exactly 3.3 volts. Eight voltages make a byte. The byte `01000110` is the letter F.

Your repaired version must mention:

- physical voltage ranges,
- logical bits,
- a byte pattern,
- and the interpretation supplied by ASCII.

---

10. Mastery checkpoint

Close the lesson and take a blank sheet.

Explain

Answer from first principles:

1. Why is electricity not naturally binary?
2. Why are two broad logical ranges useful in a noisy physical system?
3. Why does each additional independent bit double the number of patterns?
4. What is the difference between a bit pattern and an encoding?
5. Why can one pattern represent a number, character, instruction, or pixel?
6. What is the difference between storage capacity and information?

Draw

Reconstruct and annotate every arrow:

```
physical state → logical bit → bit pattern → interpretation → meaning
```

Add one concrete example beneath each stage.

Observe

Reproduce the Linux experiment with a byte of your choosing. Show the same stored byte in binary, decimal, hexadecimal, text, and instruction-decoder views. Explain why the outputs differ.

Build

Invent an encoding for eight possible machine states:

- assign each state a three-bit pattern,
- reserve at least one pattern for an error or future use,
- describe the physical states that could implement the bits,
- and explain what reader would turn the patterns into meaning.

The central answer

You have mastered the foundation when you can express this idea in your own words:

> A physical device preserves distinguishable states. Circuit design maps ranges of imperfect physical values into logical bits. Groups of bits form patterns, but storage does not attach semantic labels to them. An encoding, together with a software or hardware reader, maps a pattern to a number, character, instruction, pixel, or another meaning.

Do not mark the checkpoint complete because the paragraph looks familiar. Complete it when you can rebuild the causal chain, perform the conversions, and explain your observed Linux output without the notes.

---

11. Why this foundation matters to a CPU

We can now represent inputs and results, but we have not yet built a machine that computes with them.

Suppose we want hardware to add two encoded numbers. Several new problems appear:

1. For every possible input pattern, which output pattern should be produced?
2. How can switches implement rules such as AND, OR, and NOT?
3. How can those rules be composed into arithmetic?
4. Where can an intermediate result remain while the next operation occurs?
5. What selects addition rather than another operation?
6. What determines the order of operations?

Those questions lead from bits to logic gates, from gates to adders, from adders to a datapath, and from stored control patterns to machine instructions:

```
reliable representation
    + logic
    + arithmetic
    + storage
    + control
    + encoded instructions
    = a programmable processor
```

A conventional CPU does not literally “think.” Physical states change according to circuits and encoded instructions. The remarkable behavior of applications, operating systems, and learning systems emerges from enormous compositions of those simple, controlled transitions.

The point of Day 1 is therefore larger than learning binary conversion. We have established how a noisy physical world can support stable symbols and how stable symbols can carry different meanings. Next, we can ask how a circuit transforms those symbols according to a rule.

References used selectively

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 2–3 for codes and combinations, Chapter 9 for bits, and Chapter 15 for bytes and hexadecimal (local PDF chapter openings at pages 18, 24, 80, and 199).
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd edition, Chapter 2, “Bits, Data Types, and Operations,” book pages 21–36.
- IBM, “The punched card tabulator,” for Hollerith's use of hole/no-hole patterns and electrical contacts to mechanize census processing: https://www.ibm.com/history/punched-card-tabulator
- Computer History Museum, “Computers Timeline,” for the broader transition from mechanical calculation through relay and electronic machines: https://www.computerhistory.org/timeline/computers/

The references provide historical and technical grounding. The causal narrative, Linux observations, and integrated challenge are synthesized for this masterclass.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Bit Pattern Lab

Format: Python lab

What you will do: Week 1 Day 1 coding challenge: representation without hidden magic.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
python3 01-foundations-to-cpu/challenges/day-001-bit-pattern-lab.py
```

- View lab source on GitHub

Day 2 — Signed Bits

Curriculum alignment: Stage 1 — Foundations of Computation · Signed Numbers and Bitwise Reasoning

Target time: ~3 hours (≈75 minutes concept work, ≈90 minutes labs, ≈15 minutes mastery explanation)

> A byte arrives from a device:
>
> `text 11010110`
>
> Is it 214? Is it −42? Is it a collection of status lights? Did an arithmetic result overflow?
>
> The pattern alone cannot answer. We need the rule that gives the pattern meaning.

The previous foundation established the physical-to-logical path: a voltage falls into an agreed range, circuitry classifies it as one of two states, and we write those states as 0 and 1. Today we give groups of those states useful structure.

We will follow one engineering problem from beginning to end:

```
finite bit patterns
    → signed-number encoding
    → Boolean operations on selected bits
    → packed device fields
    → fixed-width arithmetic and flags
    → the beginnings of an ALU
```

The important habit is not merely converting binary quickly. It is asking:

> **What interpretation rule is active, what width applies, and what facts were lost when the hardware kept only that width?**

---

1. Why machines need a representation for negative values

People can write a minus sign beside a magnitude. A machine cannot get that extra symbol for free. Every stored distinction needs a physical state; every arithmetic rule needs circuitry or a sequence of operations.

This problem was visible before electronic computers. A mechanical calculator had only a fixed number of wheel positions. When a wheel passed its last position, it returned to its first and carried into the next wheel. Designers learned that complement arithmetic could turn subtraction into addition followed by discarding a carry. That mattered because one reliable adding mechanism was easier to build than unrelated machinery for every combination of signs.

Binary registers have the same circular constraint:

```
decimal wheel:   ... 8 → 9 → 0 → 1 ...
8-bit register:  ... 254 → 255 → 0 → 1 ...
```

An eight-bit register has exactly 256 physical patterns:

```
00000000 through 11111111
```

Unsigned binary assigns them to 0 through 255. But temperatures fall below zero, account balances can be negative, offsets can point backward, and subtraction can produce a negative result. We need a different mapping of those same patterns.

The design question is therefore:

> Which mapping gives us negative values while making arithmetic hardware as simple and regular as possible?

First attempt: signed magnitude

The most human-looking answer is to use the highest bit as a sign and the remaining bits as the magnitude:

```
4-bit signed magnitude

0011 → +3
1011 → −3
```

But this produces two zeros:

```
0000 → +0
1000 → −0
```

It also complicates arithmetic. Before adding, the machine must inspect both signs and decide whether to add magnitudes, subtract them, and which sign to attach. The notation is easy for a person to glance at, but the datapath needs extra cases.

Signed magnitude still exists where its trade-offs make sense, including the sign-and-significand view of common floating-point formats. It was simply a poor universal answer for ordinary integer arithmetic.

Second attempt: one's complement

One's complement represents a negative value by inverting every bit of the positive value:

```
0011 → +3
1100 → −3
```

This moves closer to complement arithmetic, but it still has two zeros:

```
0000 → +0
1111 → −0
```

Addition also requires an end-around-carry correction: a carry leaving the highest position must be added back into the low position. That is workable, but it leaves an irregular rule in a very common operation.

The extra one that regularizes the circle

Two's complement forms `−x` by inverting the bits of `x` and adding one:

```
+3          0011
invert      1100
add one   + 0001
            ----
−3          1101
```

The added one is not a ritual to memorize. It removes negative zero and makes the encoding line up with fixed-width modular arithmetic.

---

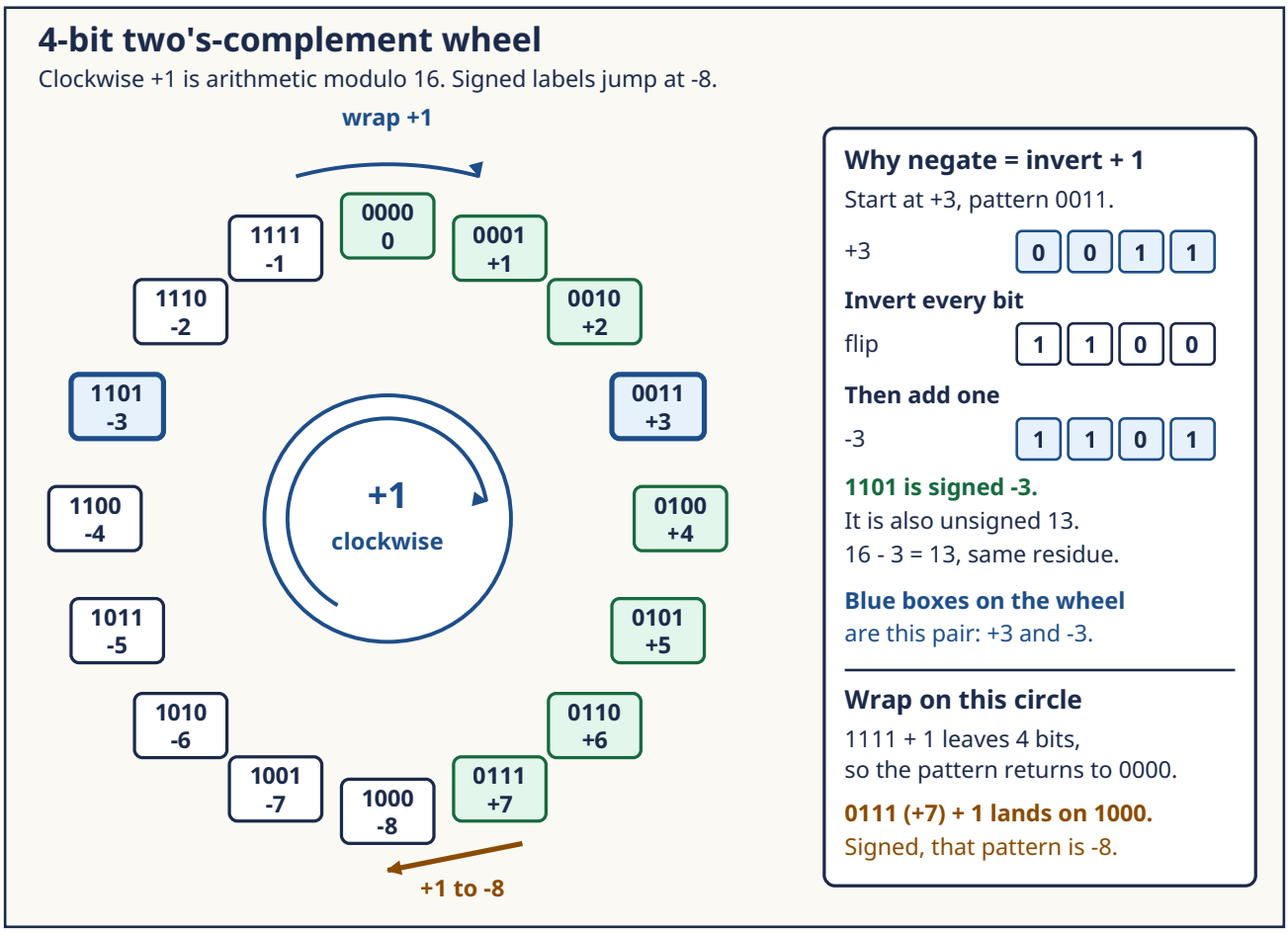
2. Two's complement as arithmetic modulo 2^n

With four bits, hardware retains only the low four result bits. There are $2^4 = 16$ positions, so arithmetic circles modulo 16.

```
pattern   unsigned reading   signed two's-complement reading
-------   ----------------   --------------------------------
0000              0                        0
0001              1                        1
...
0111              7                        7
1000              8                       −8
1001              9                       −7
...
1111             15                       −1
```

To encode `−3`, choose the non-negative representative of the same residue:

```
−3 mod 16 = 16 − 3 = 13 = 1101₂
```



Adding one walks clockwise modulo 16, +7 steps onto -8, and +3 becomes -3 because inverting 0011 and adding one lands on 1101.

This gives two equivalent ways to decode an `n`-bit pattern whose high bit is 1:

```
signed value = unsigned value − 2ⁿ
```

or:

```
invert → add one → read magnitude → attach minus sign
```

Try the running byte:

```
11010110
```

Unsigned:

```
128 + 64 + 16 + 4 + 2 = 214
```

Signed, using the modular rule:

```
214 − 256 = −42
```

Signed, using invert-plus-one:

```
11010110
invert → 00101001
add 1  → 00101010 = 42
therefore the original pattern represents −42
```

The two methods agree because they are two views of the same mathematics.

Why one binary adder can serve both interpretations

Consider four-bit `5 + (−3)`:

```
  0101
+ 1101
------
1 0010
```

The four-bit destination retains `0010`, which is 2. The carry leaving the width is discarded. No separate negative-number adder was needed.

The gates in an adder do not label one input “signed” and another “unsigned.” They combine bit positions and generate sums and carries. Signedness tells us how to interpret the operands, result, and status flags.

That regularity is the central engineering reason two's complement became the normal integer representation:

- one representation for zero;
- addition and subtraction can share an adder;
- negation is invert-plus-one;
- the high bit directly reveals the sign under the signed interpretation;
- widening can preserve a signed value by copying that high bit.

The range and its asymmetry

For `n` bits:

```
unsigned:          0 through 2ⁿ − 1
two's complement: −2ⁿ⁻¹ through 2ⁿ⁻¹ − 1
```

For eight bits:

```
unsigned:           0 through 255
two's complement: −128 through 127
```

There are 128 negative values, one zero, and 127 positive values. The most negative pattern, `10000000`, has no positive counterpart in the same width. That is why operations such as taking the absolute value of the most negative signed integer require special care.

Quick derivation, not recognition

Without a calculator, decode `11100101`:

1. Read it as an unsigned sum of place values.
2. Subtract 256 for the signed reading.

3. Verify by invert-plus-one.

If you can derive both readings, you understand the representation. Merely remembering that “a leading 1 means negative” is not enough; that statement is true only after selecting a signed two's-complement interpretation.

---

3. When a word is a control panel, not a number

Now reinterpret our running pattern as a fictional device register:

```
bit:       7       6         5   4   3       2       1       0
meaning: READY   ERROR        MODE[2:0]      RSV    IRQ_EN  START
```

Suppose the device specification says:

```
bit 7      READY     device has completed an operation
bit 6      ERROR     operation failed
bits 5:3   MODE      operating mode, values 0–7
bit 2      reserved  preserve or write zero as specified
bit 1      IRQ_EN    interrupt generation enabled
bit 0      START     request a new operation
```

The byte is still physically `11010110`, but adding its place values is now usually the wrong question. Software needs to inspect or modify selected positions while preserving the others.

The tools come from Boolean algebra. Long before transistor CPUs, logical algebra described relationships such as AND, OR, and NOT. Communications engineers later had large networks of electromechanical relays: current in a coil moved a contact, and contacts in series or parallel naturally behaved like logical conditions. Work connecting Boolean algebra to switching networks showed that an expression on paper could systematically become wiring.

Relay adders then made the bridge concrete:

```
logical relation
    → arrangement of switches
    → physical output state
    → calculation
```

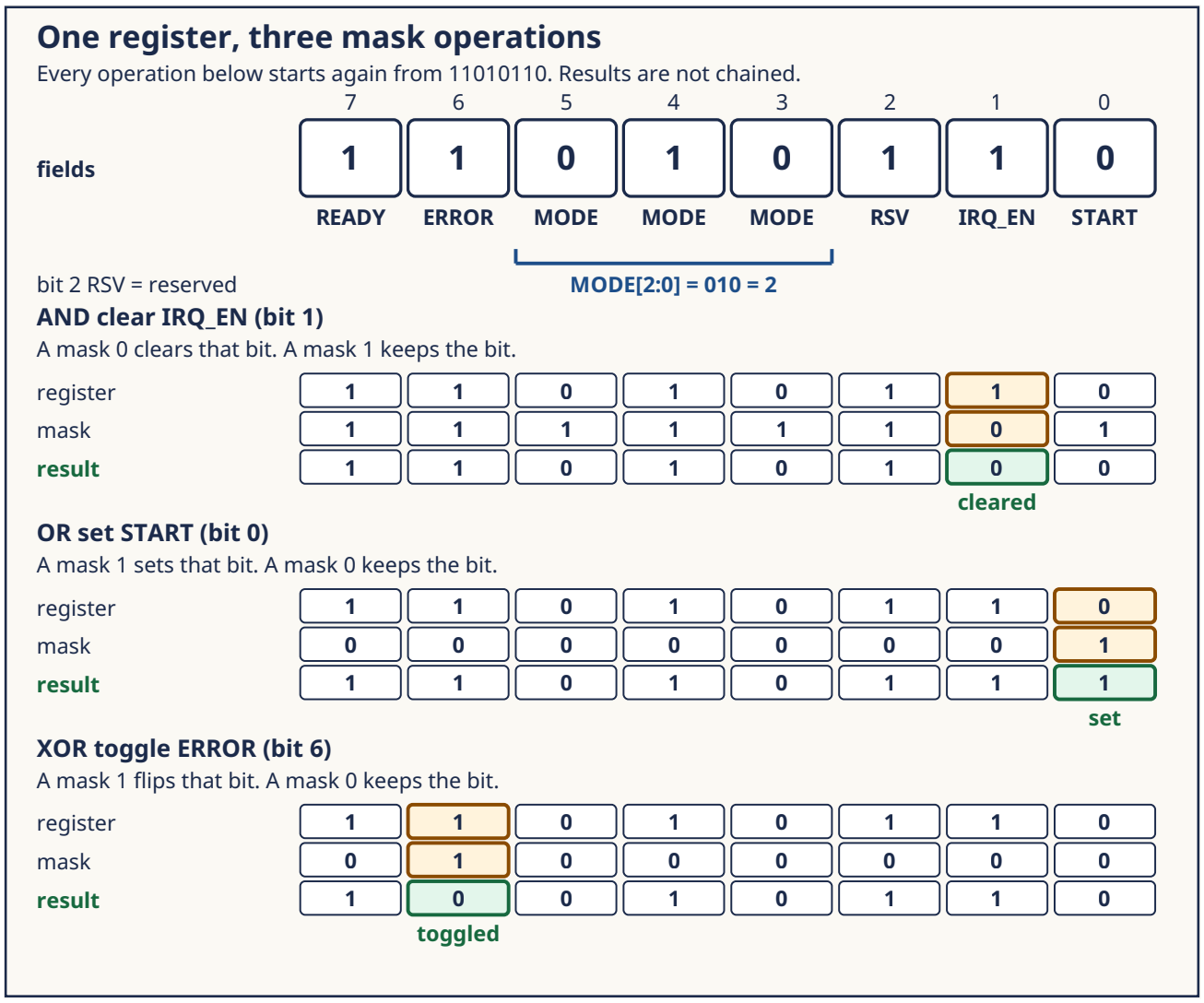
Modern CPUs replace moving relay contacts with transistor networks, but the logic remains recognizable. A C expression such as:

```
reg & ERROR_MASK
```

is an abstract request that ultimately becomes controlled state transitions through logic gates.

---

4. Four bitwise operations, four mask intentions



From the same byte 11010110, AND with a 0 clears IRQ_EN, OR with a 1 sets START, and XOR with a 1 toggles ERROR, while every other bit stays put.

AND: keep selected information

AND produces 1 only if both input bits are 1:

```
0 & 0 = 0    0 & 1 = 0
1 & 0 = 0    1 & 1 = 1
```

Test the ERROR bit, bit 6:

```
register: 11010110
mask:     01000000
          -------- &
result:   01000000
```

The nonzero result says bit 6 was set.

```
if (reg & (1u << 6))
    puts("ERROR is set");
```

AND can also force selected positions to zero. To clear bit 1:

```
reg &= ~(1u << 1);
```

The inner shift makes `00000010`; NOT makes a mask with zero only at the target position; AND preserves every other bit.

OR: force selected bits on

OR produces 1 if either input is 1:

```
0 | 0 = 0    0 | 1 = 1
1 | 0 = 1    1 | 1 = 1
```

Set IRQ_EN:

```
reg |= 1u << 1;
```

A 1 in the mask forces the target on. A 0 in the mask preserves the original bit.

XOR: toggle selected bits

XOR produces 1 when its inputs differ:

```
0 ^ 0 = 0    0 ^ 1 = 1
1 ^ 0 = 1    1 ^ 1 = 0
```

Toggle START:

```
reg ^= 1u << 0;
```

A 1 in the mask flips a bit; a 0 preserves it. Applying the same XOR mask twice restores the original:

```
x ^ mask ^ mask = x
```

That reversible property helps explain XOR's appearance in checksums, difference tracking, and cryptographic constructions. Those uses add further rules; XOR by itself is not encryption.

NOT: invert the active width

NOT flips every bit:

```
~00110110 = 11001001    if the active width is exactly eight bits
```

The width qualification matters in C. Small integer operands normally undergo integer promotion before `~`:

```
uint8_t x = 0x36;
uint8_t inverted = (uint8_t)~x;
```

The operation commonly occurs at `int` width, and the cast narrows the result back to eight bits. This is our first warning that a variable's storage width does not necessarily determine an expression's evaluation width.

---

5. Shifts construct and position fields

Shifts move bit positions:

```
00000001 << 3 = 00001000
00110000 >> 4 = 00000011
```

For unsigned values, a representable left shift by one corresponds to multiplication by two; right shift by one corresponds to integer division by two. In systems code, their more important role is often positional.

Our MODE field occupies bits 5:3. Its mask is:

```
00111000 = 0x38
```

Extract it in two steps:

```
unsigned mode = (reg & 0x38u) >> 3;
```

```
register: 11010110
mask:     00111000
AND:      00010000
shift:    00000010  → MODE is 2
```

The general shape is:

```
field = (word & FIELD_MASK) >> FIELD_SHIFT;
```

Replacing a field also takes two deliberate steps:

```
reg &= ~MODE_MASK;                       /* clear old field */
reg |= (new_mode << MODE_SHIFT) & MODE_MASK;
```

Masking the shifted value prevents an oversized input from leaking into neighboring fields. Validating the input may still be appropriate.

Do not use this:

```
reg |= new_mode << MODE_SHIFT;
```

OR can set field bits but cannot clear old ones. If the old MODE is `111` and the new MODE is `010`, blind OR leaves `111`.

Reserved and side-effect bits

“Reserved” does not mean “available to software.” A device specification may require such bits to be written as zero, preserved during read-modify-write, or ignored when read.

Some registers add another complication:

- writing 1 may clear a status bit;
- reading may acknowledge an event;
- hardware may change bits between software's read and write;
- some fields may be read-only.

Therefore a correct mask is not enough. The device contract also defines whether the access sequence is safe. This distinction will matter constantly in driver and kernel work:

```
bitwise mechanism + register semantics = correct operation
```

---

6. Lab A — Operate a packed device register

Open the existing challenge:

`01-foundations-to-cpu/challenges/day-002-register-mask-lab.c`

It provides a fictional eight-bit register and tests for functions that:

1. set one bit;
2. clear one bit;
3. toggle one bit;
4. test one bit;
5. extract MODE;
6. replace MODE while preserving neighboring positions.

Before editing each TODO, draw three rows:

```
original
mask
result
```

Then build and run:

```
gcc -std=c17 -Wall -Wextra -Wconversion -O0 -g \
  01-foundations-to-cpu/challenges/day-002-register-mask-lab.c \
  -o /tmp/day-002-register-mask-lab

/tmp/day-002-register-mask-lab
```

For every passing function, explain why zeros or ones in the mask preserve, force, clear, or toggle the corresponding source position. Passing tests without that explanation is useful coding evidence, but not yet mastery.

---

7. Fixed-width arithmetic: the answer may not fit

Pure mathematics does not run out of integer digits. Physical arithmetic units do.

A mechanical calculator has finite wheels. A relay calculator has finite relay groups. An electronic CPU has finite registers, wires, and adder stages. New switching technologies made machines dramatically faster, smaller, more reliable, and less power-hungry, but width never became free. Wider datapaths still cost circuitry, switching energy, routing space, and time.

Add one to the largest eight-bit pattern:

```
  11111111
+ 00000001
----------
1 00000000
```

The full answer needs nine bits. An eight-bit destination stores `00000000`. The extra bit is a carry-out.

The adder did not make a mathematical mistake. It produced a fixed-width result. What that result means depends on the chosen interpretation.

Unsigned wrap is modular arithmetic

For an `n`-bit unsigned integer:

```
arithmetic is modulo 2ⁿ
```

At eight bits:

```
255 + 1 ≡ 0   mod 256
0 − 1   ≡ 255 mod 256
```

The C language defines unsigned integer arithmetic modulo the type's range. Wraparound is therefore defined behavior. It is useful for counters, hashes, ring indices, checksums, and algorithms deliberately built on modular arithmetic.

Defined does not mean automatically safe. If an allocation-size calculation wraps to a small value, later writes may exceed the allocation. The language can define the arithmetic while the program remains dangerously wrong.

Signed overflow is a different contract

Now add signed eight-bit `127 + 1`:

```
  01111111   +127
+ 00000001     +1
----------
  10000000   looks like −128 under two's complement
```

The low eight result bits are ordinary binary addition, but the mathematical answer `+128` is outside the signed eight-bit range.

At the CPU instruction-set level, typical arithmetic instructions produce low result bits and update status information.

At the C-language level, signed integer overflow is undefined behavior. The compiler may assume that a valid program does not perform it and optimize accordingly.

```
physical CPU / ISA              C abstract machine
------------------              ------------------
produces low result bits        signed overflow has no defined result
may update status flags         optimizer may assume it never occurs
```

Observing wraparound in one unoptimized executable does not turn it into a portable C guarantee.

Production C should check before the operation or use an overflow-checking facility, for example:

```
int result;
if (__builtin_add_overflow(a, b, &result)) {
    /* mathematical result did not fit */
}
```

---

8. Carry and overflow answer different questions

One binary adder serves both unsigned and signed interpretations, so the CPU needs separate facts.

Carry asks:

> Did the unsigned mathematical result need a bit beyond the available width?

Signed overflow asks:

> Did the signed mathematical result fall outside the two's-complement range?

Compare two additions.

```
11111111 + 00000001 = 1 00000000
```

As unsigned: `255 + 1` does not fit, so there is carry.

As signed: `−1 + 1 = 0` fits, so there is no signed overflow.

Now:

```
01111111 + 00000001 = 0 10000000
```

As unsigned: `127 + 1 = 128` fits, so there is no carry-out.

As signed: `127 + 1` does not fit, so signed overflow occurred.

For signed addition, a useful detection rule is:

```
same-sign operands + different-sign result → signed overflow
```

At the gate level, an adder is already producing carry signals between bit positions. Logic around the highest position can derive the carry and overflow facts. Status flags preserve those facts after the result has been truncated to the register width.

This is a first glimpse of an ALU interface:

```
operand A ─┐
           ├─ arithmetic/logic network ─→ result bits
operand B ─┘                            └──→ status flags
```

The result alone is sometimes insufficient. `00000000` could be a genuine zero or the wrapped low portion of a larger answer. The flag provides additional context about the operation that produced it.

---

9. Changing width: preserve which interpretation?

Suppose the eight-bit pattern is:

```
11110110
```

Under unsigned interpretation it is 246. Under signed two's complement it is −10.

To preserve the unsigned value while widening to 16 bits, **zero-extend**:

```
00000000 11110110 → 246
```

To preserve the signed value, **sign-extend** by copying the old high bit:

```
11111111 11110110 → −10
```

Why does copying ones work?

```
8-bit representation of −10:   2⁸  − 10
16-bit representation of −10:  2¹⁶ − 10
```

The new high ones contribute exactly the place values needed to select the same negative residue at the wider modulus.

The general rule is:

```
unsigned widening → fill new high bits with 0
signed widening   → replicate the old sign bit
```

Extension is therefore not “just adding space.” It preserves a chosen numeric interpretation while changing width.

CPUs commonly expose this distinction directly through zero-extending and sign-extending instructions. A compiler chooses between them according to the source type and operation.

---

10. C preview: storage width is not expression width

Consider:

```
uint8_t a = 255;
uint8_t b = 1;
printf("%d\n", a + b);
```

It commonly prints 256, not 0. Before addition, both operands undergo **integer promotion**. On a system where `int` can represent every `uint8_t` value, both become `int`, and the addition occurs at `int` width.

Only a later conversion narrows:

```
uint8_t result = a + b;   /* 256 converted to uint8_t, yielding 0 */
```

This distinction explains why the overflow lab shows both:

```
promoted expression result
stored eight-bit result
```

It also explains the earlier NOT example: `~x` can invert an `int`-width promoted value even when `x` is stored in eight bits.

We will derive the full promotion and conversion rules in later C work. For today, keep this preview:

> Always distinguish the declared object's width, the promoted expression type, and the destination width.

One more language boundary matters in the lab. Converting an unsigned value that is not representable in the corresponding signed type has implementation-defined behavior in C17. On the intended GCC, two's-complement target, converting `0xf6` through `int8_t` produces the expected −10 view. Treat that as an observed implementation choice, not a universal license to reinterpret every out-of-range conversion silently.

---

11. Lab B — Predict, observe, explain overflow

Open:

`01-foundations-to-cpu/challenges/day-002-overflow-lab.c`

Do not run it immediately. Read each numbered output expression and predict:

- the promoted expression value;
- the value eventually stored;
- whether the behavior is defined by C;
- whether zero extension or sign extension occurs;
- where instrumentation should report signed overflow.

Build normally:

```
gcc -std=c17 -Wall -Wextra -O0 -g \
  01-foundations-to-cpu/challenges/day-002-overflow-lab.c \
  -o /tmp/day-002-overflow-lab

/tmp/day-002-overflow-lab
```

Then build with undefined-behavior instrumentation:

```
gcc -std=c17 -Wall -Wextra -O1 -g \
  -fsanitize=undefined -fno-sanitize-recover=undefined \
  01-foundations-to-cpu/challenges/day-002-overflow-lab.c \
  -o /tmp/day-002-overflow-lab-ubsan

/tmp/day-002-overflow-lab-ubsan
```

Record four columns:

```
prediction | observation | C rule | hardware/CPU connection
```

In particular, explain why:

1. `u8_max + 1` can be 256 as a promoted expression;
2. storing that result in `uint8_t` yields 0;
3. zero extension and sign extension produce different high bits;
4. checked addition can report failure without first invoking signed overflow;
5. ordinary execution's apparent wrapped signed result is not a C promise;
6. UBSan stops at the intentional signed-overflow expression.

This is the complete learning loop:

```
predict → run → observe → explain → connect to the machine
```

---

12. Integrated investigation: one byte, several contracts

Return to:

```
11010110
```

Work through each lens without mixing rules.

Lens A: unsigned number

Use positive powers-of-two place values. State the range for eight bits and the resulting value.

Lens B: signed two's-complement number

Use both `unsigned − 256` and invert-plus-one. State why the high bit has negative significance only under this rule.

Lens C: packed register

Using the layout from section 3:

- identify READY and ERROR;
- extract MODE;
- identify IRQ_EN and START;
- write a mask to clear IRQ_EN;
- replace MODE with 5 while preserving every other bit.

Lens D: arithmetic result

Add each operand separately to the original byte:

```
00000001
00110000
10000000
```

For each addition:

1. calculate the low eight result bits;
2. record carry-out;
3. decide whether signed overflow occurred;
4. read the result as unsigned;
5. read the result as signed.

Do not say merely “it overflowed.” State **which interpretation's range did not contain the mathematical answer**.

Lens E: widened value

Write the 16-bit zero extension and 16-bit sign extension. State which eight-bit interpretation each preserves.

---

13. Mastery checkpoint

The tracker checkpoint is:

> **Given an eight-bit pattern, identify plausible interpretations and the interpretation rule.**

Use a fresh pattern:

```
10110101
```

A mastery-level response should cover at least these plausible interpretations:

- **Unsigned integer:** use positive binary place values.
- **Signed integer:** specify eight-bit two's complement, then use the negative high-bit weight, subtract `2⁸`, or invert-plus-one.

- **Packed flags and fields:** obtain a bit layout from the device or data format specification, then use masks and shifts.
- **Character data:** obtain a character encoding; one byte may be a complete code unit, part of a multi-byte sequence, or invalid in that context.
- **Machine instruction:** obtain the ISA, execution mode, and instruction boundary.
- **Pixel or compressed data:** obtain the format, dimensions, channel layout, and bit-order rules.

Then say the governing principle in your own words:

> Bits constrain the possible interpretations, but a type, format, ISA, protocol, or device specification selects the active one.

Complete the evidence in four forms:

1. **Explain** why two's complement lets one fixed-width adder serve signed and unsigned arithmetic.
2. **Draw** the pattern, masks, field boundaries, result width, carry, and extension.
3. **Observe** both C labs, including the instrumented overflow run.
4. **Build** the TODO implementations in the register-mask lab.

You are ready to advance when you can:

- explain why signed magnitude and one's complement have two zeros;
- derive two's complement from modular arithmetic;
- derive the signed range rather than recite it;
- construct set, clear, toggle, and test masks;
- extract and replace a packed field safely;
- distinguish unsigned carry from signed overflow;
- distinguish C's defined unsigned wrap from signed-overflow undefined behavior;
- choose zero extension or sign extension for a stated reason;
- notice when integer promotions change the width of a C expression;
- and name the interpretation rule before assigning meaning to a pattern.

If one item is weak, revisit that section and create a smaller example. Do not repeat the entire unit mechanically.

---

14. Where this moves us: toward the ALU and CPU

We now have more than representations.

```
AND, OR, XOR, NOT       transform bit vectors logically
shifts                  reposition bits and fields
addition                combines values and carries
status logic            reports zero, sign, carry, and overflow facts
```

These are central operations of an **arithmetic and logic unit**. At the physical level, transistor networks implement the gates. At the datapath level, buses bring operand patterns from registers, control signals select an operation, and result bits return to a destination register.

```
register A ──────┐
                 │
register B ──────┼──→ ALU operation network ──→ result register
                 │                          └──→ condition flags
control select ──┘
```

But an ALU is still not a CPU. We still need:

- storage so one operation can use an earlier result;
- a clock and state elements to coordinate changes;
- control logic to select ADD, AND, shift, or another operation;
- an instruction encoding that says which operation and operands to use;
- a program counter and memory path to fetch the next instruction;
- conditional behavior that can choose what happens next.

The stored-program idea makes instructions themselves bit patterns. An ISA supplies their interpretation rule, just as a device specification supplied the interpretation rule for our register:

```
instruction bits + ISA rule → requested CPU action
```

The specification is a contract. The instruction bits occupy storage. The decoder and datapath are physical circuitry. Keeping those layers distinct prevents the vague claim that meaning somehow lives inside the bits.

A conventional computer does not literally think at this level. Gates react to electrical states; registers preserve patterns; control circuitry drives sequences of transitions. Rich behavior emerges when enormous compositions of those operations implement algorithms, operating systems, applications, and learned models.

Today's operations are therefore not isolated binary tricks. They are pieces of the machinery we will combine into an ALU, then a controlled datapath, and eventually a minimal CPU.

References used selectively

- Patt and Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, integer data types, arithmetic, logical operations, conversion, sign extension, and overflow.
- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 2, information storage, integer representations, bit-level operations, and integer arithmetic.
- Charles Petzold, *Code*, “But What About Subtraction?”, for the path from complement arithmetic to binary subtraction machinery.
- Current Linux kernel headers `include/linux/bits.h` and `include/linux/bitfield.h` as examples of mask and field abstractions.
- GCC documentation for integer overflow built-ins and undefined-behavior instrumentation.

After Day 2: share the two lab results, the fresh-pattern mastery explanation, and any point where prediction differed from observation. That evidence determines whether the next unit should repair a gap or move toward logic gates, adders, and the ALU.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Overflow Lab

Format: C lab

What you will do: Day 2 challenge B — predict every line before running.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 01-foundations-to-cpu/challenges/day-002-overflow-lab.c
-lm -o /tmp/day-002-overflow-lab && /tmp/day-002-overflow-lab
```

- View lab source on GitHub

Put it into practice — Register Mask Lab

Format: C lab

What you will do: Day 2 challenge A — implement bit and field operations.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 01-foundations-to-cpu/challenges/day-002-register-mask-
lab.c -lm -o /tmp/day-002-register-mask-lab && /tmp/day-002-register-mask-lab
```

- View lab source on GitHub

Day 3 — Boolean Logic and Physical Gates

Target time: approximately 3 hours

Suggested rhythm: 70 minutes at the whiteboard · 65 minutes in the Logic Gate Lab · 30 minutes observing code · 15 minutes explaining from memory

> A switch can be open or closed. That gives us a state.
>
> But how could a pile of switches decide, “turn on only when both conditions are true”?

Why this day exists

We already know how a physical system can carry a bit:

```
continuous physical quantity
        ↓ engineered thresholds
stable LOW or HIGH
        ↓ agreed encoding
logical 0 or 1
```

That solves **representation**. It does not yet solve **computation**.

A loose switch is only a controllable break in a path. It does not inherently mean AND, OR, permission, equality, or addition. To compute, switches need:

1. an arrangement—series, parallel, inversion, or a larger network;
2. a convention—what voltage ranges count as 0 and 1;
3. a rule connecting input states to an output state;
4. restoration—each stage must produce a signal clean enough to drive the next stage.

The arrangement is physical. The rule is logical. A **logic gate** is where those two descriptions meet.

By the end of Day 3, you should be able to look at a Boolean function, write its truth table, imagine a physical switching network that realizes it, and explain why the circuit computes without understanding what its bits mean.

The larger path is:

```
switching device
    → gate
    → Boolean expression
    → combinational circuit
    → arithmetic and selection
    → ALU
    → CPU datapath
```

Today earns the first four arrows. We will glimpse one-bit addition, but save full adders, storage, clocks, and registers for their own treatment.

Part I — The historical problem: rules need machinery

Boole separated reasoning from the things being reasoned about

Human reasoning contains patterns such as:

```
the machine may run IF power is present AND the guard is closed
the alarm sounds IF smoke is detected OR heat is excessive
the lamp is on IF the switch is NOT open
```

George Boole showed that such statements could be treated algebraically. Instead of arguing about machines, smoke, or guards, we can use variables and operations:

```
P AND G
S OR H
NOT O
```

This was a major abstraction step. A variable could stand for any two-way proposition. The same algebra worked regardless of the proposition's meaning.

But an algebra on paper is not yet a computer. The next engineering question was:

> Can a physical network obey the same algebra?

Shannon saw algebra inside relay circuits

Telephone networks used electromagnetic relays. A small current in a coil moved a contact, opening or closing another electrical path. Engineers already combined contacts to route calls and control equipment, but large networks were difficult to reason about systematically.

Claude Shannon made the crucial connection: a relay contact has two useful states, and networks of contacts can be described with Boolean algebra.

```
contact closed  ↔ condition true  ↔ 1
contact open    ↔ condition false ↔ 0
```

Two contacts in series require both to close: AND. Two contacts in parallel allow either path to conduct: OR. A normally closed contact opens when activated: NOT-like behavior.

The practical gain was not merely philosophical. A messy wiring problem could be written as an expression, simplified algebraically, and rebuilt with fewer contacts. Logic became a design language for physical machinery.

Why relays did not become the final switching element

Relays make the idea visible, but their contacts physically move. Movement takes time; contacts bounce, wear, make noise, and consume substantial space. Vacuum tubes removed mechanical motion but were bulky, hot, and power-hungry.

The transistor supplied a compact solid-state switch. In modern CMOS logic, MOSFETs are arranged so an input voltage controls conducting paths toward the power rail or ground. No single transistor is “the Boolean algebra.” The **network topology** makes the Boolean function.

> **CMOS preview:** the transistor sketches here support Boolean intuition; they are not yet a device-level treatment. Week 4 develops the CMOS inverter and its noise margins before building CMOS NAND, NOR, and gate networks.

That causal chain matters:

```
Boolean algebra: describe two-valued rules
        ↓
relay networks: embody those rules in controlled contacts
        ↓
transistors: embody them without moving contacts
        ↓
integrated circuits: compose enormous numbers of gates
```

Boole did not design a CPU, Shannon did not invent every switching circuit, and a transistor alone is not a computer. The breakthrough is the composition of ideas across layers.

Part II — From a switch to a function

A switch by itself does almost nothing

Imagine a battery, lamp, and one switch:

```
power ──[ A ]── lamp ── return
```

If `A` closes, current can flow. If `A` opens, it cannot. We can choose:

```
A open   → A = 0
A closed → A = 1
lamp off → Y = 0
lamp on  → Y = 1
```

Then this circuit follows `Y = A`. It copies the logical condition.

Now ask for a different rule: “lamp on only when A and B are both active.” A pile containing two switches does not answer that question. Their **arrangement** does.

Series contacts create an AND intuition

```
power ──[ A ]──[ B ]── lamp ── return
```

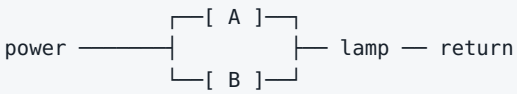
There is one path. A break at either contact stops current:

A	B	Y = A AND B
0	0	0

A	B	Y = A AND B
0	1	0
1	0	0
1	1	1

Read the table row by row. A **truth table** is a complete behavioral specification: it lists the output for every possible input combination. With two inputs there are $2^2 = 4$ rows.

Parallel contacts create an OR intuition



Either branch can complete a path:

A	B	Y = A OR B
0	0	0
0	1	1
1	0	1
1	1	1

This physical picture is useful, but do not stretch it too far. A real electronic gate is designed to produce valid output voltage levels, supply limited current to later gates, meet timing constraints, and tolerate noise. A lamp circuit is an intuition, not a transistor-level schematic.

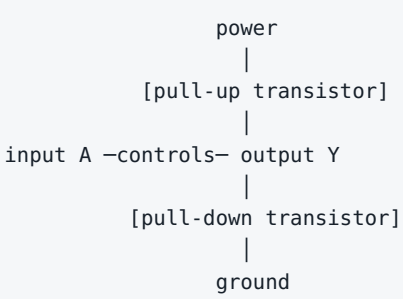
Inversion needs a complementary path

For NOT, we want the output active precisely when the input is inactive:

A	Y = NOT A
0	1
1	0

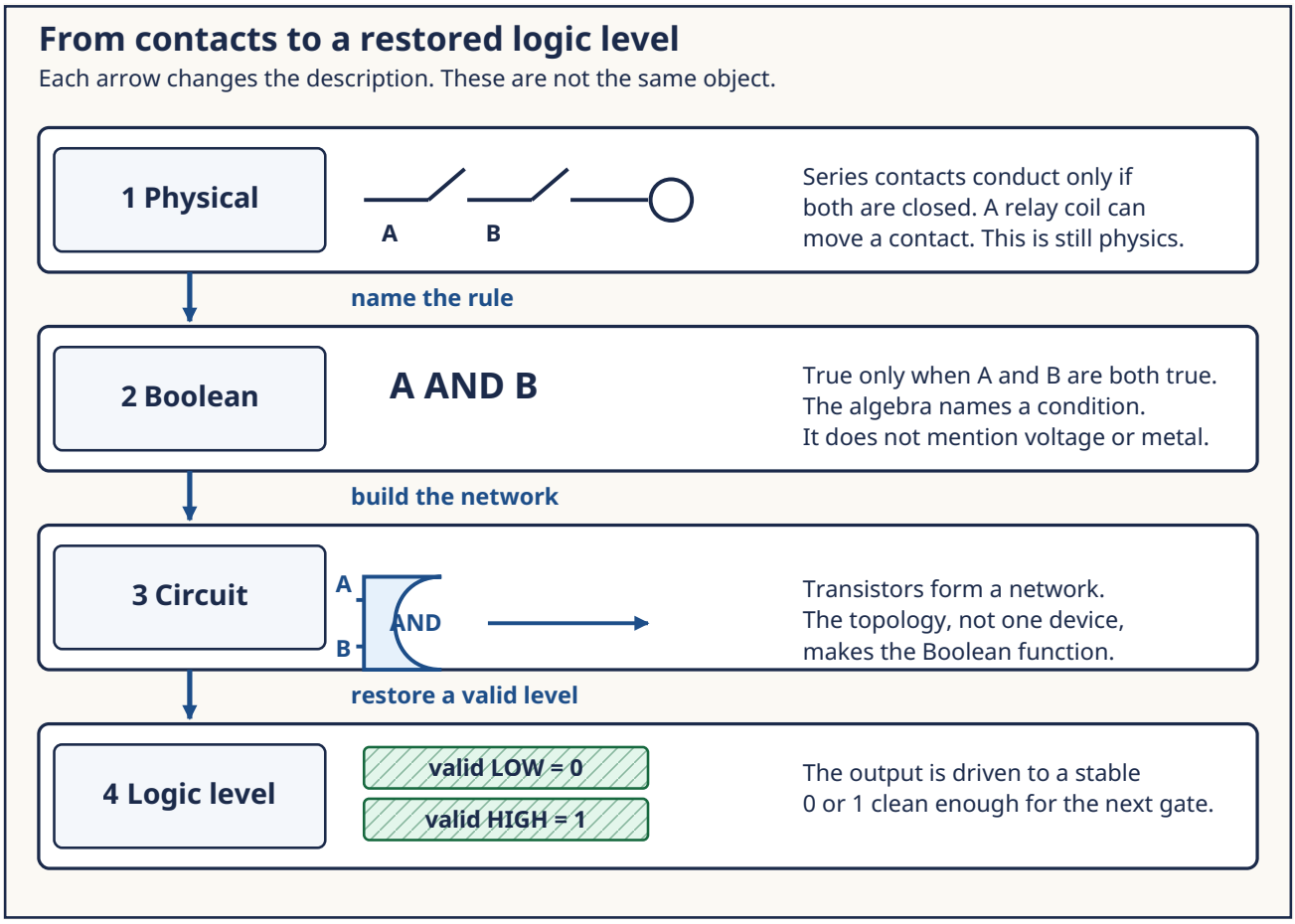
A relay can use a normally closed contact: when its coil is inactive, the contact conducts; energizing the coil opens it.

In a simplified CMOS inverter:



When `A` is LOW, the pull-up path conducts and drives `Y` HIGH. When `A` is HIGH, the pull-down path conducts and drives `Y` LOW. The devices are chosen and connected so that, in normal steady operation, one path wins and the other is off.

This is more than “blocking electricity.” The network actively drives the output toward a valid rail. That restoring behavior lets one imperfect physical signal become another clean logical signal.



A series contact, the condition A AND B, a transistor gate network, and a restored 0 or 1 are four different descriptions, linked but not the same object.

Pause and predict

Without looking ahead:

1. What arrangement of ordinary contacts naturally means AND?
2. What arrangement naturally means OR?
3. Why can NOT not be explained as merely “put another ordinary switch in series”?
4. Which parts of these descriptions are physical, and which are conventions?

Part III — Six gates, derived rather than memorized

We will use these symbols:

```
NOT A       ¬A       or !A in many programming languages
A AND B     A ∧ B    or A & B for one-bit bitwise logic
A OR B      A ∨ B    or A | B for one-bit bitwise logic
A XOR B     A ⊕ B    or A ^ B
```

Symbols vary by context. The truth table is the unambiguous definition.

NOT — opposite state

```
Y = ¬A
```

A	Y
0	1
1	0

Physical intuition: activating the control disconnects one route and enables its complement. Logical use: invert a condition, form an active-low control, or turn “permission” into “deny.”

AND — every required condition

```
Y = A ∧ B
```

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Physical intuition: series contacts. Logical use: `enabled AND ready`.

OR — at least one sufficient condition

```
Y = A ∨ B
```

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Physical intuition: parallel paths. Logical use: `interrupt_A OR interrupt_B`. OR is inclusive: both inputs being 1 still gives 1.

XOR — the inputs differ

```
Y = A ⊕ B
```

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XOR means **exactly one of two inputs** is 1, equivalently “the two bits are different.” That second phrasing scales better:

```
A ⊕ B = (A ∧ ¬B) ∨ (¬A ∧ B)
```

Physical intuition: two mutually exclusive conducting cases—A without B, or B without A—joined by OR. XOR is more complex than plain series or parallel contacts because the circuit must reject both-equal cases.

Logical uses include toggling controlled bits, parity, inequality detection, and the low result bit of one-bit addition.

NAND — AND followed by NOT

```
Y = ¬(A ∧ B)
```

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Physical intuition: the output stays active unless both controlled conditions create the one state that pulls it inactive. In CMOS, NAND has a particularly natural transistor arrangement: the pull-down path uses two nMOS devices in series, so it can pull LOW only when both inputs are HIGH; a complementary parallel pull-up network handles the other rows.

NOR — OR followed by NOT

```
Y = ¬(A ∨ B)
```

A	B	Y
0	0	1
0	1	0

A	B	Y
1	0	0
1	1	0

Physical intuition: the output is active only while neither input creates an active path. In CMOS, the nMOS pull-down devices can be parallel, so either HIGH input pulls the output LOW; the complementary pull-up requires both inputs LOW.

One table to compare them

A	B	AND	OR	XOR	NAND	NOR
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

Do not memorize this as 24 disconnected cells. Derive each column from one sentence:

- AND: all inputs are 1.
- OR: at least one input is 1.
- XOR: the inputs differ.
- NAND: not all inputs are 1.
- NOR: no input is 1.

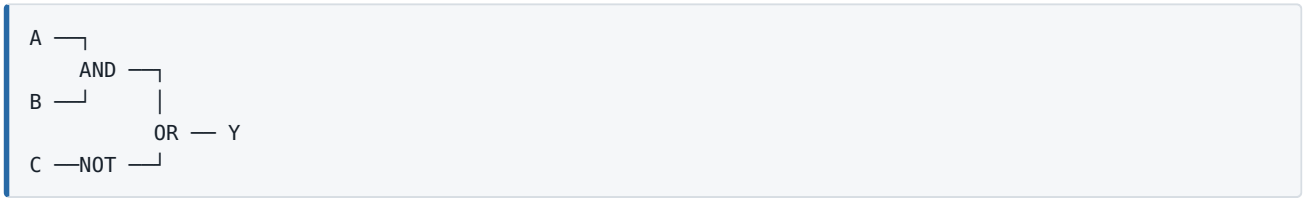
Part IV — Boolean expressions are circuit descriptions

Consider:

```
Y = (A ∧ B) ∨ ¬C
```

Read it as a recipe:

1. send `A` and `B` into an AND gate;
2. send `C` into a NOT gate;
3. OR those two intermediate results.



The wires carry intermediate Boolean values. Parentheses state grouping, just as they do in arithmetic.

Derive a truth table systematically

Name intermediate results:

```
P = A ∧ B
Q = ¬C
Y = P ∨ Q
```

A	B	C	P = A∧B	Q = ¬C	Y = P∨Q
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Three inputs produce `2³ = 8` rows. Intermediate columns turn a complicated guess into a sequence of small mechanical steps.

Expression, circuit, and truth table are three views

```
Boolean expression     compact symbolic rule
gate diagram           structural implementation
truth table            complete observable behavior
```

Different gate diagrams can implement the same truth table. This is important: hardware design is not merely finding *a* circuit; it is often finding one that meets constraints such as fewer transistors, shorter delay, lower power, or easier physical layout.

Part V — De Morgan's laws: moving inversion through a network

Two identities appear everywhere in hardware and software:

```
¬(A ∧ B) = ¬A ∨ ¬B
¬(A ∨ B) = ¬A ∧ ¬B
```

Say them in plain language:

- “not both” means “at least one is not.”
- “neither” means “the first is not and the second is not.”

Do not trust the slogan—verify all cases

A	B	¬(A∧B)	¬A∨¬B
0	0	1	1
0	1	1	1

A	B	¬(A∧B)	¬A∨¬B
1	0	1	1
1	1	0	0

The output columns match in every row, so the expressions specify the same Boolean function.

For the second law:

A	B	¬(A∨B)	¬A∧¬B
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Why hardware designers care

Suppose the available physical gate is NAND. De Morgan's laws let us reshape an expression around that primitive. They also help us reason about active-low signals, where an asserted condition is represented by LOW rather than HIGH.

The laws are not tricks that alter behavior. They prove that two different structures have the same input/output function.

Part VI — Universal gates: one kind can build every Boolean function

A gate set is **functionally complete** if it can express every Boolean function.

NAND alone is functionally complete.

Build NOT from NAND

Tie the two inputs together:

```
NAND(A, A) = ¬(A ∧ A) = ¬A
```

Build AND from NAND

Invert the NAND output using another NAND:

```
P = NAND(A, B) = ¬(A ∧ B)
AND(A, B) = NAND(P, P)
```

Build OR from NAND

Use De Morgan:

```
A ∨ B = ¬(¬A ∧ ¬B)
      = NAND(NAND(A,A), NAND(B,B))
```

Once we can build NOT, AND, and OR, we can build any truth table by combining the input cases that should produce 1.

NOR is also universal:

```
NOT A = NOR(A, A)
OR    = invert a NOR result
AND   = NOR(NOR(A,A), NOR(B,B))
```

“Universal” does not mean every chip is physically made from literal two-input NAND symbols. Real libraries contain many gate shapes optimized for electrical and layout constraints. Universality is a statement about what can be expressed.

Part VII — Combinational logic: outputs depend on inputs now

A **combinational circuit** has no intentional memory. Its output is a Boolean function of its current inputs:

```
Y = f(A, B, C, ...)
```

Change an input and physical effects propagate through the gates. After the finite propagation delay, the outputs settle to the new function value.

Examples:

- a decoder selects one output from an encoded input;
- a multiplexer chooses one data input;
- a comparator reports equality or ordering;
- an adder computes result and carry bits;
- an ALU combines arithmetic and logic functions.

The word “now” is an abstraction. Real transistors do not respond instantaneously. During transitions, intermediate outputs may briefly differ from the final truth-table result. Later we will care about propagation delay, hazards, clocks, and state. For Boolean reasoning today, we analyze settled values.

What combinational logic cannot do

If every input returns to its old value, a pure combinational circuit returns to its old output. It cannot remember that something happened earlier.

Memory requires another idea—feedback and controlled sampling. Do not smuggle memory into today's model.

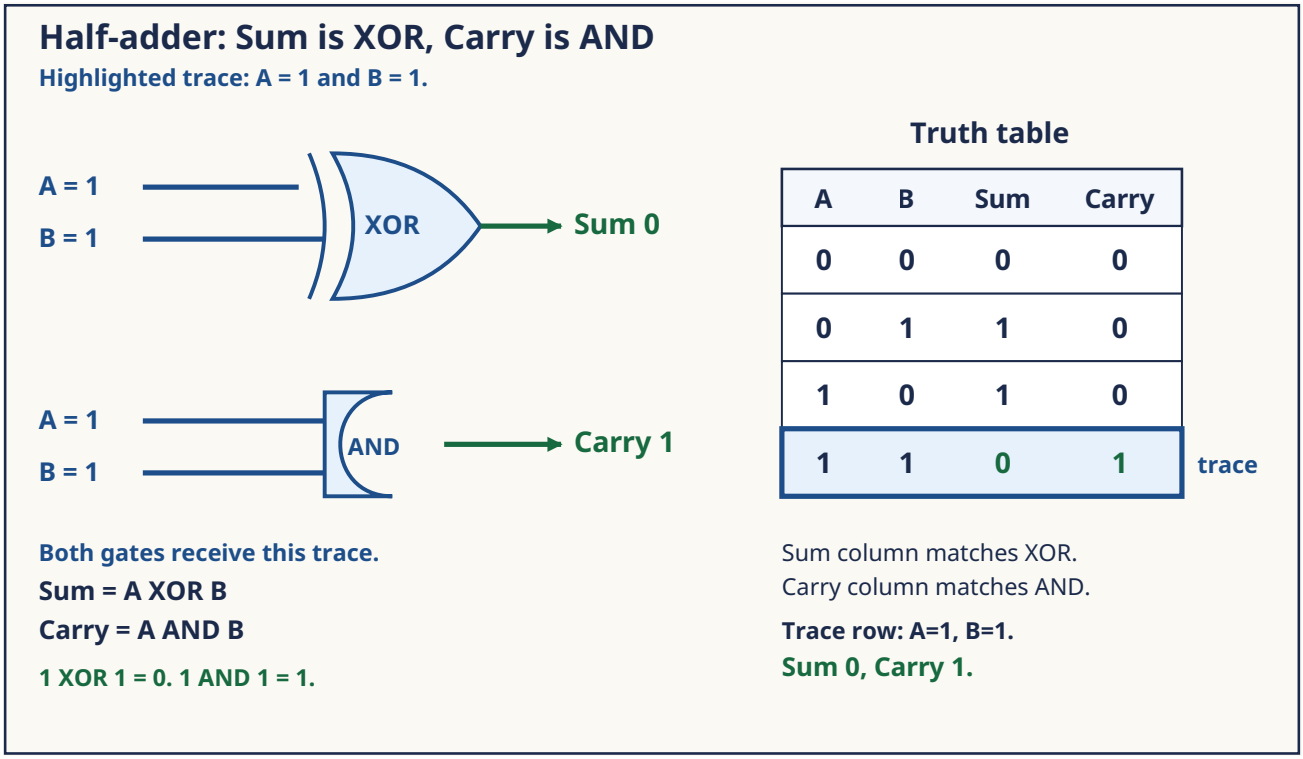
Part VIII — A glimpse of arithmetic: the half-adder

Add two one-bit numbers:

A	B	binary total	Sum	Carry
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	10	0	1

Compare the output columns with the gate tables:

```
Sum   = A XOR B
Carry = A AND B
```



For the highlighted inputs A=1 and B=1, the XOR gate produces Sum 0 and the AND gate produces Carry 1, matching that row of the truth table.

This is a **half-adder**. It adds two bits, but it does not accept a carry from a less-significant position. That limitation is exactly why it is only a preview.

Later we will ask:

- How do we include carry-in?
- How do many one-bit stages form a wider adder?
- How long does carry take to propagate?
- How does a circuit remember operands or results?

Those questions lead to full adders, registers, timing, and storage. Stop here for now and preserve that conceptual space.

Part IX — From gates toward an ALU and CPU

An arithmetic logic unit does not “know arithmetic” in a human sense. It contains circuits whose physical topology implements useful Boolean functions.

One route toward a simplified ALU is:

```
inputs A and B
    ├── AND network ──────┐
    ├── OR network ───────┤
    ├── XOR network ──────┼── selector ── result
    └── adder network ────┘
                control bits choose a path
```

Replicate one-bit slices across a word, add carry connections and status logic, and the machinery can operate on multi-bit values. Put registers around the datapath, add a clock and control logic, and we approach a CPU execution unit.

But preserve the layers:

```
transistors do not see integers
gates do not see C variables
the ALU does not understand "salary" or "pixel"
the CPU does not understand the purpose of your program
```

Each layer follows local physical and encoding rules. Meaning is supplied by the system built above it.

Part X — Hands-on Logic Gate Lab

Open this local, offline challenge in a browser:

`01-foundations-to-cpu/challenges/day-003-logic-gate-lab.html`

Use this loop:

1. **Predict** an output before touching the controls.
2. **Set** the input switches.
3. **Observe** the gate and truth-table row.
4. **Explain** the result in one sentence.
5. **Build** the requested expression and complete the scored challenge.

The lab deliberately hides challenge answers until you submit an attempt. Explore mode may show gate behavior, but do not use it to bypass your prediction.

Suggested time:

- 20 minutes: gate explorer and physical switch intuition;
- 20 minutes: expression builder and De Morgan comparison;
- 25 minutes: scored prediction rounds.

Part XI — Observe the same functions in Python

This optional exercise makes a useful distinction: Python simulates the truth table in software; a physical gate realizes it through electrical behavior.

Predict the rows before running:

```
python3 - <<'PY'
def bit_not(a):
    return 1 - a

def gates(a, b):
    return {
        "NOT A": bit_not(a),
        "AND":   a & b,
        "OR":    a | b,
        "XOR":   a ^ b,
        "NAND":  bit_not(a & b),
        "NOR":   bit_not(a | b),
    }

for a in (0, 1):
    for b in (0, 1):
        print(a, b, gates(a, b))
PY
```

Now verify De Morgan's laws exhaustively:

```
python3 - <<'PY'
for a in (0, 1):
    for b in (0, 1):
        law1 = (not (a and b)) == ((not a) or (not b))
        law2 = (not (a or b)) == ((not a) and (not b))
        print(f"A={a} B={b}  law1={int(law1)} law2={int(law2)}")
PY
```

Every line should report both laws as true. The program did not prove the laws for arbitrary multi-valued systems; it enumerated every state in this two-valued domain.

Optional small C observation

```
#include <stdio.h>

int main(void)
{
    puts("A B | AND OR XOR NAND NOR");
    for (int a = 0; a <= 1; ++a) {
        for (int b = 0; b <= 1; ++b) {
            printf("%d %d |  %d   %d   %d    %d    %d\n",
                   a, b, a && b, a || b, a != b,
                   !(a && b), !(a || b));
        }
    }
    return 0;
}
```

Build with warnings enabled:

```
gcc -std=c17 -Wall -Wextra -O0 -g logic_table.c -o logic_table
./logic_table
```

In C, `&&`, `||`, and `!` are logical operators with short-circuit behavior; `&`, `|`, `^`, and `~` operate bitwise on integer representations. For inputs restricted to 0 and 1, several result columns coincide, but the language semantics are not interchangeable in general.

Part XII — Mastery checkpoint

Close the lesson and answer:

> **How can a physical circuit implement a Boolean function without understanding it?**

A strong answer should connect all of these:

1. Input meanings are encoded as physical voltage ranges.
2. Transistors or relay contacts respond locally to physical conditions.
3. Their arrangement constrains which conducting paths exist for each input combination.
4. Those paths drive an output into a physical state interpreted as 0 or 1.
5. The resulting input/output mapping matches a truth table.
6. No component needs a concept of the application-level meaning; the “function” is our abstract description of deterministic physical behavior.

Do not memorize that list. Rebuild the explanation from a concrete example, such as series contacts implementing AND.

Explain

Without notes:

- derive NOT, AND, OR, XOR, NAND, and NOR;
- explain why OR is inclusive and XOR is different;
- state both De Morgan laws in plain language;
- explain why NAND or NOR can be universal.

Draw

On one page:

```
two physical inputs
    → gate network
    → output truth table
    → half-adder glimpse
    → ALU path
```

Include one series-contact AND and one parallel-contact OR.

Observe

- complete at least one prediction set in the HTML lab;

- run either the Python or C truth-table program;
- record one prediction that was wrong and why.

Build

Using only NAND, draw:

1. NOT;
2. AND;
3. OR.

Then derive the truth table of:

```
Y = (A XOR B) AND (NOT C)
```

Do not consult a generated answer until every one of the eight input rows has an attempted output.

Mental model at the end of Day 3

```
continuous electrical behavior
        ↓ threshold convention
logical input bits
        ↓ transistor network topology
gate behavior
        ↓ composition
Boolean expression / combinational circuit
        ↓ repeated one-bit structures
arithmetic, comparison, selection
        ↓ state + control added later
ALU and CPU
```

The essential sentence is:

> A Boolean function is not a thought occurring inside the circuit. It is our compact description of how an arranged physical network maps every allowed input state to an output state.

Why? notebook

1. Why is one isolated switch a state-holder or path-controller, but not yet a general computer?
2. Where exactly does AND “exist”: in the expression, truth table, circuit topology, voltage behavior, or all of these at different layers?
3. Why must a gate restore its output toward valid voltage ranges?
4. Why can two structurally different circuits implement the same function?
5. Why does functional completeness not imply that every real chip uses only one physical gate type?

6. Why can a half-adder not yet add arbitrary multi-bit numbers?
7. What new capability is needed before a circuit can remember a previous result?

References used selectively

Local reading:

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 10–12, from “Logic and Switches” through the opening of “A Binary Adding Machine” (local PDF pages 98–145). Local catalog entry: `references/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf`
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, Chapter 3, §§3.1–3.3 and §3.3.5, on transistor switches, gates, De Morgan's law, combinational logic, and logical completeness (book pages 51–64). Local catalog entry: `references/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf`

Historical and primary context:

- University of St Andrews, MacTutor biography of George Boole and the development of algebraic logic: https://mathshistory.st-andrews.ac.uk/Biographies/Boole/
- MIT Libraries, Claude Shannon's thesis, *A Symbolic Analysis of Relay and Switching Circuits*: https://dspace.mit.edu/handle/1721.1/11173
- Computer History Museum, the Bell Labs search for a semiconductor replacement for relays and vacuum tubes: https://www.computerhistory.org/siliconengine/invention-of-the-point-contact-transistor/
- NAND2Tetris, Boolean logic and gate-construction material: https://www.nand2tetris.org/course

Next bridge: combine gates into wider arithmetic and selection circuits, then ask what physical mechanism can preserve a result after the inputs change.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Boolean Functions Lab

Format: Python lab

What you will do: Week 1 Day 3 coding challenge: derive gates and universal NAND logic.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
python3 01-foundations-to-cpu/challenges/day-003-boolean-functions-lab.py
```

- View lab source on GitHub

Put it into practice — Logic Gate Lab

Format: interactive HTML lab

What you will do: Day 3 — Logic Gate Lab

- Open the interactive lab
- View lab source on GitHub

Day 4 — Adders, Selection, and Memory

Target time: approximately 3 hours

- Whiteboard derivation: about 75 minutes
- Digital Circuits Lab: about 75 minutes
- Draw/build evidence and explanation from memory: about 30 minutes

> Gates can transform bits. But a useful machine must do three more specific things: calculate a result, choose which result matters, and keep that result after the inputs move on.

That is today's path:

```
gates → adder → selector → stored state → register-transfer datapath
```

The previous lesson ended with `Sum = A XOR B` and `Carry = A AND B`. We will rebuild that result rather than merely inherit it, repair its missing carry-in, connect stages into a four-bit adder, then ask two questions that arithmetic alone cannot answer:

1. If several circuits produce values, which one reaches the destination?
2. If the inputs disappear, what keeps the chosen value available?

Those questions lead naturally to multiplexers and storage. They are also the first recognizable pieces of a CPU datapath.

---

1. Why engineers needed these circuits

Mechanical calculators used gears and wheels to embody place-value arithmetic. One wheel passing its final position pushed a carry into the next wheel. The rule was visible, but moving parts imposed friction, wear, size, and speed limits.

Relay calculators replaced much of that motion with electrically controlled contacts. Relays could implement Boolean relationships, but their armatures still moved. Vacuum tubes and then transistors removed the mechanical contact from the switching path. Integrated circuits made it practical to repeat a small logic pattern thousands, then millions, then billions of times.

The engineering need did not change:

```
represent a digit
    → combine digits according to arithmetic
    → carry information between positions
    → choose an operation or source
    → hold intermediate results
    → coordinate a sequence of steps
```

A fast combinational adder by itself is not a programmable computer. It continuously reacts to whatever is on its inputs. It has no opinion about whether its output should be used, and no memory of the previous answer. Selection and storage turn a collection of functions into a datapath that can perform a sequence.

---

2. Derive the half-adder from ordinary addition

Add every possible pair of one-bit unsigned numbers:

A	B	binary total	Sum	Carry
0	0	`00`	0	0
0	1	`01`	1	0
1	0	`01`	1	0
1	1	`10`	0	1

The low result bit is 1 when the inputs differ:

```
Sum = A XOR B
```

The high result bit is 1 only when both inputs are 1:

```
Carry = A AND B
```

Together those gates form a **half-adder**:

```
A ──┬─── XOR ── Sum
    │
B ──└─── AND ── Carry
```

Why “half”? It handles the two bits belonging to one column, but not the carry arriving from the column to its right. It works at the least significant bit of a multi-bit addition, where there is normally no incoming carry. Every later column needs a third input.

Pause and derive

Do not quote the equations. Explain why `1 + 1` produces Sum 0 and Carry 1. Then point to the physical output wires on which those two facts would be represented.

---

3. The missing input creates the full-adder

A general bit position receives:

- `A`, one bit of the first operand;
- `B`, one bit of the second operand;
- `Cin`, carry from the less-significant stage.

It produces:

- `Sum`, the result bit for this position;

- `Cout`, carry to the more-significant stage.

Write all eight cases:

A	B	Cin	total	Sum	Cout
0	0	0	0	0	0
0	0	1	1	1	0
0	1	0	1	1	0
0	1	1	2	0	1
1	0	0	1	1	0
1	0	1	2	0	1
1	1	0	2	0	1
1	1	1	3	1	1

The Sum is 1 when an odd number of inputs are 1:

```
Sum = A XOR B XOR Cin
```

Carry-out is 1 when at least two inputs are 1—the majority function:

```
Cout = (A AND B) OR (Cin AND (A XOR B))
```

Another equivalent form is:

```
Cout = (A AND B) OR (A AND Cin) OR (B AND Cin)
```

Build it from two half-adders

```
P  = A XOR B
C1 = A AND B

Sum = P XOR Cin
C2  = P AND Cin

Cout = C1 OR C2
```

```
A,B ── half-adder ── P,C1
 P,Cin ─ half-adder ── Sum,C2
 C1,C2 ──────── OR ──────── Cout
```

This is a **full-adder**. Its name does not mean it adds a whole machine word. It means one bit position now has the full interface required to participate in a chain.

Carry generate and propagate

Two intermediate ideas make wider adders easier to reason about:

```
G = A AND B       this position generates a carry
P = A XOR B       this position propagates an incoming carry
Cout = G OR (P AND Cin)
```

If `G = 1`, this position creates a carry regardless of `Cin`. If `P = 1` and `G = 0`, the outgoing carry follows the incoming carry. If both operand bits are 0, the chain stops.

---

4. Ripple-carry: repeat a small correct stage

Connect each `Cout` to the next stage's `Cin`:

```
                c1             c2             c3
Cin=0 ─→ [FA bit 0] ─→ [FA bit 1] ─→ [FA bit 2] ─→ [FA bit 3] ─→ Cout
          A0 B0          A1 B1          A2 B2          A3 B3
            |              |              |              |
           S0             S1             S2             S3
```

This is a four-bit **ripple-carry adder**. It is regular, easy to build, and correct after its signals settle.

The qualification matters. Gates have propagation delay. Stage 1 cannot know its final `Cin` until stage 0 produces `c1`; stage 2 may wait for `c2`, and so on. A carry may ripple from the least significant position through every stage. Intermediate sum bits can briefly show non-final values.

For example:

```
  0111
+ 0001
------
  1000
```

The low position generates a carry. At each following position, `A XOR B = 1`, so the carry propagates through. In a simple timing model, the most significant answer cannot be trusted until that path settles.

This is why wider arithmetic creates a timing problem. A 64-bit ripple-carry adder is not wrong; its worst-case path is simply long. Faster designs predict or combine carry information using carry-lookahead, carry-select, prefix, or other structures. Those designs spend more circuitry and wiring to reduce delay. The deep trade-off is:

```
simple repeated structure ↔ shorter critical path
```

---

5. Trace 3 + 5 all the way to 8

The tracker checkpoint asks more than binary arithmetic. We must cross every layer.

Meaning and encoding

We choose unsigned four-bit binary:

```
3 → 0011
5 → 0101
8 → 1000
```

That mapping is an encoding convention. The wires themselves do not carry “three” or “five.”

Logical inputs

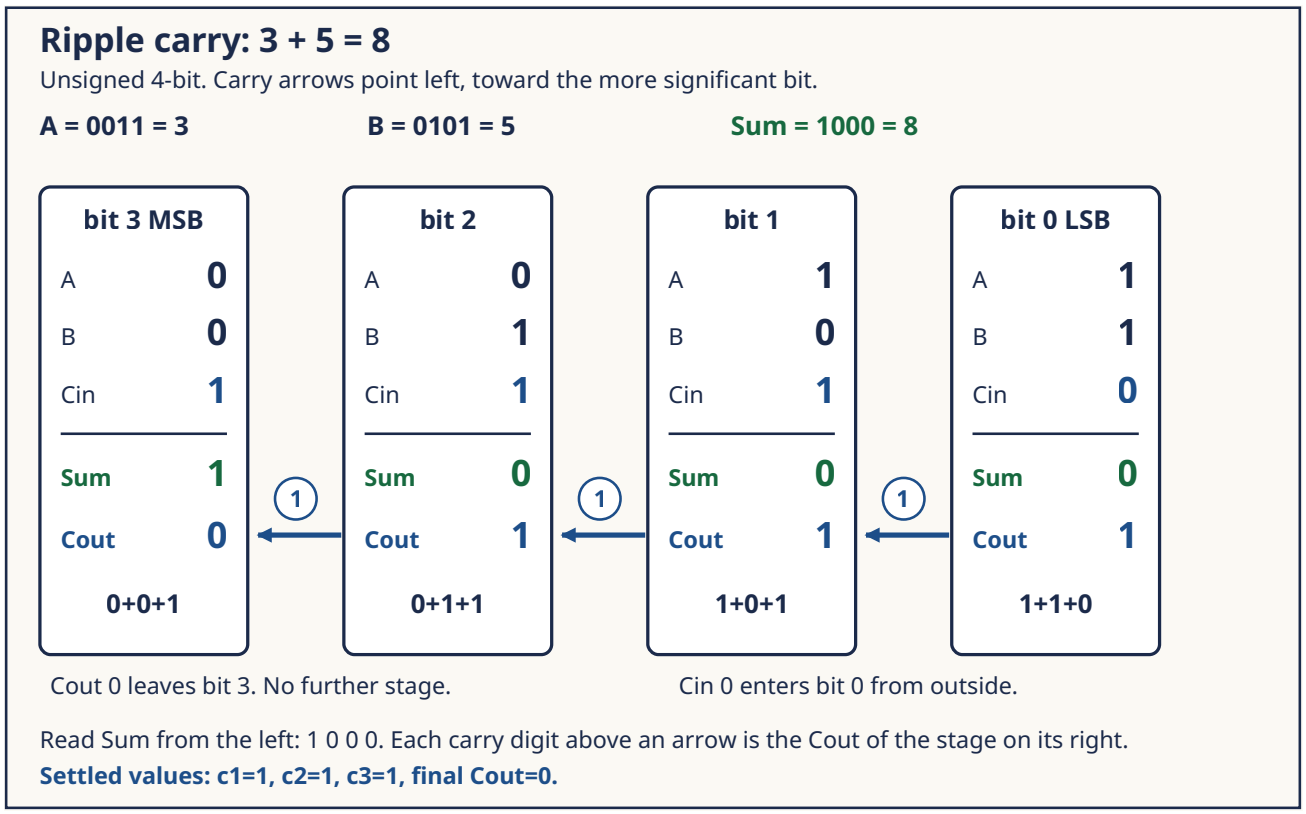
Write bits least significant first for the stage trace:

```
A = 0011 → A3 A2 A1 A0 = 0 0 1 1
B = 0101 → B3 B2 B1 B0 = 0 1 0 1
Cin to bit 0 = 0
```

bit	A	B	Cin	Sum	Cout	reason
0	1	1	0	0	1	two ones generate carry
1	1	0	1	0	1	operand difference propagates carry
2	0	1	1	0	1	operand difference propagates carry
3	0	0	1	1	0	incoming carry becomes this sum bit

Read `S3 S2 S1 S0`:

```
1000₂ = 8
```



Each stage shows its bits for unsigned 3 + 5, and each carry arrow points toward the more significant bit until the settled sum is 1000.

Physical implementation

In one possible CMOS implementation:

1. Driver circuits establish voltages on eight operand wires.
2. Receivers classify those voltages as logical `0011` and `0101`.
3. Transistor networks arranged as XOR, AND, and OR gates respond locally.
4. Internal wire voltages representing carry change and propagate from bit 0 toward bit 3.
5. After the worst-case combinational delay, output wires settle to voltage ranges classified as `1000`.
6. A destination register may sample those outputs at an allowed clock edge.
7. Under the chosen unsigned interpretation, software or hardware treats `1000` as eight.

The arithmetic is our abstract description of that causal physical behavior. No transistor sees the symbols `3`, `+`, `5`, or `8`.

What the clock did not do

The clock did **not** cause every transistor in the adder to switch. The combinational gates respond whenever their input voltages change, and many transistors do not change state for a particular operation. In a synchronous datapath, the clock defines when storage elements are permitted to capture a settled result. It provides timing discipline, not a universal command that all transistors toggle.

---

6. Selection: one destination, several possible sources

Suppose an ALU has already computed:

```
A AND B
A OR B
A + B
```

Connecting all three outputs directly to the same destination wire would create electrical contention, not a useful choice. The datapath needs a controlled selector.

The 2-to-1 multiplexer

A **multiplexer**, or mux, chooses one data input:

```
S = 0 → Y = D0
S = 1 → Y = D1
```

S	D0	D1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

`X` means “do not care for this row,” not an unknown electrical state.

Derive the expression:

```
Y = (NOT S AND D0) OR (S AND D1)
```

When `S=0`, the first path is enabled and the second suppressed. When `S=1`, the roles reverse. A word-wide mux repeats that one-bit selection for every bit while sharing the select signal.

Muxes answer “which value proceeds?” They appear in ALU output selection, register input paths, operand forwarding, program-counter choice, and memory interfaces.

Decoder: encoded choice becomes one-hot permission

A **decoder** maps an `n`-bit code to one of up to `2ⁿ` output lines. A 2-to-4 decoder behaves like this when enabled:

A1	A0	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

The outputs are **one-hot**: exactly one is asserted. A register file can decode a destination number into one write-enable line. An instruction decoder can turn opcode fields into control conditions. Real decoders often also have an enable input.

Encoder: asserted line becomes a code

An **encoder** performs the conceptual reverse:

```
one of several input lines → compact binary code
```

A plain encoder assumes only one input is asserted. If several can be active, the output is ambiguous. A **priority encoder** adds a rule—perhaps the highest-numbered asserted input wins—and often a valid output indicating that some request exists.

Keep the jobs distinct:

```
mux      many data values → one selected data value
decoder  compact code     → one-hot control lines
encoder  one-hot/request  → compact code
```

---

7. Combinational versus sequential logic

A combinational circuit's settled output depends only on current inputs:

```
output = f(current inputs)
```

An adder, mux, decoder, and encoder are combinational. Remove or change the inputs, wait for propagation, and their outputs follow.

A sequential circuit's behavior also depends on stored state:

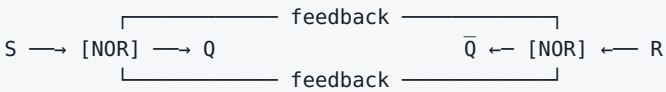
```
next state = f(current state, current inputs)
output     = g(current state, current inputs)
```

The word *sequential* does not merely mean “gates drawn in a sequence.” It means past activity can affect current behavior because some physical state persists.

---

8. Why feedback can create state

Feed a combinational output back into an input and the circuit can become self-reinforcing. Two cross-coupled NOR gates illustrate the idea:



In the idle condition `S=0, R=0`, suppose `Q=1` and $\overline{Q}$`=0`. Each output helps force the other gate to preserve that arrangement. If Set is pulsed, the network moves to the set state; after Set returns inactive, feedback maintains it. The current input alone no longer determines the output. History matters.

This is an **SR latch**. For a NOR-based active-high form:

S	R	next behavior
0	0	hold previous state
1	0	set `Q=1`
0	1	reset `Q=0`
1	1	forbidden/problematic condition

The final row forces both outputs low, violating the normal complementary relationship. If S and R then return low nearly together, small physical delay differences can decide the resulting state; the circuit may also take time to resolve. Do not describe this row as a useful third stored value.

NAND-based SR latches commonly use active-low inputs and therefore have a different-looking table. Always state the circuit and signal polarity.

Metastability is physical, not a third logic value

If storage timing requirements are violated, an internal node can temporarily hover between valid logical levels before resolving. Digital diagrams hide that analog behavior, but hardware cannot. Designers specify setup, hold, and recovery timing and use synchronizer structures when crossing asynchronous boundaries. No synchronous design can promise that arbitrary asynchronous input timing is physically harmless.

---

9. From SR to D storage

Most datapaths do not want callers to coordinate separate Set and Reset inputs. They want one data input:

```
store D when permitted; otherwise retain the old Q
```

D latch: level-sensitive

A D latch uses gating so the forbidden SR request is not intentionally generated.

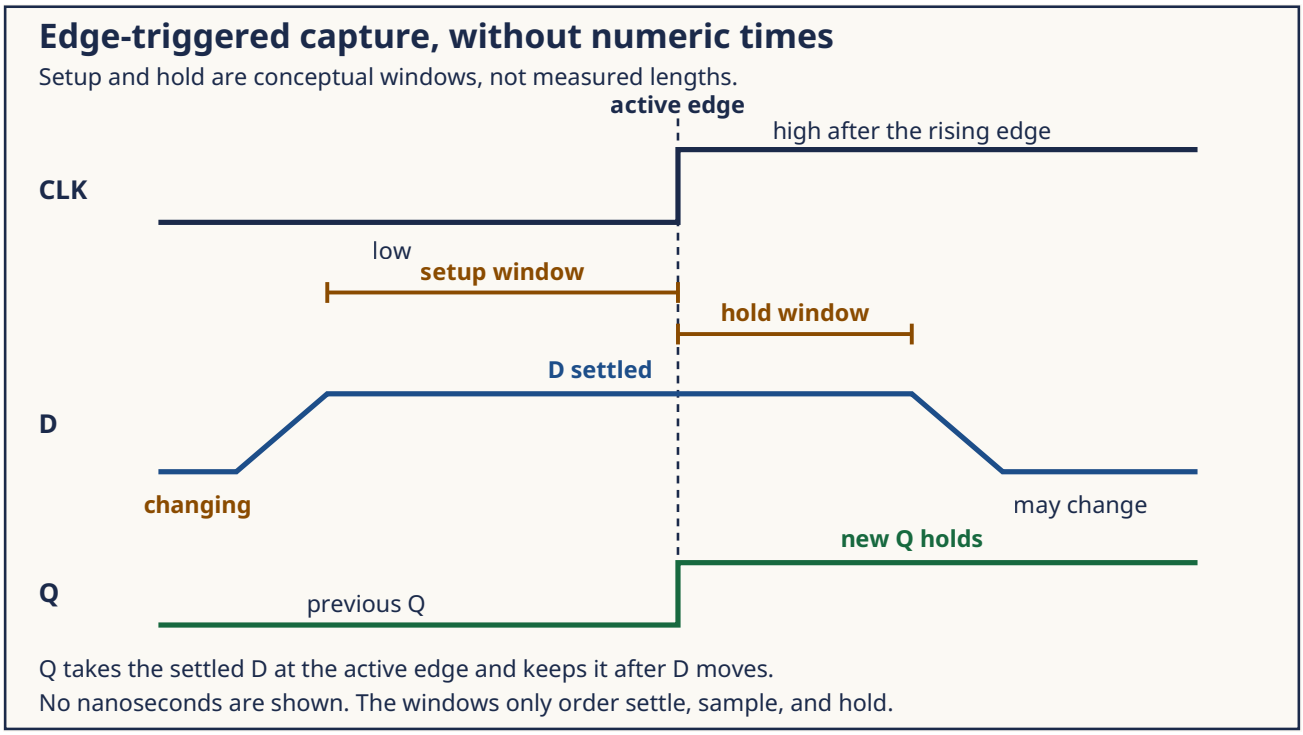
```
Enable = 1 → transparent: Q follows D after propagation delay
Enable = 0 → closed: Q holds the last captured value
```

Calling the latch “transparent” matters. While Enable remains active, several changes at D may pass through to Q. It does not capture only at the instant Enable rises.

D flip-flop: edge-triggered

An edge-triggered D flip-flop samples D around a specified clock transition, such as the rising edge. After its clock-to-Q delay, Q reflects the sampled value and remains stable until a later active edge.

```
before edge: D must satisfy setup time
at edge:     sampling event
after edge:  D must satisfy hold time
later:       Q changes after clock-to-Q delay
```



D is already stable in the setup window, Q takes that value at the active edge and holds it, and D stays stable through a hold window that has no numeric duration.

“At the edge” is a useful logical model, not a claim of zero-duration or perfect simultaneity. Real storage has an aperture defined by setup and hold requirements.

Conceptually:

```
D latch       level-sensitive while enabled
D flip-flop   edge-triggered around a clock transition
```

Some implementations construct an edge-triggered flip-flop from two opposite-phase latches; other circuit styles exist. The behavioral contract is what the surrounding datapath relies on.

---

10. Why a clock exists

Combinational signals do not settle simultaneously. A synchronous design creates a timing contract:

```
source register
    → clock-to-Q delay
    → combinational logic and routing delay
    → destination setup requirement
    → next active clock edge
```

The clock period must be long enough for the worst allowed path, plus timing margin and clock-distribution effects. Hold-time constraints impose a minimum path requirement after an edge.

The clock provides regular sampling opportunities and divides a large sequential computation into stages. It does not make propagation delay disappear, guarantee arbitrary asynchronous inputs are safe, or command all transistors to switch.

Clock networks themselves consume energy and need careful distribution. Real systems may gate clocks, use multiple clock domains, or include asynchronous blocks. “A CPU has a clock” is the beginning of timing analysis, not its end.

---

11. Registers: store a word, not just one bit

A **register** groups storage elements so several bits are captured under a shared control:

```
D3 D2 D1 D0
 |  |  |  |
[FF][FF][FF][FF]  ← shared clock
 |  |  |  |
Q3 Q2 Q1 Q0
```

Four D flip-flops can hold a four-bit pattern. Practical registers may add reset, enable, scan, or other controls.

Load enable is selection before storage

If a register should retain its value when `load=0` and capture `input` when `load=1`, place a mux before each D input:

```
next = load ? input : current
```

```
external input ──┐
                 MUX ──→ D flip-flop ──→ Q/current
current Q ───────┘       select=load
```

Feedback supplies the hold value; the mux controls whether new or old data reaches D; the edge-triggered storage determines when Q may update.

Design a two-bit register

Use two identical bit slices sharing `clock` and `load`:

```
D1 ─┐                            ┌─→ Q1
Q1 ─┴─ [2:1 mux, select=load] ─→ [DFF]

D0 ─┐                            ┌─→ Q0
Q0 ─┴─ [2:1 mux, select=load] ─→ [DFF]
                                      ↑
                                same active clock edge
```

Next-state equations:

```
Q1_next = load ? D1 : Q1
Q0_next = load ? D0 : Q0
```

State-transition table:

current Q1Q0	load	input D1D0	next Q1Q0 at active edge
`00`	0	`11`	`00`
`00`	1	`10`	`10`
`10`	0	`01`	`10`
`10`	1	`11`	`11`
`11`	1	`00`	`00`

Between active edges, changing D or load does not immediately change Q in this edge-triggered model. At an edge, both bits conceptually capture together, provided timing requirements are met.

Draw the complete state graph yourself. It has four nodes—`00`, `01`, `10`, `11`. With `load=0`, each node has a self-loop. With `load=1`, each node can transition to the input-selected destination.

Register storage is not every kind of memory

A flip-flop stores one logical bit under a timing contract; a register groups flip-flops so a word can be captured together. That is the right model for CPU state in this lesson, but large memories use different physical organizations. An SRAM cell uses a small feedback circuit to retain a bit while powered and is addressed through array wiring. A DRAM cell represents a bit with charge on a tiny capacitor; leakage and reading require sensing and periodic refresh. SRAM is generally faster and less dense than DRAM, while DRAM trades more complex sensing and refresh for much greater density.

These are qualitative boundaries, not circuit schematics. The lab models logical values, enables, clock edges, and state transitions. It does not model analog node voltages, stored charge, leakage, sense amplifiers, refresh, or the timing aperture of a physical storage element.

---

12. Put the pieces around one addition

Now we can design a tiny register-transfer step:



The roles are sharply separated:

- registers preserve operand and result patterns;
- the adder continuously forms a combinational sum;
- a mux selects which candidate value reaches the destination D inputs;
- decoder/control logic supplies select and load signals;
- an active clock edge makes the chosen next state become stored state.

For `3 + 5`, operand registers expose `0011` and `0101`. Carries propagate until the adder settles at `1000`. Control selects the adder output and enables the result register. At the permitted edge—after timing constraints are satisfied—the result register captures `1000`.

This is already the shape of a CPU datapath. What is still missing is a control mechanism that interprets instruction bits, chooses registers and operations, and updates a program counter in the correct sequence. That is the next bridge.

---

13. Offline Digital Circuits Lab

Open:

`01-foundations-to-cpu/challenges/day-004-digital-circuits-lab.html`

Use it in this order:

1. **Half/full adder:** derive Sum and Carry before toggling inputs.
2. **Four-bit ripple adder:** enter two values, predict the result, then step across the visible carry chain.
3. **Mux/decoder:** predict the selected data and one-hot decoder line.
4. **D storage/register:** stage D and load values, then apply a clock edge. Observe that Q does not follow D between edges.
5. **Scored predictions:** answers remain hidden until each attempt.

The lab is a logical simulation. It does not model transistor analog behavior, metastability, or exact nanosecond timing. Its carry animation shows causal stage order, not a calibrated physical delay.

---

14. Evidence: Explain, Draw, Observe, Build

Explain

Without notes:

1. Derive half-adder Sum and Carry from the four addition cases.
2. Explain why a full-adder needs `Cin` and why `Cout` is a majority function.
3. Explain what “carry propagation” means both logically and physically.
4. Distinguish mux, decoder, and encoder by the direction of information flow.
5. Explain why feedback lets past state affect present output.
6. Distinguish a level-sensitive D latch from an edge-triggered D flip-flop.
7. Explain what a clock coordinates without claiming that it switches every transistor.

Draw

On one uninterrupted page, draw:

```
half-adder
    → full-adder
    → four-stage ripple adder
    → mux
    → two-bit enabled register
    → result path in a CPU-like datapath
```

Label `Cin`, `Cout`, `select`, `load`, `D`, `Q`, and the active clock edge.

Observe

In the lab:

- find an addition whose carry crosses all four stages;
- record each `Cin`, Sum, and `Cout`;
- show that changing register D does not change Q before a clock edge;
- show one decoder input and its one-hot output;
- record one prediction that differed from observation and repair the mental model.

Build

Complete both:

1. Build and document a four-bit ripple addition, including the carry chain.
2. Design the two-bit register above and write at least six state transitions, including hold, load, overwrite, and return-to-zero cases.

Passing the HTML score is useful evidence, but it cannot replace the drawing or verbal explanation.

---

15. Mastery checkpoint: 3 + 5, from physics to 8

Close everything and answer:

> How are 3 and 5 transformed into electrical states and how does the machine produce and retain 8?

A complete trace must include:

1. unsigned encoding: `3=0011`, `5=0101`;
2. physical voltage/charge states implementing those logical bits;
3. one-bit full-adder behavior at all four positions;
4. carry sequence `c1=1`, `c2=1`, `c3=1`, final `Cout=0`;
5. settled output `1000`;
6. mux/control selection of the adder output;
7. capture by a destination register at an allowed edge;
8. interpretation of `1000` as unsigned 8;
9. the distinction between combinational propagation and clocked storage.

If you say “the clock makes the adder calculate,” repair the explanation. The input transition makes combinational circuitry respond. The clock coordinates when a storage element samples a result that must already satisfy timing requirements.

Why? notebook

Write before looking back:

1. Why is a half-adder insufficient everywhere except a position with no incoming carry?
2. Why can a ripple adder be logically correct but too slow for a target clock period?
3. Why is a mux not merely an OR gate?
4. Why does a decoder need a one-hot output, and what could use it?
5. Why is a plain encoder ambiguous when several inputs are active?
6. Why does feedback change a circuit from a current-input function into a stateful system?
7. Why is the forbidden SR condition a physical timing concern, not just an inconvenient row in a table?
8. Why can a D latch pass several changes while a D flip-flop captures at an edge?
9. Why must data settle before an active edge and remain stable briefly after?
10. Why does an enabled register contain both a selector and storage?
11. Where does the value 8 “exist” physically, logically, and semantically?
12. Which new machinery is still needed to turn this datapath into a CPU that follows instructions?

Mental model at the end

```
physical voltage ranges
    → logical operand bits
    → combinational gates
    → full-adder carry chain
    → settled sum bits
    → mux-selected next value
    → edge-triggered register state
    → later operations consume that state
```

The essential distinction is:

> Combinational logic transforms present inputs after propagation delay. Sequential logic preserves state, so past events can influence the present. A synchronous datapath uses clocks to coordinate when storage samples combinational results; it does not make all transistors switch together.

References used selectively

Local reading:

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 12–16, for binary addition, feedback, relays, selection, and the construction of larger digital systems. Local catalog path: `references/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf`
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd edition, Chapter 3, especially the sections on combinational circuits, decoders, muxes, adders, storage elements, and finite-state behavior. Local catalog path: `references/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf`

Historical and technical context:

- Claude E. Shannon, *A Symbolic Analysis of Relay and Switching Circuits*, MIT Libraries: https://dspace.mit.edu/handle/1721.1/11173
- Computer History Museum, “The Transistor and Portable Electronics,” for the transition from relay/vacuum-tube constraints to solid-state switching: https://www.computerhistory.org/siliconengine/invention-of-the-point-contact-transistor/
- Nand2Tetris, sequential-logic and computer-construction materials: https://www.nand2tetris.org/course
- MIT OpenCourseWare, *Computation Structures*, digital logic and sequential circuit materials: https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

These sources ground the circuit definitions and historical chain. The whiteboard derivation, 3 + 5 trace, state-transition exercise, and integrated datapath narrative are synthesized for this masterclass.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Digital Circuits Lab

Format: interactive HTML lab

What you will do: Day 4 — Digital Circuits Lab

- Open the interactive lab
- View lab source on GitHub

Put it into practice — Four Bit Adder Lab

Format: Python lab

What you will do: Week 1 Day 4 coding challenge: build addition from one-bit stages.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
python3 01-foundations-to-cpu/challenges/day-004-four-bit-adder-lab.py
```

- View lab source on GitHub

Day 5 — Building a Minimal CPU

Curriculum alignment: Stage 2 — CPU Architecture and Instruction Execution · The Minimal CPU

Target time: approximately 3 hours

- Historical problem and stored programs: about 25 minutes
- Clock, state, datapath, and control: about 55 minutes
- Tiny ISA design and encoding: about 35 minutes
- Cycle-by-cycle trace: about 30 minutes
- Python build, observation, and mastery work: about 35 minutes

> We can already build logic that adds two bit patterns. Why does that not already give us a computer that can calculate `3 + 5`?

An adder answers one fixed question whenever its inputs settle. It does not remember yesterday's answer, choose its own next operands, or decide that the next operation should be a store rather than another addition. A programmable machine needs a controlled sequence of remembered states.

This lesson builds that machine without hiding the transitions behind the phrase “the CPU executes an instruction.” We will design an explicit four-instruction architecture, encode a program as bytes, and account for what changes—and what does not—at each clock edge.

The central distinction is:

> **Combinational circuitry computes between state updates. A clock edge coordinates when selected state elements capture new values. The clock does not create the computation.**

---

1. The problem that fixed-purpose calculators could not solve

A gate network can implement a truth table. An adder network can continuously produce `A + B`. A mechanical calculator or a relay machine can similarly be wired so its structure enforces an operation.

But suppose the desired procedure is:

```
put 3 somewhere
put 5 somewhere else
add the two stored values
save the result
stop
```

Four new problems appear:

1. **Storage:** where do operands and intermediate results persist?
2. **Sequence:** which operation happens first, and which follows?
3. **Selection:** how does the same hardware perform different operations?
4. **Repetition:** how can changing a stored description change the procedure without rewiring the machine?

Early machines often received control from plugboards, switches, or punched media. Those approaches can be programmable, but instructions are supplied by a mechanism distinct from the machine's working memory. The stored-program idea makes a more powerful move: encode instructions as bit patterns and keep them in addressable electronic memory.

The Manchester Small-Scale Experimental Machine, or “Baby,” demonstrated this idea in working electronic hardware. Its first successful program was stored in memory along with data. It was a deliberately small experimental machine, largely built to test a practical electronic store, yet it established the essential loop:

```
address an instruction
    → fetch its stored bits
    → interpret those bits as control
    → transform machine state
    → select the next instruction
```

This is why a stored program is more than “data in memory.” The machine has a path that treats selected memory bits as instructions and lets those instructions determine future state transitions.

---

2. State is the CPU's memory of the past

A **state element** preserves a value after the signals that produced it have changed. Registers, flip-flops, and memory cells are stateful. An adder, decoder, or multiplexer considered by itself is combinational: its settled output is a function of its current inputs.

For our minimal CPU, the important state is:

```
PC              address of the next instruction to fetch
IR              instruction byte currently being executed
R0..R3          four general-purpose registers
data memory     sixteen stored data bytes
phase           whether the next cycle is FETCH or EXECUTE
halted          whether state updates should stop
```

There are two useful categories here.

Architectural state

Architectural state is the state software can observe according to the instruction-set contract. In our tiny design it includes `R0..R3`, data memory, and the sequencing behavior represented by the PC. If two implementations finish each instruction with the same architectural state, software sees the same machine.

Microarchitectural state

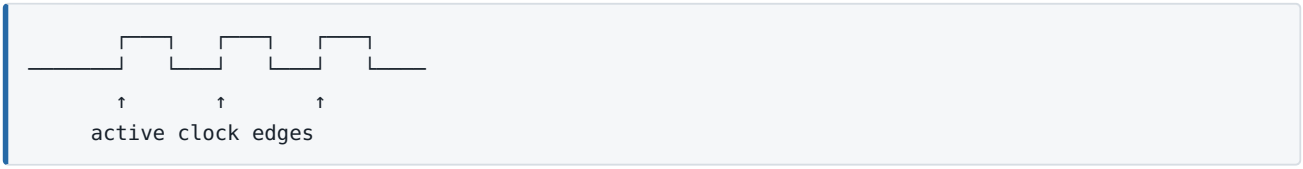
Microarchitectural state exists because of how this particular CPU is built. Our `IR` and `phase` register are examples. Another implementation of the same ISA might use more pipeline registers, no separately visible IR, or several internal steps per instruction. Those differences need not change the ISA.

The boundary can be subtle. A debugger may expose an implementation's PC or internal registers, but that does not automatically make every internal latch part of the ISA contract. Always ask: **must every conforming implementation present this behavior to software, or is this one hardware design's method?**

---

3. What the clock does—and does not do

A clock is a periodic timing signal, often drawn as alternating LOW and HIGH:



An edge-triggered register samples its input at an active edge and preserves that sampled value until a later permitted edge.

Between edges, signals propagate through combinational paths:

```
old register outputs
        ↓
decoder, multiplexers, ALU, address logic
        ↓
candidate next values settle
        ↓
next clock edge captures selected candidates
```

The clock does **not** add numbers. It does not decode opcodes. It does not push data through wires. Transistor networks perform that combinational work in response to their inputs. The clock provides coordinated moments at which state elements commit selected results.

This leads to a precise account of adjacent cycles:

1. Just after edge `N`, registers expose their newly stored outputs.
2. During the interval after edge `N`, combinational circuitry reacts and settles toward candidate next values.
3. Setup-time requirements demand that required candidates be stable before edge `N+1`.
4. At edge `N+1`, enabled state elements capture those candidates.
5. Just after that edge, their changed outputs begin another round of propagation.

The clock period must be long enough for the slowest required path, plus the timing margins of the state elements. A faster clock is not free: if the next edge arrives before the combinational result is valid, the CPU may capture the wrong state.

For now, use that as a conceptual timing contract. Week 4 returns to the quantitative constraints—setup and hold time, clock-to-Q delay, critical paths, asynchronous inputs, synchronizers, and metastability—in Sequential Timing and Metastability.

A snapshot is not the whole cycle

When a trace prints one line per edge, it hides continuous activity between the lines. “Cycle 4: `R0` becomes 8” really means:

- before the edge, `R0=3` and `R1=5` drive ALU inputs;
- the ADD network settles to `8`;
- control asserts the write enable for `R0`;
- at the edge, `R0` captures `8`;
- after the edge, the register's output changes to `8`.

The clock edge commits the answer; it did not produce it.

---

4. The core components

Program counter (PC)

The PC is a register holding the address of the **next instruction to fetch**. During our FETCH cycle, instruction memory is read at `PC`, and the candidate next PC is `PC + 1`. At the fetch edge, the IR captures the instruction and the PC captures that incremented address.

This convention matters. During the following EXECUTE cycle, the PC already points past the instruction in the IR.

Instruction memory

Instruction memory maps an address to an instruction byte. We model it as read-only while the CPU runs:

```
instruction_address = PC
instruction_byte    = instruction_memory[PC]
```

Real machines may share physical storage paths for code and data or use separate caches and buses. Separate instruction and data arrays make our first datapath easier to see; that is a microarchitectural choice.

Instruction register (IR)

The IR preserves the fetched byte while the PC moves on and the decoder uses the byte's fields. Without an IR in this two-cycle design, changing the PC during fetch would also change the instruction-memory output being decoded.

Register file

The register file holds four eight-bit values: `R0`, `R1`, `R2`, and `R3`. Read ports expose selected values to combinational logic. A write port accepts a destination number, value, and write-enable. On an active edge with writing enabled, exactly the selected register captures the result.

Arithmetic and logic unit (ALU)

Our minimal ALU only needs addition, plus pass-through behavior useful for displaying immediate and store values. It is combinational:

```
ALU_out = operation(ALU_A, ALU_B)
```

The ALU output is not automatically remembered. A register write or memory write must be enabled at an edge for the result to become state.

Data memory

Data memory contains sixteen eight-bit locations. `STORE` selects an address, places a register value on the write-data path, and asserts memory write enable. Our simulator commits that write during the EXECUTE state update.

Control unit

The control unit examines the current phase and, during EXECUTE, the IR's opcode and fields. It emits decisions such as:

```
ir_load       should IR capture instruction memory?
pc_increment  should PC capture PC + 1?
alu_op        which combinational ALU function?
reg_write     should a register capture ALU_out?
mem_write     should data memory capture a value?
halt          should the halted state be set?
```

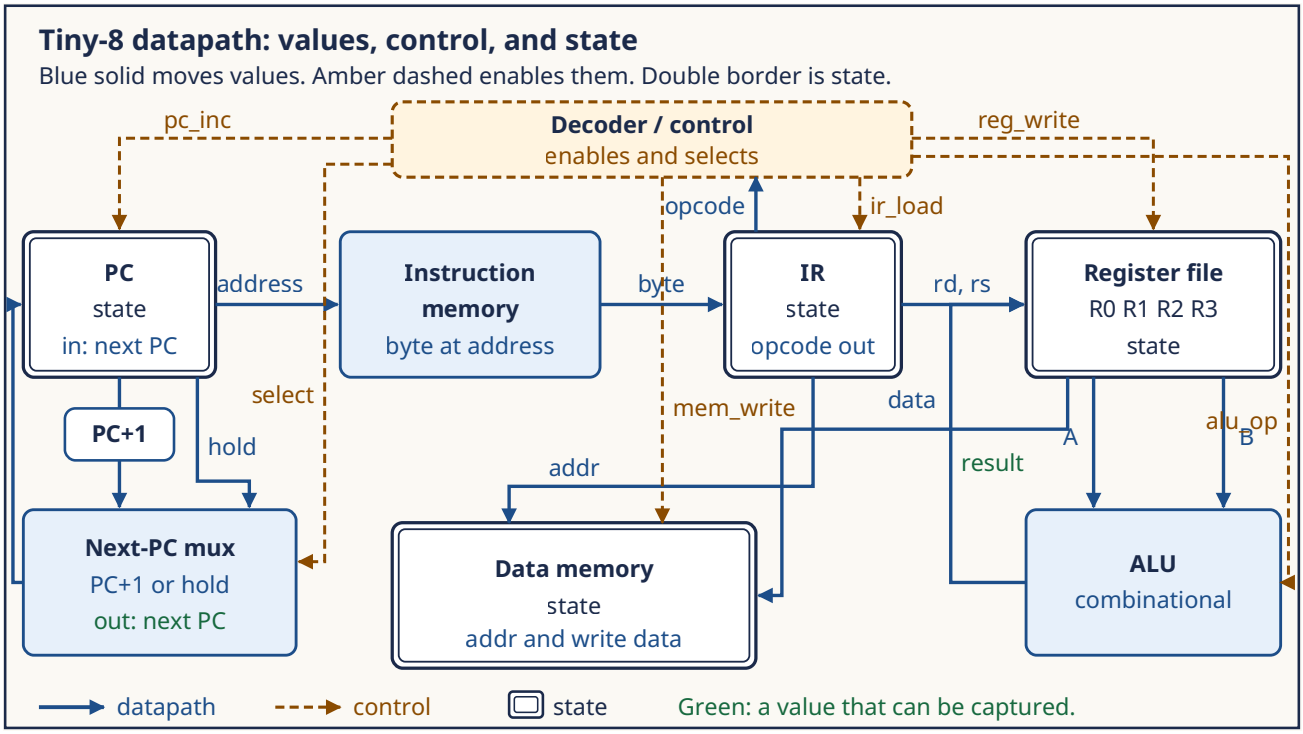
Control does not carry the operand value itself. It opens and closes paths, selects operations, and enables state updates.

---

5. Datapath versus control

The **datapath** contains the places where values are stored and transformed: PC, IR, register file, buses, multiplexers, ALU, and memory interfaces.

The **control path** decides which datapath action is allowed now.



Solid blue wires carry values, dashed amber wires carry enables and selects, and double borders mark state captured at a clock edge.

The diagram is a logical block diagram, not a claim that values flow in a single direction on every real wire. A practical register file has multiple ports; multiplexers select among sources; wires fan out; timing and electrical loading matter.

Why make the control/datapath split? Because the same expensive value-moving hardware can be reused. The ADD instruction selects two registers and enables write-back. STORE selects a register and enables a memory write. Different control patterns cause one datapath to realize different state transitions.

---

6. An explicit tiny four-instruction ISA

We will call the architecture **Tiny-8**.

Programmer-visible contract

- Instructions are one byte each.
- `PC` addresses instruction bytes and normally advances by one.
- Four registers, `R0..R3`, each hold an unsigned eight-bit pattern.
- Addition wraps modulo 256.
- Data memory has sixteen byte locations, addresses `0x0..0xF`.
- Execution begins with `PC=0`, all registers and data memory zero.
- The four instructions are `LDI`, `ADD`, `STORE`, and `HALT`.

Encoding

Every instruction begins with a two-bit opcode:

```
bits 7..6    opcode
00           LDI
01           ADD
10           STORE
11           HALT
```

The complete formats are:

```
LDI Rd, imm4       00 dd iiii
ADD Rd, Rs         01 dd ss 00
STORE Rs, addr4    10 ss aaaa
HALT               11 000000
```

`dd` or `ss` selects one of four registers:

```
00 → R0    01 → R1    10 → R2    11 → R3
```

Instruction behavior is:

```
LDI Rd, imm4       R[dd] ← zero_extend(imm4)
ADD Rd, Rs         R[dd] ← (R[dd] + R[ss]) mod 256
STORE Rs, addr4    MEM[addr4] ← R[ss]
HALT               halted ← 1
```

Some encodings contain reserved zeros. For `ADD`, bits `1..0` must be `00`. For `HALT`, bits `5..0` must be zero. Reserving patterns makes malformed instructions detectable and leaves design space for later extensions.

Encode the `3 + 5` program

```
address  assembly       fields       binary       hex
0        LDI R0, 3      00 00 0011   00000011     0x03
1        LDI R1, 5      00 01 0101   00010101     0x15
2        ADD R0, R1     01 00 01 00  01000100     0x44
3        STORE R0, 0    10 00 0000   10000000     0x80
4        HALT           11 000000     11000000     0xC0
```

The program is therefore the byte sequence:

```
03 15 44 80 C0
```

The byte `0x44` does not intrinsically mean `ADD R0, R1`. Under ASCII it is `D`; under unsigned interpretation it is 68. Tiny-8's ISA encoding and a Tiny-8 decoder give it instruction meaning.

---

7. Four layers that must not be collapsed

1. ISA specification

The ISA is a contract written for programmers and hardware implementers. It says which architectural state exists and how each legal instruction changes that state. `ADD R0, R1` must leave `R0` holding the wrapped sum.

2. Instruction bytes

`0x44` is a stored pattern selected by the PC. It is an instance of the encoding. The storage cell does not carry a tag saying “instruction.”

3. Hardware decoder

The decoder is combinational circuitry that recognizes opcode `01`, extracts `dd` and `ss`, selects register ports, chooses ADD, and prepares a write enable. It physically responds to signals; it does not understand the programmer's intent.

4. Microarchitecture

The microarchitecture is the organization that fulfills the ISA: this two-cycle FETCH/EXECUTE controller, its IR, buses, register file, and timing. A different Tiny-8 CPU could fetch and execute in one long cycle, use several short cycles, or pipeline instructions. If the ISA-visible results agree at the required boundaries, all can implement the same ISA.

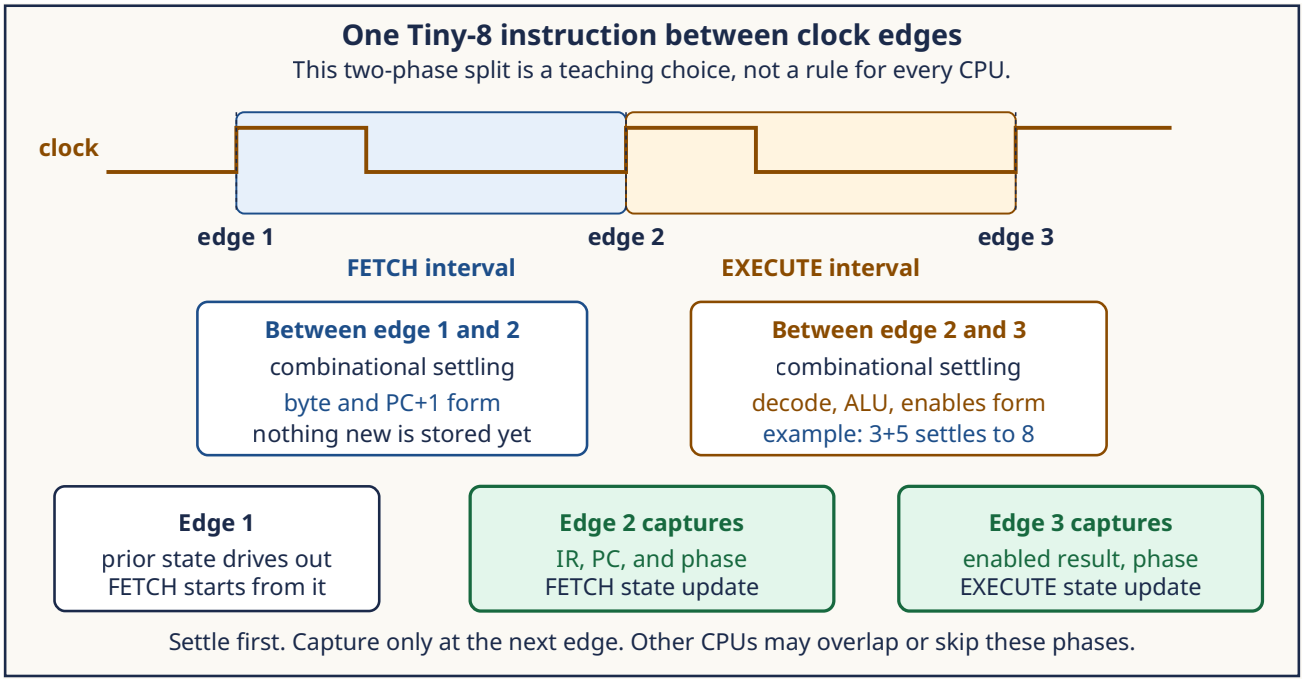
Keep the chain explicit:

```
ISA contract
    ↓ defines legal encodings and effects
instruction byte in memory
    ↓ drives
physical decoder and control signals
    ↓ steer
one chosen microarchitecture
    ↓ commits
architectural state transition
```

---

8. Fetch–decode–execute as physical and stateful events

“Fetch, decode, execute” is a conceptual sequence, not a universal promise of three clock cycles. In our microarchitecture, decoding is combinational during the EXECUTE interval, so one instruction takes two state-update cycles.



Combinational results settle during each interval, and the next edge captures only the enabled state in this two-phase teaching design.

FETCH interval and edge

Before the edge:

```
PC output ──────────────► instruction-memory address
instruction-memory data ─► candidate IR input
PC output ─► incrementer ─► candidate PC input
control: ir_load=1, pc_increment=1
```

At the edge:

```
IR ← instruction_memory[old PC]
PC ← old PC + 1
phase ← EXECUTE
```

Registers and data memory do not change.

EXECUTE interval and edge

Between edges, the IR's bits continuously drive the decoder. For an ADD:

```
IR opcode → control chooses ADD and reg_write
IR dd     → destination/read port selects R0
IR ss     → source read port selects R1
R0, R1    → ALU inputs
ALU sum   → candidate register-file write data
```

At the edge:

```
R[dd] ← ALU_out
phase ← FETCH
```

PC and IR remain unchanged during this edge. The IR may continue holding the old instruction while the phase change causes fetch controls to become active.

For a STORE, data memory changes instead of a register. For HALT, only the halted state changes. `LDI` routes its immediate field through a zero-extension path and writes the selected register.

What “decode” changes

Combinational decoding usually changes signals, not remembered state. As IR bits settle, control wires settle. No separate decode register exists in this minimal design. If we inserted one, decoding could become its own clocked stage; that would be a microarchitectural change.

---

9. Trace the program cycle by cycle

Initial state:

```
PC=0  IR=0x00  R0=0 R1=0 R2=0 R3=0  MEM[0]=0  phase=FETCH
```

Instruction 1: `LDI R0, 3`

Cycle 1 — FETCH

Between edges, `PC=0` addresses instruction memory, which presents `0x03`. The incrementer presents `1`.

At the edge:

```
IR: 0x00 → 0x03
PC: 0 → 1
phase: FETCH → EXECUTE
```

Cycle 2 — EXECUTE

The decoder sees opcode `00`, destination `R0`, immediate `3`. The immediate path presents `3`, and control asserts `reg_write`.

At the edge:

```
R0: 0 → 3
phase: EXECUTE → FETCH
```

PC remains 1; IR remains `0x03`.

Instruction 2: `LDI R1, 5`

Cycle 3 — FETCH

Instruction memory at address 1 presents `0x15`.

At the edge:

```
IR: 0x03 → 0x15
PC: 1 → 2
phase: FETCH → EXECUTE
```

Cycle 4 — EXECUTE

The decoder selects `R1`; the zero-extended immediate is 5.

At the edge:

```
R1: 0 → 5
phase: EXECUTE → FETCH
```

Instruction 3: `ADD R0, R1`

Cycle 5 — FETCH

Instruction memory at address 2 presents `0x44`.

At the edge:

```
IR: 0x15 → 0x44
PC: 2 → 3
phase: FETCH → EXECUTE
```

Cycle 6 — EXECUTE

Between edges:

```
decoder output: alu_op=ADD, reg_write=1, destination=R0
register outputs: R0=3, R1=5
ALU inputs: A=3, B=5
settled ALU output: 8
```

At the edge:

```
R0: 3 → 8
phase: EXECUTE → FETCH
```

Nothing “travels into R0 one arithmetic step at a time.” Carries propagate through the combinational adder before the edge; then the complete eight-bit result is captured.

Instruction 4: `STORE R0, 0`

Cycle 7 — FETCH

At the edge:

```
IR: 0x44 → 0x80
PC: 3 → 4
phase: FETCH → EXECUTE
```

Cycle 8 — EXECUTE

The decoder selects `R0=8`, address 0, and `mem_write=1`.

At the edge:

```
MEM[0]: 0 → 8
phase: EXECUTE → FETCH
```

Instruction 5: `HALT`

Cycle 9 — FETCH

At the edge:

```
IR: 0x80 → 0xC0
PC: 4 → 5
phase: FETCH → EXECUTE
```

Cycle 10 — EXECUTE

At the edge:

```
halted: 0 → 1
```

Final architectural result:

```
R0=8  R1=5  R2=0  R3=0  MEM[0]=8
```

This trace covers more than three instructions, but the mastery target is not the quantity. It is the ability to distinguish:

- values that merely settle on combinational wires;
- control decisions that enable a future write;
- and state that actually changes at the next edge.

---

10. Build and observe the Tiny-8 simulator

The challenge is:

`01-foundations-to-cpu/challenges/day-005-tiny-cpu.py`

First, predict the final values of `PC`, `IR`, `R0`, `R1`, and `MEM[0]`. Then run the complete trace:

```
python3 01-foundations-to-cpu/challenges/day-005-tiny-cpu.py
```

Each row exposes:

- state before the edge;
- the active control signals;
- ALU inputs and output when relevant;
- state after the edge;
- a compact list of committed state changes.

Now step manually:

```
python3 01-foundations-to-cpu/challenges/day-005-tiny-cpu.py --step
```

Before pressing Enter, cover the “after” state and answer:

1. Which combinational outputs should be settling now?
2. Which write enables are active?
3. Which state elements will change at the edge?
4. Which values will remain unchanged even though circuitry is active?

Finally, inspect the source. Locate three boundaries:

```
decode()        combinational control decision
combinational() candidate values between edges
clock()         committed state update at the edge
```

Python executes these sequentially, whereas real combinational circuits react in parallel and settle over propagation time. The simulator is a state- transition model, not a transistor timing simulation.

---

11. Common wrong models to repair

“The clock pushes the instruction through the CPU”

The clock coordinates state capture. Voltage changes propagate because circuits respond to their inputs. There is no clock hand carrying bytes.

“The PC is the current instruction”

In this design, after FETCH the PC already points to the next instruction, while the IR preserves the current one. Other architectures define and expose PC behavior differently, so state the convention.

“Decode is where the CPU changes state”

Our decoder is combinational. Its output can change when IR or phase changes, but architectural state changes only when an enabled storage element captures a value.

“The ISA diagram is the hardware”

An ISA specifies behavior. A block diagram describes one organization. A layout contains actual devices and wires. These are related descriptions at different abstraction levels.

“Instruction memory knows that `0x44` is ADD”

Memory returns a pattern. The Tiny-8 decoder interprets fields according to the Tiny-8 encoding. A different reader gives the same pattern another meaning.

“Every instruction always has fetch, decode, execute as three cycles”

Those are logical activities. A CPU may combine them in one clock interval, split them across many intervals, overlap several instructions in a pipeline, or translate instructions internally. The ISA does not normally dictate that schedule.

---

12. Mastery checkpoint

Close the lesson and answer:

> **What changes in this CPU between two adjacent clock cycles?**

A mastery-level answer must separate three moments:

1. **Just after the first edge:** state elements expose the values captured at that edge.
2. **Between edges:** those outputs drive combinational instruction memory, decoding, selection, and ALU paths; candidate next values and control signals settle, but no enabled state update has yet been committed.
3. **At the next edge:** only enabled state elements capture candidates—for example IR and PC on FETCH, one register or memory location on EXECUTE, or the halted bit for HALT.

It should also say that unchanged registers preserve their values, and that the clock coordinates capture rather than creating the computation.

Explain

Without notes:

- explain why a combinational adder is not a programmable CPU;
- distinguish architectural state from microarchitectural state;
- explain the jobs of PC, instruction memory, IR, register file, ALU, data memory, and control unit;
- distinguish datapath from control;
- distinguish ISA specification, instruction byte, hardware decoder, and microarchitecture;
- explain why “fetch–decode–execute” need not mean three cycles.

Draw

Recreate the minimal CPU block diagram. Your arrows must show:

- PC to instruction-memory address;
- instruction-memory byte to IR;
- IR fields to the control unit and register selectors;
- register outputs to ALU;
- ALU result back to register write data;
- register value and IR address field to data memory;
- control signals reaching every state element they enable.

Beside the diagram, draw two vertical clock edges. In the space between them, write the combinational work for `ADD R0, R1`. At the second edge, mark only `R0` and `phase` as changing.

Observe

Run both normal and `--step` simulator modes. Capture one FETCH transition and one ADD transition. For each, record:

```
state before | combinational/control activity | enabled writes | state after
```

Explain why the ALU may display a value even on a cycle where no register captures it.

Build

1. Re-encode the sample program by hand and compare it with the simulator.
2. Change the immediates to compute `9 + 6`, storing at address 3.
3. Complete the intentional extension described in the challenge: add a zero flag, or design a branch instruction and state exactly which existing encoding must change. Do not silently overload a bit field.

Why? notebook

1. Why does the PC advance during FETCH rather than because “time passed”?
2. Why is an IR useful after the PC changes?
3. Why can the ALU be active when no architectural state changes?
4. Why does a register need both data input and write enable?
5. Why is the decoder hardware rather than meaning attached to memory?
6. Why can two CPUs implement Tiny-8 differently yet run the same byte program?
7. Why must the clock period account for combinational propagation delay?
8. Why is the phase register microarchitectural in this design?
9. Where, physically, does `3 + 5 = 8` happen, and when does 8 become durable CPU state?

Mental model at the end

```
stored instruction pattern selected by PC
        ↓ instruction-memory combinational read
IR captures the pattern at a clock edge
        ↓ decoder and register reads settle
control selects a datapath operation
        ↓ combinational ALU/address work
candidate result becomes stable
        ↓ next enabled clock edge
selected architectural state captures the result
        ↓
PC identifies the next stored instruction
```

The CPU is not a box in which the clock makes computation happen. It is a network of state elements separated by combinational paths. Stored instructions cause the control network to select paths and write enables. Repeated, coordinated state transitions turn fixed circuits into a programmable machine.

References used selectively

Local reading:

- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 4, “The von Neumann Model,” for memory, processing unit, input/ output, control unit,

instruction cycle, PC, and IR; and Chapter 5, “The LC-3,” for the distinction between an ISA specification, encoded instructions, registers, memory, and instruction behavior. Local PDF: `references/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf`

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 14, 16, and 17—“Feedback and Flip-Flops,” “An Assemblage of Memory,” and “Automation”—for the progression from remembered state to memory and automatic instruction sequencing; Chapter 18, “From Abaci to Chips,” for historical context. Local PDF: `references/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf`

Stored-program history:

- University of Manchester Department of Computer Science, “History and heritage,” on the Small-Scale Experimental Machine and its first successful stored-program run: https://www.cs.manchester.ac.uk/about/history-and-heritage/
- University of Manchester Computer Science curation, “The First Program,” describing Tom Kilburn's factor program, its instruction count, and the Baby's hardware subtractor: https://curation.cs.manchester.ac.uk/digital60/www.digital60.org/birth/program/firstprog.html

The Tiny-8 ISA and simulator are teaching designs created for this lesson. They are intentionally small enough that every instruction bit, control decision, combinational result, and state update can be inspected.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Tiny Cpu

Format: Python lab

What you will do: A cycle-visible simulator for the Tiny-8 teaching CPU.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
python3 01-foundations-to-cpu/challenges/day-005-tiny-cpu.py
```

- View lab source on GitHub

Day 6 — Instruction Sets and Machine Code

Target time: approximately 3 hours

Suggested rhythm: 75 minutes building the model · 35 minutes comparing real instruction sequences · 45 minutes in the C/disassembly lab · 35 minutes designing and testing Cedar-16 · 10 minutes explaining without notes

> A CPU contains gates that can add, compare, move bits, and choose a next state. How does a compiler ask that machinery to perform one particular operation—and how can a different CPU understand the same request?

Why this day exists

The previous path assembled the ingredients of a minimal processor:

```
gates → arithmetic and selection → registers → datapath + control
      → fetch → decode → execute
```

There is still a missing agreement.

Imagine buying a new processor whose designer tells you:

> “It has registers, an ALU, and a very fast decoder.”

That does not tell a compiler how to generate a program for it. Which bit pattern requests addition? How many registers can software name? Can an arithmetic instruction read memory directly? How is a branch target encoded? What happens when arithmetic overflows? What does an invalid bit pattern do?

Hardware and software need a durable contract. That contract is the **instruction set architecture**, or **ISA**.

Today we will make that contract concrete at three levels:

```
ISA specification       written rules and programmer-visible behavior
machine-code bytes      encoded instructions in an object file or memory
processor implementation decoder and execution machinery that obey the rules
```

Those are related, but they are not the same object and they are not stored in one place called “the ISA.”

By the end, the central mastery question should feel inevitable:

> **Why can two CPUs implement the same ISA differently?**

Part I — The historical problem: programs needed a stable language

Early machines often exposed the machine too directly

Before stored-program computers became normal, “programming” could mean rewiring a machine, setting switches, or arranging external media. The operation and the physical setup were tightly coupled. Changing the problem could mean changing the machine's configuration.

The stored-program idea made a powerful move: represent instructions as patterns in memory, alongside data. The control unit could fetch one pattern, interpret it, perform an operation, and fetch another.

That solved one problem and created the next:

> Which patterns mean which operations?

A bit pattern has no intrinsic command hidden inside it. A designer must assign an interpretation, document the assignment, and build a processor that behaves accordingly.

Suppose an early design chooses:

```
0001 ddd aaa bbb 000
```

and declares it to mean:

```
ADD register[a] and register[b], place the result in register[d]
```

The declaration alone does not execute anything. The physical decoder must recognize `0001`, select register ports, command the ALU to add, enable a destination write, and advance the program counter. Conversely, the circuitry alone is not a usable software interface until its behavior is specified.

This separation became increasingly valuable as computer families evolved. Software investments were expensive. Customers wanted newer, faster machines without rewriting every program. A compatible architecture could remain stable while vacuum tubes gave way to transistors, narrow datapaths widened, caches appeared, pipelines deepened, and execution became speculative and out of order.

Compatibility is therefore not historical clutter by definition. It is an engineering asset with costs and benefits.

Architecture became a boundary

The ISA boundary says, roughly:

```
software may rely on these visible operations and results
hardware may choose how to produce those results
```

This boundary lets:

- a compiler target an architecture rather than one transistor layout;

- an operating system manage defined registers, privilege states, and exceptions;
- multiple processor vendors or generations run compatible binaries;
- hardware designers improve performance without changing every program;
- simulators and virtual machines implement the same visible behavior in software.

The boundary is not perfectly simple. Real ISAs contain optional extensions, implementation-defined behavior, privileged state, memory-ordering rules, and versioned features. Software may query which features exist. But the central idea remains: the architecture specifies the machine that software is allowed to see.

Part II — ISA versus microarchitecture

What belongs to the ISA?

An ISA normally specifies programmer-visible matters such as:

- available instruction encodings and their semantics;
- architectural registers and their widths;
- supported data types and arithmetic behavior;
- how instructions name register, memory, and immediate operands;
- address calculation and the architectural address space;
- program-counter behavior and control transfers;
- visible status flags or control/status registers;
- exceptions caused by invalid operations or access conditions;
- privilege levels and privileged instructions;
- atomic operations and memory-ordering rules;
- sometimes optional extensions that software can detect.

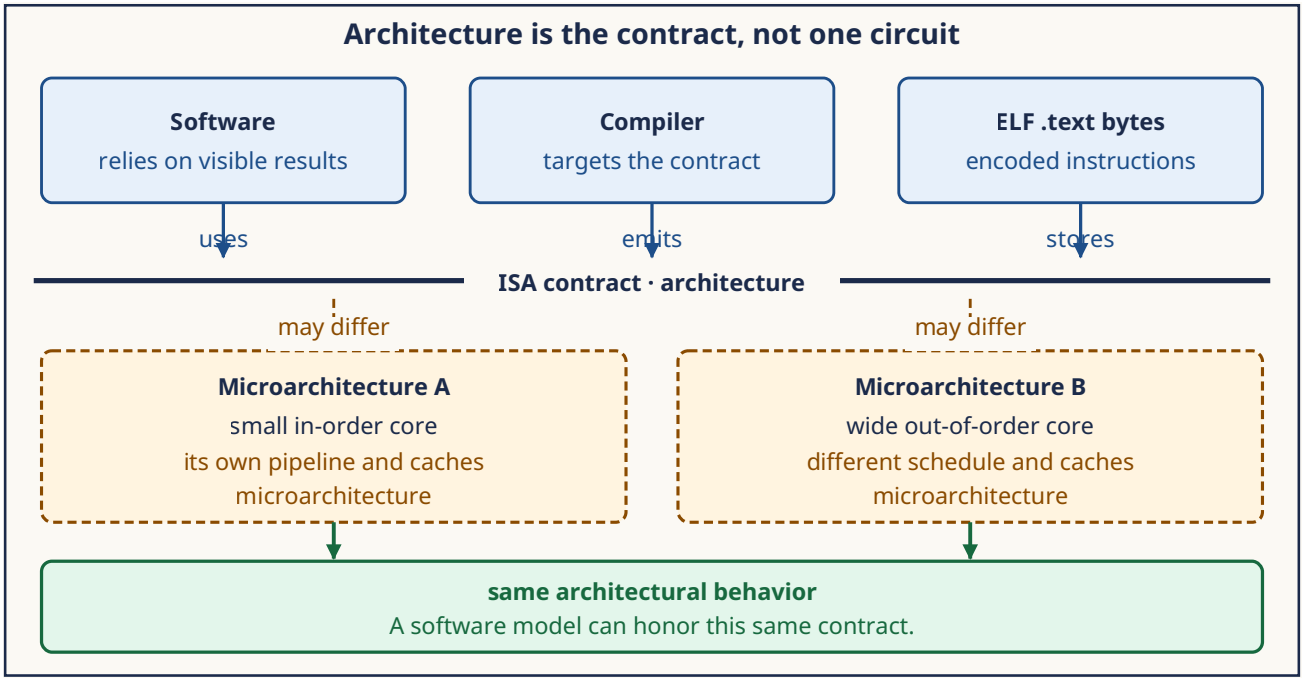
If two conforming processors execute the same valid architectural program from the same architectural starting state, software expects the specified architectural outcome.

What belongs to a microarchitecture?

A **microarchitecture** is one particular internal organization used to implement an ISA. It may choose:

- pipeline depth;
- number and kind of execution units;
- in-order or out-of-order scheduling;
- cache sizes, associativity, and replacement details;
- branch predictor design;
- instruction fetch and decode width;
- internal physical-register count and register-renaming scheme;
- whether an instruction uses hardwired control, several internal operations, or a microcode-assisted sequence;
- power-management and clocking strategies.

These choices affect speed, power, area, cost, and sometimes timing-visible side effects. They do not normally change the required architectural answer.



Compilers and ELF bytes target the architectural contract, while two different internal organizations can still produce the same visible behavior.

A compact low-power x86-64 core and a wide server x86-64 core can decode the same architectural `ADD` instruction while using very different pipelines, caches, predictors, and internal schedules. Likewise, many different RISC-V implementations can implement the same selected base ISA and extensions.

Architectural state versus hidden working state

Architectural state is the state the ISA says software can observe: for example, the program counter, general-purpose registers, and defined status state.

Microarchitectural state helps an implementation do its work:

```
branch-predictor entries
cache replacement metadata
reorder-buffer entries
physical-register mappings
decoded-instruction queues
```

Ordinary instructions do not name those structures directly. They may affect performance and, as security research has taught us, hidden state can sometimes leave observable timing traces. That does not turn predictor tables into architectural registers. It shows that “not architecturally visible” is not identical to “physically nonexistent” or “incapable of side effects.”

Pause and explain

For each item, classify it as ISA, microarchitecture, or possibly both at different levels:

1. `RAX` is a 64-bit programmer-visible register.
2. The L1 data cache is 48 KiB.

3. A particular opcode performs integer addition.
4. The processor can issue six internal operations in one cycle.
5. A misaligned load either succeeds or raises a specified exception.
6. A branch predictor guesses the next address.

The useful test is:

> Must normal software know this to produce the architecturally correct result, or may the processor designer change it while preserving the contract?

Part III — What is a machine instruction?

A **machine instruction** is an ISA-defined operation represented by an instruction encoding. When fetched and successfully decoded, it requests a defined architectural state transition.

Consider an abstract instruction:

```
ADDI r3, r2, 12
```

Its fields have different jobs:

- **opcode**: selects the operation family, here add-immediate;
- **destination operand**: `r3`, where the architectural result goes;
- **source operand**: `r2`, one input value;
- **immediate**: `12`, a constant carried inside the instruction encoding.

A possible fixed-width encoding might be:

```
bits:  15         12 11       9 8        6 5                0
       +------------+-----------+----------+------------------+
       | opcode=0101| rd = 011  | rs1=010  | immediate=001100 |
       +------------+-----------+----------+------------------+
```

The immediate bits occupy space that could otherwise name more registers, more operations, or a larger constant. Encoding is resource allocation inside a finite number of bits.

Opcode does not always mean “one neat field”

Teaching diagrams often show one contiguous opcode field. That is useful, but real encodings can be more involved.

An instruction's operation may be selected by a combination of:

- a primary opcode;
- extension fields;
- prefix bytes;
- function fields;
- bits that also influence operand size or addressing.

RISC-V base instruction formats use a 7-bit major opcode plus fields such as `funct3` and `funct7` to distinguish operations. x86 encodings may use legacy, REX, VEX, or EVEX prefixes, one- or multi-byte opcode maps, ModR/M and SIB bytes, displacements, and immediates.

The conceptual word “opcode” still helps: it is the encoded information that selects the operation. Do not insist that every architecture puts all of that information in one rectangle.

Operands answer “operation on what?”

Common operand sources are:

1. **register operands** — names select values in architectural registers;
2. **memory operands** — an effective address identifies a memory location;
3. **immediate operands** — constant bits are embedded in the instruction;
4. **implicit operands** — the ISA definition names a value not written explicitly in assembly syntax.

Examples in x86-64 AT&T syntax:

```
addq %rcx, %rax          # register source, register destination
addq $7, %rax            # immediate source, register destination
addq (%rdi), %rax        # memory source, register destination
```

AT&T syntax writes source before destination and prefixes registers with `%` and immediates with `$`. Intel syntax commonly reverses operand order and uses different notation. Syntax is an assembler language convention. The CPU sees the encoded bytes, not `%`, `$`, commas, or source comments.

Immediate values are not tiny memory variables

In:

```
addq $7, %rax
```

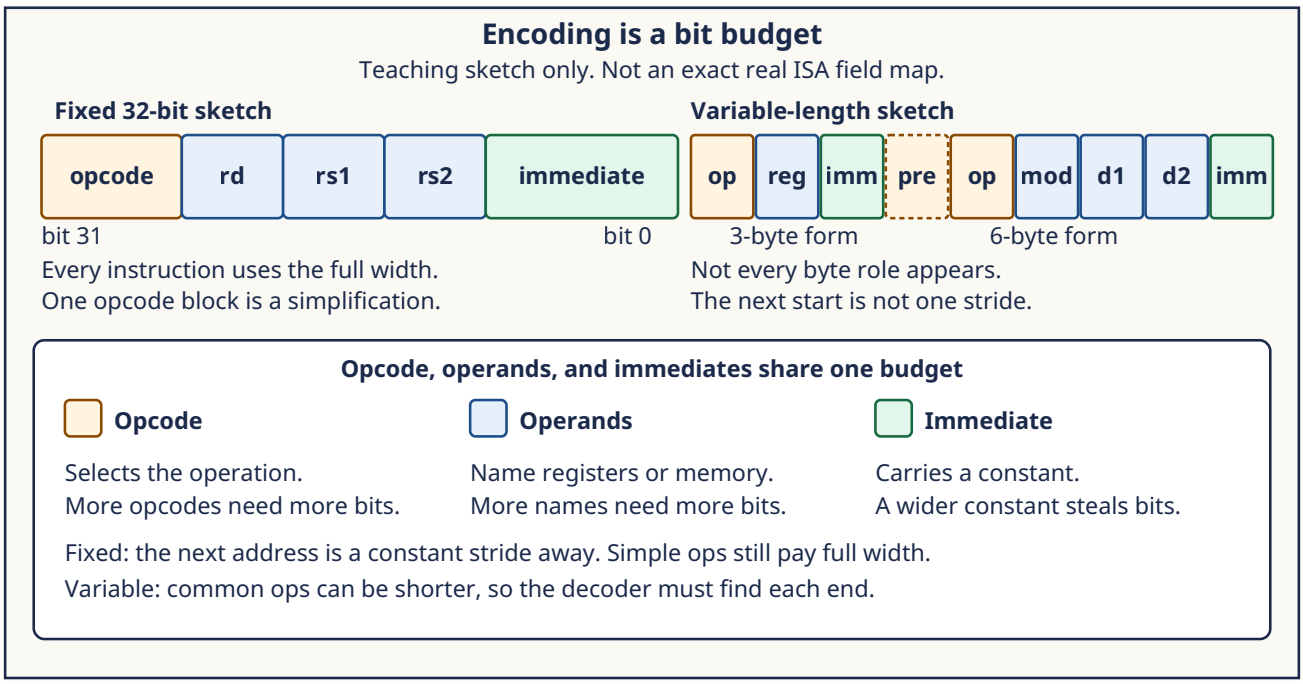
the value `7` is encoded as part of the instruction. Fetching the instruction brings in those immediate bits. The processor does not first load `7` from a data object at address 7.

Immediate width forces tradeoffs. A small field makes common constants compact but cannot directly represent every value. Architectures provide strategies such as:

- sign-extend a small immediate;
- use several instructions to construct a large constant;
- load a constant from memory;
- provide special upper-immediate or literal-loading forms.

Sign extension is an architectural rule, not a guess made by the decoder.

Part IV — Encoding is a constrained design problem



Opcode, operand, and immediate fields share a limited bit budget, and these rectangles are a sketch rather than an exact real encoding.

Fixed-length instructions

A fixed-width design makes instruction boundaries easy to find:

```
32-bit instruction | 32-bit instruction | 32-bit instruction
```

Benefits can include:

- regular fetch alignment;
- simpler parallel field extraction;
- predictable next sequential address;
- a relatively regular decoder.

Costs include:

- simple operations still occupy the full width;
- large register sets, large immediates, and many opcodes compete for fields;
- some operations need multiple instructions;
- code density may suffer.

The base RISC-V integer ISA uses fixed 32-bit instructions. The optional compressed extension adds 16-bit encodings for common operations, so a real RISC-V instruction stream need not be purely 32-bit. “RISC-V is fixed-width” is therefore incomplete unless the chosen ISA profile excludes compressed instructions.

Variable-length instructions

x86 instructions are variable length, architecturally from 1 through 15 bytes. A conceptual layout is:

```
[prefixes] [opcode] [ModR/M] [SIB] [displacement] [immediate]
```

Not every instruction has every part.

Benefits include:

- frequent simple operations can be compact;
- old encodings can coexist with newer extension mechanisms;
- operands and addressing forms can be described flexibly;
- code density can be good for many workloads.

Costs include:

- finding several instruction boundaries in parallel is harder;
- decode hardware is more complex;
- instruction length is not known from one universal field;
- legacy constraints consume encoding space and design effort.

Neither scheme wins every metric. A regular encoding can simplify parts of a front end but require more instructions or bytes for a task. A rich variable-length encoding can express work compactly but demand sophisticated fetch and decode machinery.

Architecture lens — x86-64, AArch64, and RISC-V

- x86-64 instructions are 1–15 bytes, expose sixteen general-purpose registers, and often use `RFLAGS` plus register-or-memory operand forms.
- AArch64 A64 instructions are fixed at 32 bits, expose thirty-one general-purpose registers plus context-dependent `SP`/zero-register encodings, and normally use explicit loads and stores; `NZCV` holds condition flags.
- Base RISC-V instructions are 32 bits, with optional 16-bit compressed encodings; its 32 integer registers include fixed-zero `x0`, and integer branches compare registers without a general condition-code register.

Those are ISA contracts. Argument-register names and preserved-register rules come from an ABI, while decode width, physical registers, and internal operations belong to a processor implementation. See Architecture Comparison.

There is no free encoding field

Suppose Cedar-16 has only 16 bits per instruction:

```
4-bit opcode + 3-bit rd + 3-bit rs1 + 3-bit rs2 + 3 spare bits
```

The 3-bit register fields name only eight registers. To name 32 registers each field would need 5 bits, adding six bits across three operands. Something else must shrink, move to another format, or require a wider instruction.

Ask these questions of any proposed format:

- How many operations can it distinguish?
- How many registers can it name?
- How large is its immediate or branch range?

- Can every bit pattern be decoded unambiguously?
- Are some patterns reserved for future expansion?
- How does a decoder find the next instruction?
- What common operations require more than one instruction?

An instruction encoding is not just a binary translation exercise. It is a budget.

Part V — Register operands, memory operands, and load/store design

Why registers matter

Registers provide a small, directly named working set near execution units. An instruction can encode a register number in a few bits, and the microarchitecture can build fast register-file access around those names.

Memory is much larger and addressed differently. A memory operand normally requires an **effective address** calculation, then access through the memory system. At the ISA level we describe the address and result. Later memory hierarchy lessons will explain why latency can vary dramatically.

Load/store architectures

In a load/store design, ordinary arithmetic operates on registers. Memory is accessed with explicit load and store instructions.

A conceptual RISC-V sequence for `*p += 7`, with `p` in `a0`, is:

```
ld   t0, 0(a0)       # memory → register
addi t0, t0, 7       # register arithmetic
sd   t0, 0(a0)       # register → memory
```

RISC-V integer arithmetic instructions do not take a general memory operand. This regularity makes data movement explicit.

x86 permits memory operands in many operations

An equivalent x86-64 sequence, with `p` in `rdi`, can be:

```
addq $7, (%rdi)      # read-modify-write the addressed 64-bit memory object
```

Architecturally this is one instruction with a memory destination and an immediate source. Do not infer that it takes one cycle, one internal action, or one memory transaction in every situation.

Also do not infer that x86 allows arbitrary combinations. Most general-purpose x86 instructions allow at most one explicit memory operand. An instruction such as a general `mov` cannot normally copy from one arbitrary memory location directly to another arbitrary memory location; a register is usually needed between them. Special string instructions have defined implicit memory operands, which is another reminder that real ISAs resist one-sentence summaries.

Instruction count is not performance

The example has one x86 instruction and three RISC-V instructions. That alone does not prove the x86 sequence is three times faster.

Performance depends on matters such as:

- the actual implementations;
- instruction bytes and front-end bandwidth;
- decoded internal operations;
- dependency chains;
- cache hits or misses;
- available execution ports;
- surrounding code and scheduling;
- branch behavior and speculation.

Instruction count is useful evidence. It is not a complete performance model.

Modern x86 and micro-operations: the careful version

Many modern high-performance x86 implementations decode architectural x86 instructions into internal **micro-operations**, often shortened to µops. Those µops can be renamed, scheduled, and executed by an out-of-order back end. Some complex instructions need several µops or microcode-assisted flows; some simple instructions map economically; implementations can also fuse certain operations in specific circumstances.

The caveats matter:

1. µops are generally microarchitectural, not the x86 ISA exposed to normal application code.
2. Intel and AMD need not use identical µops or decoding machinery.
3. “x86 becomes RISC internally” is too vague to be a sound model. Internal operations are implementation-specific and are not simply RISC-V instructions.
4. A single architectural instruction is not guaranteed to be one µop, and one source-level operation is not guaranteed to be one instruction.

The stable claim is:

> The front end accepts architectural x86 instruction bytes; a particular microarchitecture may translate them into its own internal work while preserving architectural behavior.

Part VI — Addressing modes: how an instruction names memory

An **addressing mode** defines how an instruction obtains an operand or computes its effective address.

Register direct

```
addq %rcx, %rax
```

The encoded register fields name both operand locations.

Immediate

```
addq $12, %rax
```

The source value is carried in the instruction.

Base plus displacement

```
movl 12(%rdi), %eax
```

Conceptually:

```
effective address = RDI + 12
load 4 bytes from that address
```

This is useful for a structure member at a known offset or a local object relative to a frame/base register.

Base plus scaled index plus displacement

x86-64 can express:

```
movl 8(%rdi,%rsi,4), %eax
```

Conceptually:

```
effective address = RDI + RSI × 4 + 8
```

Scale factors supported by this addressing form are 1, 2, 4, or 8. This maps naturally to indexed arrays and fields, though the compiler is free to choose another sequence.

PC-relative addressing

The effective address can be based on the current instruction location. This is useful for branches and position-independent references:

```
target = architecturally defined PC base + signed displacement
```

The exact PC base and scaling rule are ISA-specific. Do not assume every architecture uses the address of the branch instruction itself or measures the displacement in bytes.

RISC-V's regular memory address form

Base RISC-V integer loads and stores use a register base plus a signed 12-bit immediate offset. A more elaborate address is normally calculated in registers first.

Conceptually:

```
slli t0, a1, 3       # t0 = index × 8
add  t0, a0, t0      # t0 = base + scaled index
ld   t1, 16(t0)      # load at base + index×8 + 16
```

This is not a defect disguised as simplicity, nor is x86's richer mode a free win. They allocate complexity and encoding space differently.

Address calculation is not memory access

An effective address is an architectural value used to identify a memory location. Calculating it does not guarantee the access succeeds. Translation, permissions, alignment rules, and faults belong to the complete execution path.

This distinction will matter repeatedly:

```
instruction bytes specify addressing rule
        ↓
register values + encoded displacement produce effective address
        ↓
memory system attempts the access
        ↓
architectural result or defined exception
```

Part VII — RISC and CISC without caricatures

The labels **RISC** and **CISC** arose from real design debates, but slogans often erase the useful parts.

Tendencies associated with RISC designs

Historically common goals include:

- relatively regular instruction formats;
- load/store organization;
- many general-purpose registers;
- operations that are convenient to pipeline;
- letting compilers combine simpler instructions;
- reserving complex policy from the common hardware path.

RISC-V makes many of these choices visible: a small base ISA, extension structure, load/store integer operations, and regular base formats.

But “RISC means every instruction is simple and one cycle” is false. Modern RISC-family designs include floating-point, vector, atomic, cryptographic, and privileged operations. Loads can miss caches. Division can take many cycles. Implementations can be wide, speculative, and highly complex.

Tendencies associated with CISC designs

Historically common characteristics include:

- many instruction forms and addressing options;
- variable-length encodings;
- operations that can combine memory access with computation;
- strong compatibility across long-lived product families;
- instructions with substantial architectural behavior.

x86-64 exhibits these characteristics.

But “CISC means one giant instruction does the whole program” is false. Compilers usually select efficient subsets and sequences. Modern x86 cores use advanced pipelines and internal operations. Many common x86 instructions are ordinary moves, arithmetic operations, comparisons, and branches.

Better questions than “which philosophy won?”

Ask instead:

- How regular is decode?
- How dense is common code?
- How many architectural instructions are needed for this operation?
- What work does each instruction specify?
- How much compatibility must be preserved?
- How easily can implementations scale from tiny to aggressive?
- What does the compiler need to synthesize?
- What are the power, area, verification, and toolchain costs?

The boundary has also blurred. RISC architectures add extensions and compact encodings; x86 implementations translate and schedule internal operations. The architectural contracts remain meaningfully different even though implementation techniques cross historical categories.

Part VIII — Equivalent x86-64 and RISC-V sequences

These examples are conceptual but use real instruction forms. Assume the usual Unix-like calling conventions only to choose argument registers: the first pointer argument arrives in `rdi` on System V AMD64 and in `a0` on the RISC-V ELF psABI.

That sentence introduces an important distinction:

- the **ISA** defines instructions and architectural registers;
- the **ABI** defines software conventions such as argument registers, caller/callee-saved registers, stack alignment, object-file details, and system interfaces.

Another ABI could use the same ISA differently. We will study ABI mechanics in depth later.

Example A: load a 64-bit value and add seven

C intent:

```
uint64_t add_seven(const uint64_t *p)
{
    return *p + 7;
}
```

x86-64, AT&T syntax:

```
movq (%rdi), %rax
addq $7, %rax
ret
```

RV64I:

```
ld   a0, 0(a0)
addi a0, a0, 7
ret                    # assembler pseudoinstruction for jalr x0, 0(ra)
```

The sequences are similar because both perform a load, addition, and return. The exact encodings are not similar, and `ret` is not necessarily a distinct hardware opcode in both assembler languages.

Example B: increment a value in memory

C intent:

```
void increment(uint64_t *p)
{
    *p += 1;
}
```

Plausible x86-64:

```
addq $1, (%rdi)
ret
```

Plausible RV64I:

```
ld   t0, 0(a0)
addi t0, t0, 1
sd   t0, 0(a0)
ret
```

Important precision: neither sequence makes a concurrent non-atomic C update safe. “One x86 instruction” does not automatically mean an atomic read-modify-write visible to other cores; x86 uses specific locking semantics for that purpose. We are comparing operand models, not teaching concurrency yet.

Example C: indexed load

C intent:

```
uint64_t element(const uint64_t *base, size_t index)
{
    return base[index];
}
```

Plausible x86-64:

```
movq (%rdi,%rsi,8), %rax
ret
```

Plausible RV64I:

```
slli a1, a1, 3
add  a0, a0, a1
ld   a0, 0(a0)
ret
```

The x86 addressing mode carries scale information in the load. RISC-V constructs the address through register operations. This makes the design tradeoff visible, but still does not predict total runtime by counting lines.

Comparison checklist

For each pair, record:

1. architectural instruction count;
2. which instructions access memory;
3. where constants appear;
4. how the effective address is formed;
5. which register holds the return value by ABI convention;
6. which lines are assembler pseudoinstructions;
7. what cannot be concluded about cycles or µops.

Part IX — Where does the ISA actually exist?

This question deserves a precise layered answer.

1. The ISA exists as a specification

Intel, AMD, and RISC-V publish architecture manuals. These documents define encodings, behavior, exceptions, state, and extension rules. The specification is a contract expressed in documents and formal descriptions.

A PDF does not control the processor in real time. It communicates the rules to hardware designers, compiler writers, operating-system developers, verification engineers, and tool builders.

2. A program contains machine-code encodings

An ELF executable or object file contains byte sequences in sections such as `.text`. When loaded, executable mappings contain those bytes. A disassembler uses ISA decoding rules to render bytes as assembly text.

```
object/executable .text bytes
        ↓ loader maps bytes
process virtual memory contains instruction bytes
        ↓ fetch
processor receives bytes
```

Those bytes are **instances of encoded instructions**. The file does not contain the entire ISA specification.

3. A processor implements decode and behavior

The CPU front end fetches instruction bytes and identifies valid encodings. Control and execution structures cause the required architectural behavior. This implementation may include combinational logic, tables, caches of decoded work, sequencers, and microcode. The exact design is processor specific.

It is imprecise to say “the ISA is stored in the decoder.” The decoder implements part of the contract. Other parts involve execution units, architectural-state machinery, memory ordering, exception handling, and more.

4. Tools contain models of the encoding rules

Assemblers, compilers, disassemblers, emulators, debuggers, and binary translators also contain software representations of ISA rules:

```
assembler:     mnemonic + operands → bytes
disassembler:  bytes → textual instruction
emulator:      bytes + state → modeled next state
compiler:      language operations → chosen instruction sequences
```

None of these is “the one place” where the ISA exists. They are artifacts or implementations aligned to the specification.

The sentence to keep

> The ISA is a specification-level contract; machine code is its encoded program representation; decoders and execution machinery are physical implementations of its behavior.

Part X — Observe machine code in a real ELF file

Use:

`01-foundations-to-cpu/challenges/day-006-machine-code-lab.c`

The program deliberately contains:

- indexed loads from an integer array;
- widening arithmetic and multiplication;
- loops and conditional behavior;
- a `switch` with several result paths;
- separately visible functions for focused disassembly.

Step 1 — Predict before compiling

For `weighted_sum`, predict:

1. Which source operations require memory loads?
2. Which values could remain in registers through the loop?
3. Where must control return to repeat the loop?

4. What must happen when `index == count`?

For `count_above`, predict whether optimized code must contain a conditional branch for the source-level `if`. The correct prediction is not “yes because C has `if`.” A compiler may use any target sequence that preserves the language semantics.

Step 2 — Build with GCC and Clang

From the repository root:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g \
  01-foundations-to-cpu/challenges/day-006-machine-code-lab.c -o /tmp/day6-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -O2 -g \
  01-foundations-to-cpu/challenges/day-006-machine-code-lab.c -o /tmp/day6-clang

/tmp/day6-gcc
/tmp/day6-clang
```

Warnings are evidence. Do not suppress them merely to make the terminal quiet. Both compilers should build this source warning-clean.

Step 3 — Ask `readelf` what kind of object this is

```
readelf -h /tmp/day6-gcc
readelf -S /tmp/day6-gcc
readelf -sW /tmp/day6-gcc
```

Find:

- the `Class` and `Machine` fields in the ELF header;
- the `.text` section;
- symbols for `weighted_sum`, `count_above`, and `choose_operation`.

`readelf` is showing ELF structure and metadata. It is not primarily decoding the instruction stream.

Step 4 — Ask `objdump` to decode bytes

```
objdump -d -M intel --disassemble=weighted_sum /tmp/day6-gcc
objdump -d -M intel --disassemble=count_above /tmp/day6-gcc
objdump -d -M intel --disassemble=choose_operation /tmp/day6-gcc
objdump -s -j .text /tmp/day6-gcc
```

The `-d` output places instruction bytes beside disassembled text. Choose one instruction and mark:

```
virtual address | raw bytes | mnemonic | rendered operands
```

Then locate those bytes in the `.text` dump. The assembly rendering is not stored as those words in the executable. `objdump` reconstructs it by decoding bytes according to the selected architecture and syntax.

Step 5 — Change the compiler's choices

Build an unoptimized version:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g \
  01-foundations-to-cpu/challenges/day-006-machine-code-lab.c -o /tmp/day6-gcc-O0

objdump -d -M intel --disassemble=count_above /tmp/day6-gcc-O0
objdump -d -M intel --disassemble=count_above /tmp/day6-gcc
```

Compare:

- number of loads and stores;
- stack use;
- register reuse;
- loop shape;
- branch or branchless conditional handling;
- function size.

The C source did not prescribe one machine-code sequence. The compiler, optimization level, target ISA, ABI, and available extensions jointly constrain the generated program.

Step 6 — Compare GCC and Clang without declaring a winner

```
objdump -d -M intel --disassemble=weighted_sum /tmp/day6-gcc
objdump -d -M intel --disassemble=weighted_sum /tmp/day6-clang
```

Record differences. Different output is not evidence that one compiler broke the ISA. Run both programs and connect each sequence back to the required C result.

Optional: produce assembly without assembling

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -S \
  01-foundations-to-cpu/challenges/day-006-machine-code-lab.c -o /tmp/day6-gcc.s

clang -std=c17 -Wall -Wextra -Wpedantic -O2 -S \
  01-foundations-to-cpu/challenges/day-006-machine-code-lab.c -o /tmp/day6-clang.s
```

These `.s` files contain assembler text. They are not yet ELF machine-code executables. This distinction anticipates the later toolchain lesson:

```
C source → compiler-generated assembly → assembler → object bytes
         → linker → ELF executable → loader → memory → CPU fetch
```

Part XI — Build and challenge a fictional ISA

Use:

`01-foundations-to-cpu/challenges/day-006-fictional-isa.py`

It implements an encoder and decoder for **Cedar-16**, a deliberately tiny 16-bit teaching ISA. It does not emulate execution; its purpose is to make encoding choices inspectable.

The formats

```
R: [ opcode:4 | rd:3  | rs1:3 | rs2:3 | 000:3 ]
I: [ opcode:4 | rd:3  | rs1:3 | signed immediate:6 ]
B: [ opcode:4 | rs1:3 | rs2:3 | signed PC offset:6 ]
J: [ opcode:4 | signed PC offset:12 ]
```

`LOAD` and `STORE` use I-shaped fields. In a store, the field occupying `rd` names a source register rather than a destination. Offsets are counted in instructions for this fictional design.

Explore the tool

```
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py table

python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py \
  encode add r1 r2 r3

python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py \
  encode addi r1 r2 -5

python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py decode 0x1298

python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py repl
```

For each encoding, cover the hexadecimal output and predict it by hand first. Then split the 16-bit result into fields and explain every bit.

Required encoding exercise

Encode these without the tool, then verify:

```
add  r5 r2 r7
sub  r1 r1 r3
addi r4 r0 -12
ld   r2 r6 20
st   r3 r6 -8
beq  r1 r0 -4
jmp  100
halt
```

Now decode:

```
0x1688
0x5a7f
0x74b8
0x8203
0x9ffc
0xa123
0xf001
```

Some patterns may be invalid or non-canonical. A robust decoder must not pretend every 16-bit word denotes a normal instruction.

Test the boundaries

Predict whether each command succeeds:

```
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py encode addi r1 r2 31
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py encode addi r1 r2 32
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py encode addi r1 r2 -32
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py encode addi r1 r2 -33
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py encode jmp 2047
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py encode jmp 2048
```

Explain the limits from field width and two's-complement representation, without memorizing the numbers.

Modify one design choice

Choose exactly one:

- expand from 8 to 16 registers;
- expand `ADDI` to a 9-bit signed immediate;
- make all branch offsets byte-based;
- add a `MUL` instruction;
- add byte and word loads alongside the current load;
- add a 32-bit “long instruction” format.

Before editing the script, draw the revised format. State what capability is gained and what is lost. If your fields no longer fit 16 bits, that is not a small implementation inconvenience—it is the design constraint becoming visible.

Part XII — Write the one-page ISA design note

Write one page titled **Cedar-16 ISA Design Note**. This is part of the build evidence, not optional documentation after “the real coding.”

Include these six compact sections:

1. Purpose and scope

What kind of machine is Cedar-16 for? Teaching, a tiny controller, or something else? What is deliberately omitted?

2. Architectural state

State:

- instruction width;
- register count and width;
- program-counter behavior;
- byte- or word-addressed memory;
- any special meaning for `r0`;
- visible flags, if any.

Do not invent answers silently. The current encoder defines fields but leaves some execution semantics for you to design.

3. Instruction formats and table

Draw every format and list opcode, mnemonic, operands, and one-sentence semantics. Specify which bit patterns are reserved.

4. Addressing and control flow

Define:

- load/store address calculation;
- immediate sign extension;
- branch base address;
- branch/jump offset units;
- alignment behavior.

“PC-relative” is incomplete until the PC base and offset scaling are defined.

5. Exceptions and invalid encodings

What happens on:

- a reserved opcode;
- nonzero reserved bits;
- misaligned memory access;
- arithmetic overflow;
- an out-of-range address?

An ISA contract must define failures as carefully as successful operations.

6. Tradeoff and revision

Describe one choice you would change in a second version, including:

```
benefit gained → encoding/implementation/software cost → compatibility effect
```

This final section is where you demonstrate that an ISA is a set of connected tradeoffs, not a mnemonic list.

Part XIII — Mastery evidence

Explain

Without notes, answer:

> Why can two CPUs implement the same ISA differently?

A complete answer should reconstruct this chain:

1. The ISA defines software-visible encodings, state, operations, and results.
2. It does not prescribe every internal circuit or timing choice.

3. Different implementations can use different pipelines, caches, predictors, execution units, scheduling, and internal operations.
4. Each must preserve the required architectural behavior for supported ISA features.
5. Therefore compatible binaries can run on physically different processors, while performance, power use, and microarchitectural side effects differ.

Do not stop at “because ISA is software and microarchitecture is hardware.” The ISA is not software; it is a specification contract implemented by hardware or sometimes by a software model.

Also explain:

- opcode versus operand;
- register versus memory versus immediate operand;
- fixed- versus variable-length encoding tradeoffs;
- why instruction count does not directly equal cycles;
- ISA versus ABI;
- where the specification, bytes, and decoder each exist.

Draw

On one page, draw:

```
ISA manual/specification
        ↓ guides
assembler/decoder/compiler implementations
        ↓                       ↓
ELF .text bytes → memory → fetch → decode → internal work
                                      ↓
                          architectural state transition
```

Add two different CPU implementations beneath the same ISA specification. Give each different pipeline/cache/internal designs but the same architectural result.

Observe

Record:

- ELF `Machine` and `.text` information from `readelf`;
- one raw x86-64 instruction byte sequence and its decoded form;
- one meaningful GCC-versus-Clang or `-O0`-versus-`-O2` difference;
- one source-level `if` or `switch` and the actual generated control sequence;
- one place where assembly syntax could be mistaken for machine bytes.

Build

Complete:

- the Cedar-16 hand encoding and decoding set;
- boundary/error tests;
- one deliberate ISA modification;
- the one-page ISA design note.

Why?

Add short answers to the notebook:

1. Why is an ISA a contract rather than a physical component?
2. Why can a more expressive instruction require a more complex encoding?
3. Why does a load/store ISA make memory traffic explicit?
4. Why can a variable-length ISA improve density but complicate decode?
5. Why is an immediate part of an instruction rather than a register?
6. Why is an ABI not the same thing as an ISA?
7. Why can one x86 instruction become several internal operations?
8. Why can two correct compilers emit different byte sequences?
9. Why is an invalid encoding part of ISA design?
10. Where, exactly, did you observe machine code today?

Mental model at the end of Day 6

```
software needs stable operations
        ↓
ISA specifies visible state, encodings, behavior, and exceptions
        ↓
assembler/compiler selects instruction encodings
        ↓
ELF .text stores machine-code bytes
        ↓
loader maps bytes into a process
        ↓
CPU fetches and decodes according to its implementation
        ↓
microarchitecture performs internal work
        ↓
architectural state changes as the ISA requires
```

The essential sentence is:

> Two CPUs can run the same ISA program because they honor the same architectural contract; they can differ internally because the contract specifies required visible behavior, not one mandatory pipeline, cache, or circuit layout.

References used selectively

Official architecture specifications and current documentation:

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, current manuals landing page (version 092 at time of consultation), with Volume 2 covering instruction formats and the instruction reference: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2*, Chapter 2, “Instruction Format,” and §3.1.1, “Instruction Format”: https://cdrdv2-public.intel.com/819716/325383-sdm-vol-2abcd.pdf
- AMD, *AMD64 Architecture Programmer's Manual, Volume 3: General-Purpose and System Instructions*, publication 24594, especially Chapter 1 “AMD64 Instruction Encoding” and Chapter 2 “Instruction Overview.” AMD's current processor documentation identifies publication 24594 as the general-purpose/system instruction manual: https://docs.amd.com/api/khub/documents/HwPCwoUzi4L8X5f_1Khe9A/content
- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, current ratified specification index, especially “RV32I Base Integer Instruction Set,” “RV64I Base Integer Instruction Set,” and the compressed-instruction extension: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html

Local companion reading:

- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 3, §§3.1–3.4, “A Historical Perspective” through “Accessing Information,” book pages 165–211. Local catalog entry: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 4, §§4.1–4.3 on the von Neumann model and instruction processing, and Chapter 5, §§5.1–5.3 on the LC-3 ISA and instruction set, book pages 81–150. Local catalog entry: `references/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf`

Next bridge: once instructions can encode arithmetic, memory access, and next-PC choices, the CPU can form branches and loops. That raises a deeper question: how does normal execution stop for exceptions, interrupts, and privileged operating-system control?

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Fictional Isa

Format: Python lab

What you will do: Encode and decode the tiny 16-bit Cedar-16 teaching ISA.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 01-foundations-to-cpu/challenges/day-006-fictional-isa.py table
```

- View lab source on GitHub

Put it into practice — Machine Code Lab

Format: C lab

What you will do: Day 6 machine-code lab

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 01-foundations-to-cpu/challenges/day-006-machine-code-lab.c -lm -o /tmp/day-006-machine-code-lab && /tmp/day-006-machine-code-lab
```

- View lab source on GitHub

Day 7 — Control Flow, Exceptions, and Privilege

Target time: approximately 3 hours

Rhythm: 75 minutes concept and tracing · 75 minutes lab · 30 minutes diagram, explanation, and review

> The CPU is executing your loop. A timer device asks for attention. At nearly the same moment, your process executes a system call.
>
> Who is allowed to change what runs next—and how can the machine enter the kernel without trusting the program to choose an arbitrary kernel address?

That is today's problem. A useful computer must do more than calculate the next sequential result. It must choose, repeat, call, react, and cross a protection boundary under controlled conditions.

The path is:

```
compare values
    → record condition state
    → choose the next PC
    → build loops and calls
    → interrupt ordinary control flow
    → enter a trusted handler through a hardware-defined gate
    → restore the interrupted context
```

By the end, you should be able to explain how the CPU stops normal execution and safely runs an operating-system handler. “The kernel takes over” is not enough. You should be able to name what causes the transfer, how the target is chosen, what state must be saved, why privilege changes, and how execution returns.

---

1. The historical problem: useful machines cannot only march forward

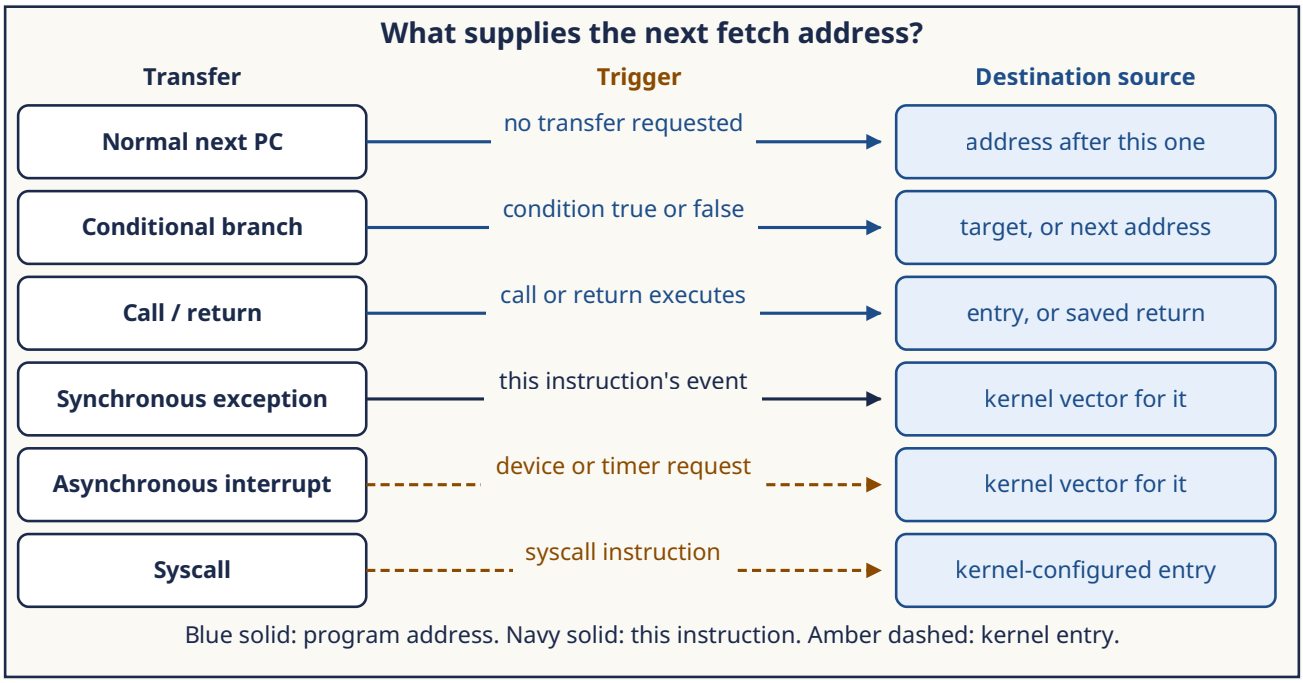
An early automatic calculator could perform a predetermined sequence of operations. That is already valuable, but consider long division: subtract a divisor repeatedly **while** the remainder is large enough. Or consider a payroll calculation: choose one formula **if** a condition holds and another otherwise.

The missing capability is not another arithmetic operation. It is the power to choose the next operation from a result already computed.

A stored-program CPU normally keeps the address of the next instruction in a **program counter**—`RIP` in x86-64 terminology. If every instruction merely advanced that address, the program would be a straight line:

```
PC = 0x1000  execute
PC = 0x1004  execute
PC = 0x1008  execute
...
```

Control flow appears when an instruction can replace the ordinary next PC:



Each path names the trigger that selects it and the source of the address that will be fetched next.

These mechanisms differ in purpose and authority, but all answer the same physical question: **which instruction address will be presented for fetch next?**

---

2. Compare, flags, and conditional branches

Suppose C says:

```
if (x < limit)
    ++x;
```

At the machine level there is usually no little object named `if`. The compiler emits operations that establish a condition and then conditionally change control flow.

On x86-64, `cmp source, destination` conceptually computes:

```
destination - source
```

It discards the arithmetic result but updates selected bits in `RFLAGS`. Important examples are:

- `ZF`, zero flag: the conceptual result was zero;
- `SF`, sign flag: the high result bit was one;
- `CF`, carry flag: unsigned subtraction required a borrow;
- `OF`, overflow flag: signed arithmetic overflowed.

A following `jcc` instruction tests a condition derived from those flags. For example, in AT&T syntax:

```
cmp  %esi, %edi       # conceptual calculation: edi - esi
jl   smaller_signed   # signed less-than: SF differs from OF
jb   smaller_unsigned # unsigned below: CF is set
je   equal            # ZF is set
```

Signed and unsigned comparisons use the **same bits** but ask different questions. That is why `jl` and `jb` are different. `SF` alone cannot reliably mean signed less-than because signed overflow can invert the apparent sign; the signed condition accounts for `OF`.

Some ISAs expose condition-code flags; some use compare-and-branch instructions or write comparison results into general registers. Do not turn the x86 arrangement into a universal law. The universal idea is that the machine derives a Boolean condition and uses it to select the next PC.

A branch is a PC multiplexer in architectural form

At a simple conceptual level:

```
                          ┌─ sequential next address
condition ──controls──────┤
                          └─ branch target
                                   │
                                   ▼
                               next PC
```

Real processors predict branches and may fetch or execute speculatively. Those mechanisms improve performance; they do not change the architectural promise. When the instruction retires, the visible state must match the path the ISA says was taken.

---

3. A loop is a backward conditional transfer

Consider:

```
int sum_to(int n)
{
    int sum = 0;
    for (int i = 0; i < n; ++i)
        sum += i;
    return sum;
}
```

One possible unoptimized shape is:

```
    sum = 0
    i = 0
test:
    compare i with n
    branch to done if i >= n
body:
    sum = sum + i
    i = i + 1
    jump back to test
done:
    return sum
```

Nothing in hardware recognizes the source-language phrase “for loop.” The loop exists as:

1. state holding `i`, `sum`, and `n`;
2. arithmetic that updates that state;
3. a comparison;
4. a conditional branch to leave;
5. a backward control transfer to repeat.

Trace three iterations by hand

For `n = 3`:

test	i	sum before body	i < n	next PC choice
1	0	0	true	body
2	1	0	true	body
3	2	1	true	body
4	3	3	false	done

The source loop feels like one repeating object. The CPU sees a sequence of individual instructions and repeatedly selects an earlier instruction address.

Calls and returns: a preview

A direct `call` also changes `RIP`, but first preserves a return address. On x86-64 a near `call` pushes the address after the call onto the current stack, then transfers to the callee. `ret` obtains the saved address and places it back into `RIP`.

```
caller: call f
            │
            ├── save address of "after call"
            └── RIP ← entry of f

f:      ... work ...
        ret ──────────→ RIP ← saved address
```

This is a preview, not yet a complete ABI lesson. Arguments, preserved registers, stack alignment, frames, and security hardening come later. For today, notice the contrast:

- branches choose among program-specified targets;
- calls preserve an ordinary program return point;

- exception entry uses targets and state rules configured for protected handling, not a return address supplied freely by untrusted user code.

---

4. Why privilege exists

Imagine every process could:

- program the disk controller;
- disable interrupts forever;
- rewrite another process's page tables;
- install its own exception target;
- read all physical memory.

One bad pointer or hostile program could destroy every protection the operating system claims to provide. Multiprogramming therefore needs more than a convention saying “applications should behave.” The processor must enforce authority.

Modern architectures provide execution modes. In ordinary x86-64 Linux:

```
ring 3 / CPL 3   user programs: restricted
ring 0 / CPL 0   kernel: privileged
```

x86 defines four rings, numbered 0 through 3, but mainstream Linux primarily uses ring 0 and ring 3. “Ring 1 and ring 2 must contain drivers” is not a rule of x86-64 Linux. Other architectures use different language and structures: AArch64 commonly runs applications at EL0 and the kernel at EL1, with EL2 available for a hypervisor.

Privilege is CPU state, not a moral description of source code. While executing with insufficient privilege, certain instructions or accesses are blocked by hardware. Page-table permissions also distinguish user-accessible from supervisor-only mappings.

User versus kernel is more than two address ranges

Keep three related ideas separate:

1. **current privilege:** which operations the CPU permits now;
2. **address translation and permissions:** which memory mappings this execution may access;
3. **software policy:** what the kernel chooses to allow after validating a request.

Changing privilege does not automatically grant an application permanent kernel authority. On a system call, the CPU begins executing a kernel entry point under kernel control. The kernel validates arguments and performs a specific service, then returns to restricted execution.

---

5. Four ways normal sequencing can change

The word “exception” is overloaded across architecture manuals, operating systems, programming languages, and conversation. Start with timing.

Synchronous exception

A synchronous event is caused by, and associated with, the instruction being executed. Repeat the same architectural state and instruction, and the event is tied to that execution.

Examples:

- invalid opcode;
- divide error;
- page fault;
- general-protection fault;
- a deliberately executed system-call instruction.

Asynchronous interrupt

An asynchronous interrupt originates outside the current instruction stream. A timer expires, a network device completes work, or an interrupt controller delivers a pending request. The CPU normally recognizes it at an architecturally defined boundary between instructions.

```
exception:  "this instruction encountered a condition"
interrupt:  "an external agent needs attention"
```

The interrupted instruction did not cause the network packet to arrive. That distinction matters even though both mechanisms eventually vector into privileged software.

x86's trap, fault, and abort classification

The Intel architecture classifies **exceptions** by where the saved instruction pointer points and whether restart is meaningful:

- **fault:** reported before the instruction completes; saved state generally allows the handler to correct the cause and restart the instruction;
- **trap:** reported after the instruction completes; saved instruction pointer generally names the next instruction;
- **abort:** severe condition for which the precise instruction location may not be reliably reported and restart is not generally possible.

A page fault is the classic fault: Linux may install a missing mapping and retry the instruction. A debug single-step event is a classic trap. Machine check conditions can be abort-class events.

Be careful with the word **trap**. Some textbooks use “trap” broadly for any synchronous transfer, some operating systems call their saved frame a trap frame, and architecture manuals classify specific events differently. ARM documentation uses its own exception terminology. State which vocabulary you mean instead of assuming one universal taxonomy.

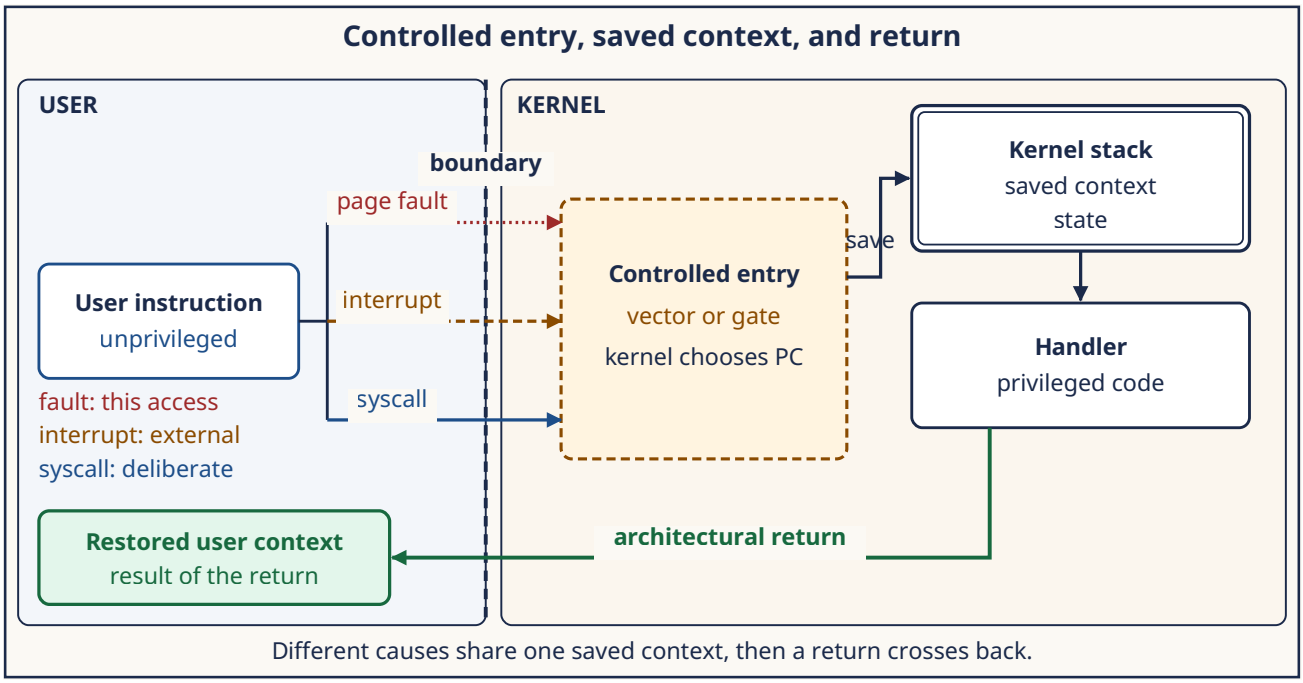
Also do not confuse a software `raise(SIGILL)` with a hardware invalid-opcode exception. `raise` asks the operating system to deliver a Unix signal; it does not execute an invalid instruction.

---

6. Vectoring: a controlled answer to “where next?”

If user code could respond to an exception by naming any ring-0 address and stack, protection would disappear. The hardware and kernel therefore prearrange legal entry paths.

On x86, exceptions and interrupts have vector numbers. The kernel installs descriptor information in an Interrupt Descriptor Table (IDT). Conceptually:



A page fault, an interrupt, and a syscall cross the user/kernel boundary for different causes, then share a saved-context entry before the return.

Exact saved fields depend on the event and transition. On x86, hardware entry preserves enough control state for the architectural return; entry assembly saves additional general registers into a kernel-defined frame. Some exceptions push an error code and others do not. Do not simplify this to “the CPU saves every register.”

Why a kernel stack?

The current user stack belongs to untrusted process state. It may be full, unmapped, maliciously positioned, or writable by user code. On entry from user mode, the kernel needs a trusted stack on which to preserve context and run nested C/assembly call sequences. Linux associates kernel execution state with the current task and enters on kernel-controlled stack storage.

The kernel stack is not “where the entire process is stored.” It is working space for privileged execution, including saved register/context data and kernel call frames.

Return from exception

After handling, architecture-specific return machinery restores control state and lowers privilege as appropriate. On x86 this family includes `iret`/`iretq`; fast syscall paths commonly use `sysret` when conditions allow. The kernel must validate and arrange the return state—it does not blindly trust arbitrary values merely because they came from user memory.

7. Concrete cases

Invalid opcode: `#UD` can lead to `SIGILL`

Suppose user code attempts bytes that do not encode a valid instruction for the current mode.

```
decoder rejects instruction
    → x86 invalid-opcode exception #UD (vector 6)
    → CPU enters kernel exception path
    → Linux decides the user task cannot continue
    → Linux delivers SIGILL
    → signal disposition handles it or terminates the process
```

`#UD` is a CPU exception. `SIGILL` is a Unix process signal chosen by the OS as the user-visible consequence. They are not the same object and the mapping is OS policy, not an intrinsic wire from the CPU to a C signal handler.

The accompanying lab uses `raise(SIGILL)` by default. That safely exercises signal delivery and handler constraints without embedding raw invalid opcode bytes. Therefore it begins at the signal-delivery part of this diagram; it does **not** claim to generate `#UD`.

Divide error: `#DE` and a possible `SIGFPE`

On x86, integer divide by zero or an unrepresentable quotient can produce the divide-error exception `#DE` (vector 0). Linux can translate a user-mode case into `SIGFPE`.

Do not write `1 / zero` in portable C and claim it is a reliable CPU experiment. Integer division by zero is undefined behavior in C: the compiler is not required to emit a hardware divide. The lab therefore uses `raise(SIGFPE)` as a safe signal-flow demonstration and labels it honestly. A hardware `#DE` experiment belongs in controlled architecture-specific assembly with its assumptions stated.

Protection example: `#GP`

A user-mode attempt to execute a privileged operation can cause a general-protection exception, `#GP` on x86. The hardware refuses the operation before letting ring-3 code alter protected machine state. Linux then decides how the user task should observe that failure; the resulting signal depends on the particular condition and kernel path.

Page fault: often normal

A page fault `#PF` does not mean “segmentation fault.” It means address translation or permission checking needs privileged attention.

```
#PF
 ├── valid VMA, lazily allocated page
 │       kernel maps a page → retry instruction → program notices nothing
 ├── copy-on-write mapping
 │       kernel creates private writable page → retry
 └── no valid mapping or forbidden access
         kernel cannot repair → often deliver SIGSEGV
```

Demand paging depends on a fault being restartable. The same architectural mechanism can be routine or fatal; the kernel's memory policy and process mapping decide which.

Hardware interrupt flow

Take a network receive interrupt:

```
1. NIC receives data and records device state.
2. NIC/MSI path notifies the interrupt controller/CPU.
3. CPU recognizes a pending, unmasked interrupt at an allowed boundary.
4. Vectoring saves return state and enters a kernel interrupt entry point.
5. Entry code saves additional context on a kernel stack.
6. Driver/kernel handler acknowledges and services or schedules work.
7. Return-from-interrupt restores the interrupted context.
8. The CPU resumes the selected task—or scheduling may choose another task.
```

The handler did not arrive by calling a normal C function from user code. Hardware established a protected control transfer. Likewise, interrupt return is not an ordinary `ret`: it restores privilege and machine state associated with the interrupted context.

---

8. System-call privilege transition

A system call is a deliberate request, but it still needs a controlled boundary crossing.

For a typical x86-64 Linux system call:

```
user code / libc wrapper
    | place syscall number and arguments per ABI
    ▼
SYSCALL instruction
    | CPU loads configured kernel entry state and changes privilege
    ▼
architecture entry assembly
    | establish safe kernel context; save registers
    ▼
Linux syscall dispatch
    | validate number, pointers, lengths, permissions
    ▼
requested kernel service
    | result or -errno
    ▼
kernel exit path
    | restore user-visible state
    ▼
SYSRET or another safe return path
    ▼
user mode resumes
```

This is not a conventional function call:

- a normal call stays at the same privilege and uses a program-chosen target;
- a syscall uses CPU entry configuration controlled by the kernel;
- user pointers remain untrusted inputs after entry;
- the kernel may block, schedule, deliver a signal, or return an error before the original thread resumes.

The x86-64 `SYSCALL` mechanism is also not the same entry mechanism as every IDT exception, even though both produce exceptional control flow and a privilege transition. Keep the common concept—controlled entry—separate from the exact architecture-specific path.

Architecture lens — x86-64, AArch64, and RISC-V

- x86-64 Linux normally uses `syscall`, CPL 3/0, and IDT-based exception vectors; AArch64 uses `svc`, EL0/EL1, vector tables selected through `VBAR_ELx`, and `eret`; RISC-V uses `ecall`, U/S privilege modes, `stvec` plus trap CSRs such as `sepc` and `scause`, and `sret`.
- These are architectural entry and privilege mechanisms. The syscall number, argument registers, and error-return convention form an OS ABI; Linux entry code, saved-frame layout, scheduling decisions, and the chosen external interrupt controller are implementation and platform details.

All three provide controlled entry rather than an unrestricted jump into privileged code, but their saved state and return paths are not interchangeable. See Architecture Comparison.

---

9. Observe: compile and trace a loop at instruction level

Use the provided lab:

```
cd /home/jd/Desktop/masterclass

gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O0 -g \
  01-foundations-to-cpu/challenges/day-007-exception-flow-lab.c \
  -o /tmp/exception-flow-lab

/tmp/exception-flow-lab normal
```

`-O0` keeps the first trace easy to relate to source. It does not promise one fixed assembly layout; compiler version, ABI, and options matter.

Disassemble before debugging

```
objdump -d -Mintel /tmp/exception-flow-lab | less
```

Search inside `less` for `<run_normal_loop>`. Identify:

1. initialization of `sum` and `i`;
2. the compare instruction;
3. the conditional branch controlling exit/repetition;
4. the instruction address targeted by the backward edge.

Then compare optimization:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -g \
  01-foundations-to-cpu/challenges/day-007-exception-flow-lab.c \
  -o /tmp/exception-flow-lab-O2
objdump -d -Mintel /tmp/exception-flow-lab-O2 | less
```

Do not ask only “where did my `for` statement go?” Ask which observable behavior the compiler had to preserve and which source-level structure it was free to rearrange.

Single-step instructions in GDB

```
gdb -q /tmp/exception-flow-lab
```

At the GDB prompt:

```
set disassembly-flavor intel
break run_normal_loop
run normal
display/i $pc
display/x $eflags
info registers rip eflags
disassemble /m run_normal_loop
si
si
```

Continue with `si` through one compare and branch. Immediately before and after the compare, run:

```
p/x $eflags
info registers
x/6i $pc
```

Record:

- `RIP` before the compare;
- the relevant values being compared;
- flags after the compare;
- `RIP` after the branch;
- whether the target or fall-through address became the next PC.

Use `ni` if you want to step over a call such as `printf` rather than entering libc instruction by instruction.

---

10. Build and observe the exception-flow lab

The actions are selectable:

```
/tmp/exception-flow-lab normal
/tmp/exception-flow-lab ill
/tmp/exception-flow-lab segv
/tmp/exception-flow-lab fpe
/tmp/exception-flow-lab all
```

The signal actions run in forked children. Their handlers use `write(2)` and `_exit(2)`, both suitable for this signal-handler path; they do not call `printf`, `malloc`, or `exit`. The parent waits, reports status, and continues, so the experiment does not crash the shell or the parent lab process.

The `segv` and `fpe` choices deliberately call `raise`. They demonstrate Unix signal flow safely, not real page-fault or divide-error generation. This distinction is part of the exercise.

Trace process and signal behavior

```
strace -f -e trace=process,signal /tmp/exception-flow-lab all
```

Look for:

- `clone`/`fork`-related process creation;
- `rt_sigaction` installing dispositions;
- a signal-send operation used by `raise` (often `tgkill` on Linux);
- `SIGILL`, `SIGSEGV`, and `SIGFPE` delivery;
- child exit and parent `wait4`/wait behavior.

Then observe a real syscall transition at the user-visible boundary:

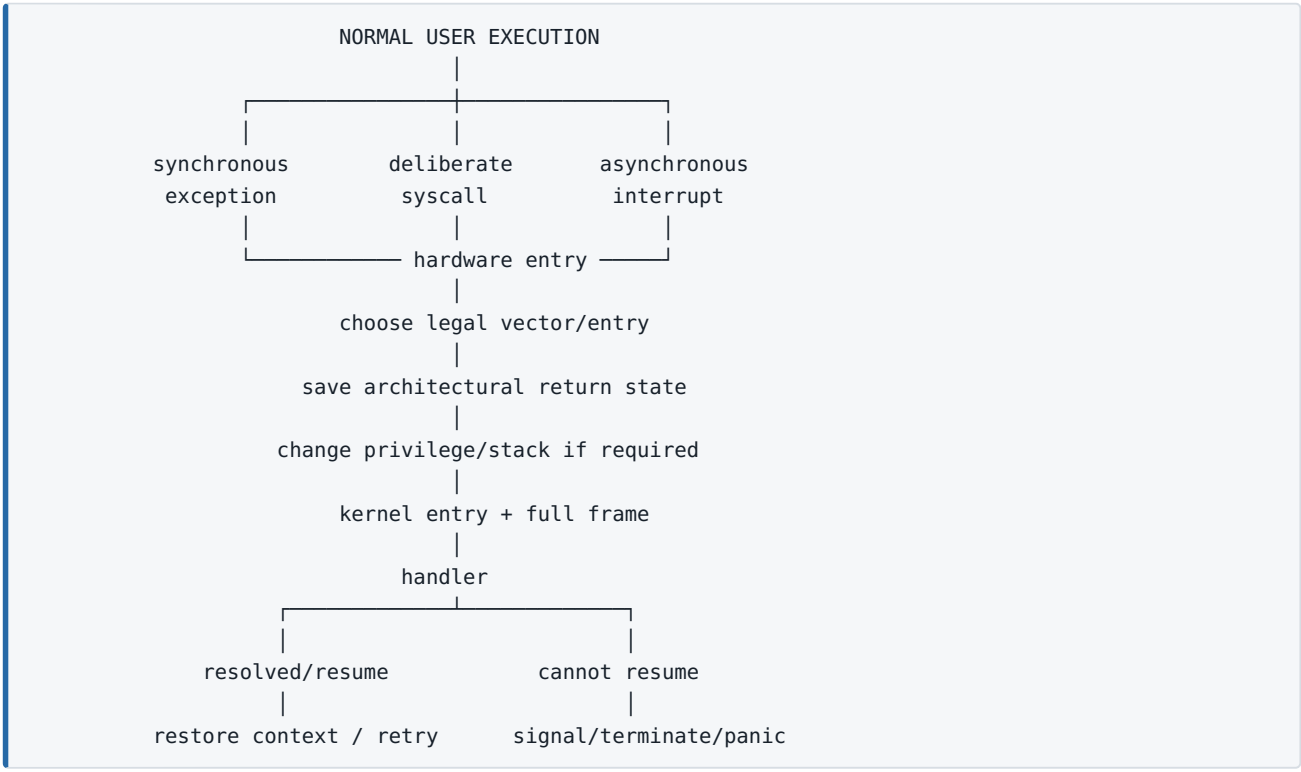
```
strace -e trace=write /tmp/exception-flow-lab normal
```

`strace` shows syscall entry/return from the tracer's viewpoint. It does not directly print every hidden CPU entry instruction, privilege bit change, or kernel-stack operation. Use the documented transition model to explain what must occur between the userspace call site and the returned syscall result.

---

11. Draw: exception-flow diagram

Redraw this without copying labels, then annotate one `#UD` path and one hardware-interrupt path:



Add these precision notes:

- asynchronous interrupts are not caused by the current instruction;
- not every exception changes privilege if it occurs while already in the kernel;
- not every page fault is fatal;
- a Unix signal is a software abstraction, not a CPU exception vector;
- the exact saved frame and return instruction are architecture-specific.

---

12. Mastery: Explain · Draw · Observe · Build · Why

Explain

Without notes, answer:

> How can the CPU stop normal execution and safely run an operating-system handler?

A complete explanation should connect:

1. event recognition;
2. vector or controlled entry selection;
3. protected privilege transition;
4. trusted kernel stack and saved context;
5. kernel entry code and handler;
6. handler decision: repair, resume, signal, schedule, or terminate;
7. validated return-from-exception restoring the prior context.

Draw

Draw both:

- compare → flags/condition → next-PC choice → loop;
- user execution → event → vector → saved context/kernel stack → handler → return.

Observe

Collect:

- one GDB trace showing the PC taking or not taking a loop branch;
- one `objdump` excerpt identifying the backward edge;
- one `strace -f` observation showing child signal delivery and parent survival.

Build

Extend only after understanding the supplied program:

1. add a `usr1` action for `SIGUSR1`;
2. keep the handler async-signal-safe;
3. run it in a child;
4. predict the trace before running;
5. explain why this is still a Unix-signal experiment, not a CPU exception.

Why?

Write short answers:

1. Why can a conditional branch be described as choosing a new PC?
2. Why do signed and unsigned branches inspect conditions differently?
3. Why is a user stack not sufficient as the only stack for kernel entry?
4. Why can a page fault be a successful part of normal execution?
5. Why is `#UD → Linux → SIGILL` not the same as `raise(SIGILL)`?
6. Why must the return path restore privilege as well as an instruction address?

Mental model

```
sequential execution
    + conditional PC selection
        = decisions and loops

ordinary calls
    + saved program return address
        = reusable procedures

hardware event recognition
    + protected vectoring
    + trusted saved context and kernel stack
    + privileged handler
    + architectural return
        = safe exceptional control flow
```

The sentence to retain is:

> The CPU does not let an event merely “jump into the kernel.” It recognizes a defined event, selects a kernel-configured entry path, preserves return state, enforces privilege and stack rules, runs trusted entry/handler code, and returns only through an architecture-defined restoration path.

References used selectively

Primary architecture and Linux references:

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Volume 3A, Chapter 6 (“Interrupt and Exception Handling”) and Chapter 7 (“Task Management”), especially the exception classes, IDT, privilege transitions, stacks, and `IRET`; Volume 2 for `CMP`, conditional jumps, `SYSCALL`, `SYSRET`, and `IRET` instruction semantics: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- Linux kernel documentation, “Kernel Entries,” including x86 syscall and interrupt/exception entry categories: https://docs.kernel.org/entry/entry.html
- Linux x86-64 syscall entry implementation, `arch/x86/entry/entry_64.S`: https://github.com/torvalds/linux/blob/master/arch/x86/entry/entry_64.S

- Linux man-pages: `sigaction(2)`, `signal(7)`, `signal-safety(7)`, `raise(3)`, `fork(2)`, and `waitpid(2)`: https://man7.org/linux/man-pages/
- `strace(1)` manual: https://man7.org/linux/man-pages/man1/strace.1.html

Local reading:

- Kaiwan N. Billimoria, *The Linux Operating System: A Brief on Its Architecture*, "Execution Privilege Levels," "Arch-specific issuing of system calls," and "Flow of a Process," local PDF pages 16–22: `references/library/training/kaiwantech/kernel-internals-2024/01-linux-system-architecture.pdf`
- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 3 on machine-level control and Chapter 8, §§8.1–8.5 on exceptional control flow, exceptions, processes, and signals: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 4 on control instructions and Chapters 8–9 on I/O, interrupts, privilege, and operating-system service: `references/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf`

Next bridge: x86-64 assembly foundations will make registers, addressing, instruction encodings, and debugger traces more precise; stack frames and calling conventions will then complete the ordinary `call`/`ret` story.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Exception Flow Lab

Format: C lab

What you will do: Use this exception flow lab exercise to test the section's model.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 01-foundations-to-cpu/challenges/day-007-exception-flow-lab.c -lm -o /tmp/day-007-exception-flow-lab && /tmp/day-007-exception-flow-lab all
```

- View lab source on GitHub